

+

Informatik 2 – Projekt 1 & Vektor Klasse

K

A



©image: various

Heute

- Projekt 1

C++

- Vector Klasse
- Wiederholung:
 - dynamische Speicherallokation
 - rekursive Baumsuche/-ausgabe

Projekt #1



Bearbeitungszeit:

05.11.25 - 19.11.25

Gruppenarbeit: max. 4 Mitglieder/Gruppe.

mind. zwei Laptops (Win oder Linux) pro Gruppe (für Projekt #2)

Abgabe:

19.11.25 bis 23:50Uhr

Jede Gruppe übermittelt den C++ Quellcode (1x pro Gruppe) via ILIAS.

Testat:

In den Übungen ab dem 20.11.25, werden Ihre Programme vor Ort geprüft & testiert.

Testate müssen vor Beginn des zweiten Projekts erfolgen.

Jedes Gruppenmitglied muss die Funktion des Source Code erklären können.

Das **Bestehen** der **beiden Projekte** ist **notwendige** und hinreichende Bedingung, um die Übungen zur Vorlesung Informatik 2 zu bestehen.

Projekt #1



Bearbeitungszeit:

05.11.25 - 19.11.25

Gruppenarbeit: max. 4 Mitglieder/Gruppe.

mind. zwei Laptops (Win oder Linux) pro Gruppe (für Projekt #2)

Abgabe:

19.11.25 bis 23:50Uhr

Jede Gruppe übermittelt den C++ Quellcode (1x pro Gruppe) via ILIAS.

Testat:

In den Übungen ab dem 20.11.25, werden Ihre Programme vor Ort geprüft & testiert.

Testate müssen vor Beginn des zweiten Projekts erfolgen.

Jedes Gruppenmitglied muss die Funktion des Source Code erklären können.

Das **Bestehen** der **beiden Projekte** ist **notwendige** und hinreichende Bedingung, um die Übungen zur Vorlesung Informatik 2 zu bestehen.

Projekt #1 – Abgabe des Source Codes



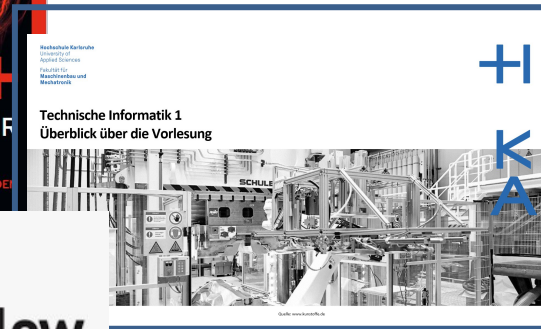
Fügen Sie bitte folgenden Kommentar in den Source-Code Ihre Projektarbeit vor der Abgabe via Ilias ein:

```
/*  
Projektarbeit 1 –   
  
Gruppe:  
[author #1]  
[author #2]  
[author #3]  
[author #5]  
  
Wir stimmen der Veröffentlichung unseres Source Code in anonymisierter Form zu.  
oder  
Wir stimmen der Veröffentlichung unseres Source Code mit Namensnennung zu.  
oder  
Wir stimmen der Veröffentlichung unseres Source Code *nicht* zu.  
  
Copyright (C) [year] [authors #1-#4]  
SPDX-License-Identifier: MIT  
*/
```

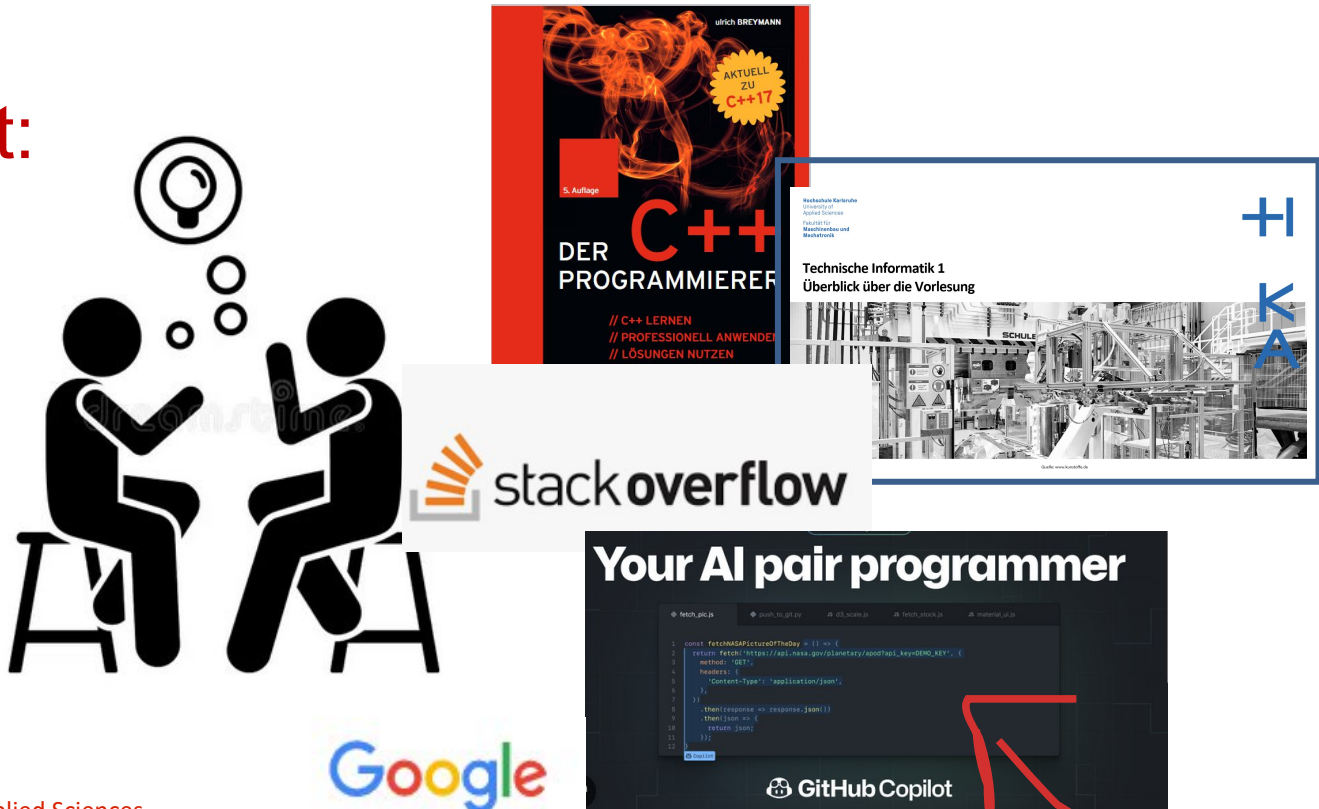
Projekt #1



Erlaubt:



Erlaubt:

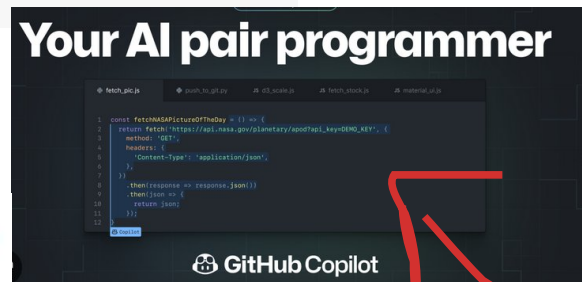
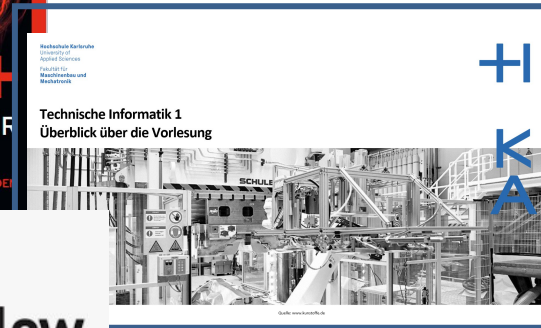


Nicht
empfohlen

Projekt #1



Erlaubt:



Nicht Erlaubt:

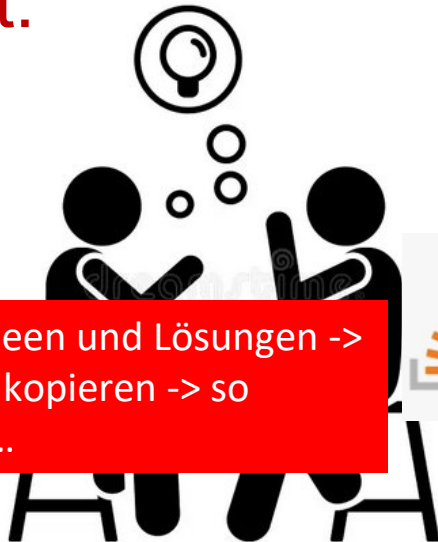
- Source Code zu kopieren!
- Vibe Coding

Nicht
empfohlen

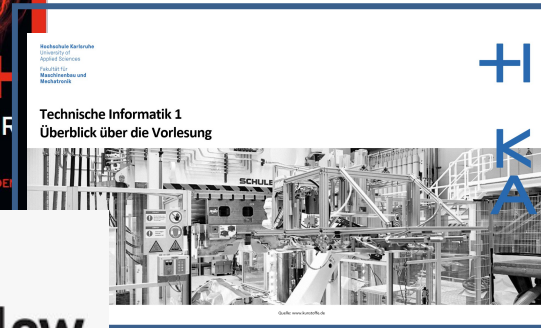
Projekt #1



Erlaubt:

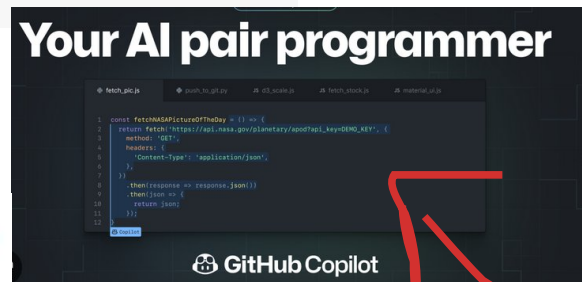


Diskutieren Sie Ideen und Lösungen -> nicht einfach nur kopieren -> so lernen Sie nichts...



Nicht Erlaubt:

- Source Code zu kopieren!
- Vibe Coding



Nicht empfohlen

Projekt #1



Karlsruhe University of Applied Sciences

Foto: dpa

<https://www.badische-zeitung.de/wie-funktioniert-ein-navigationssystem>





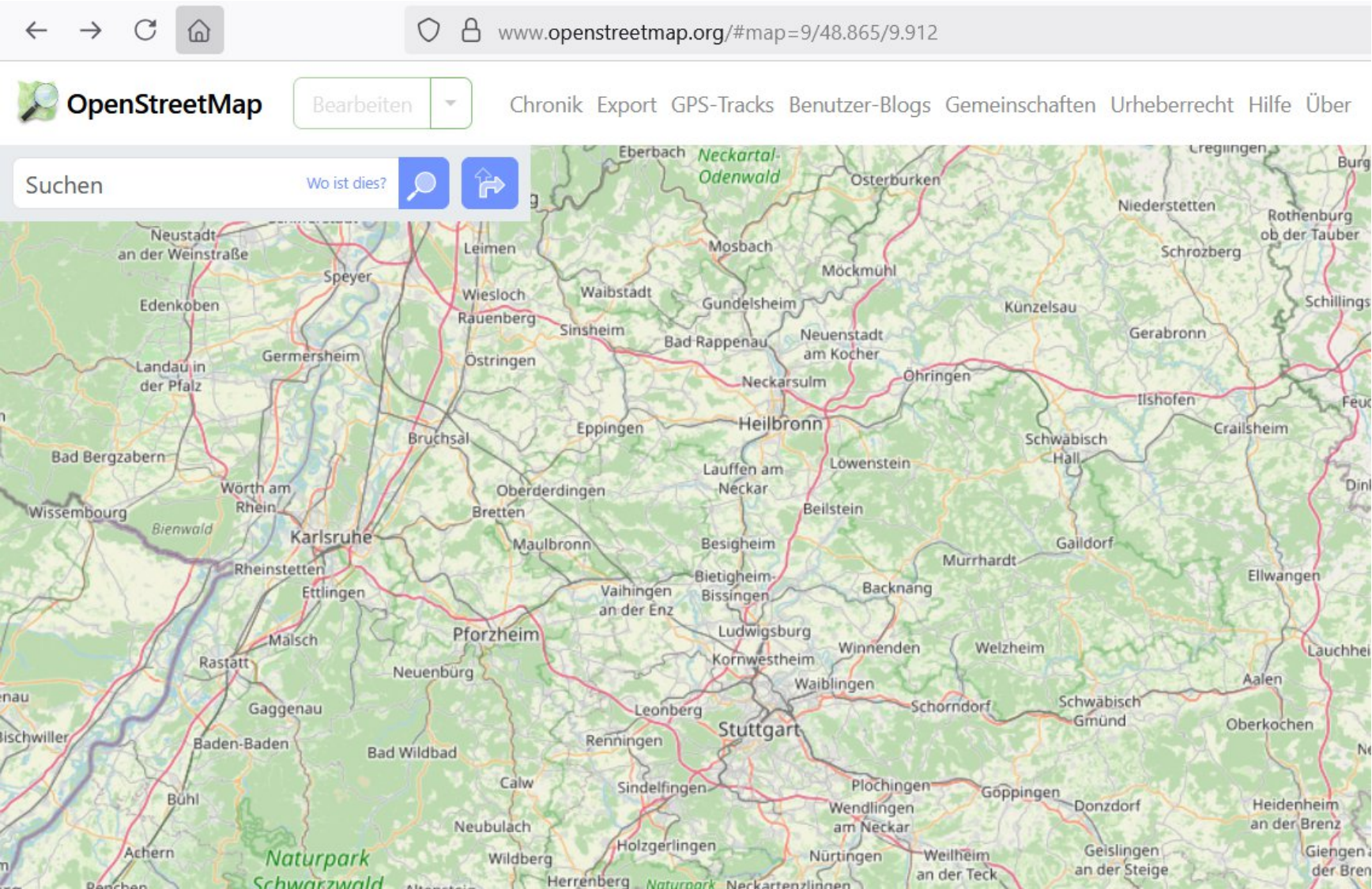
Nutzen Sie KI-Methoden, um auf einer Straßenkarte den kürzesten Pfad zwischen einem Start- und einem Zielpunkt zu finden.

Schreiben Sie hierzu ein C++ Programm, welches

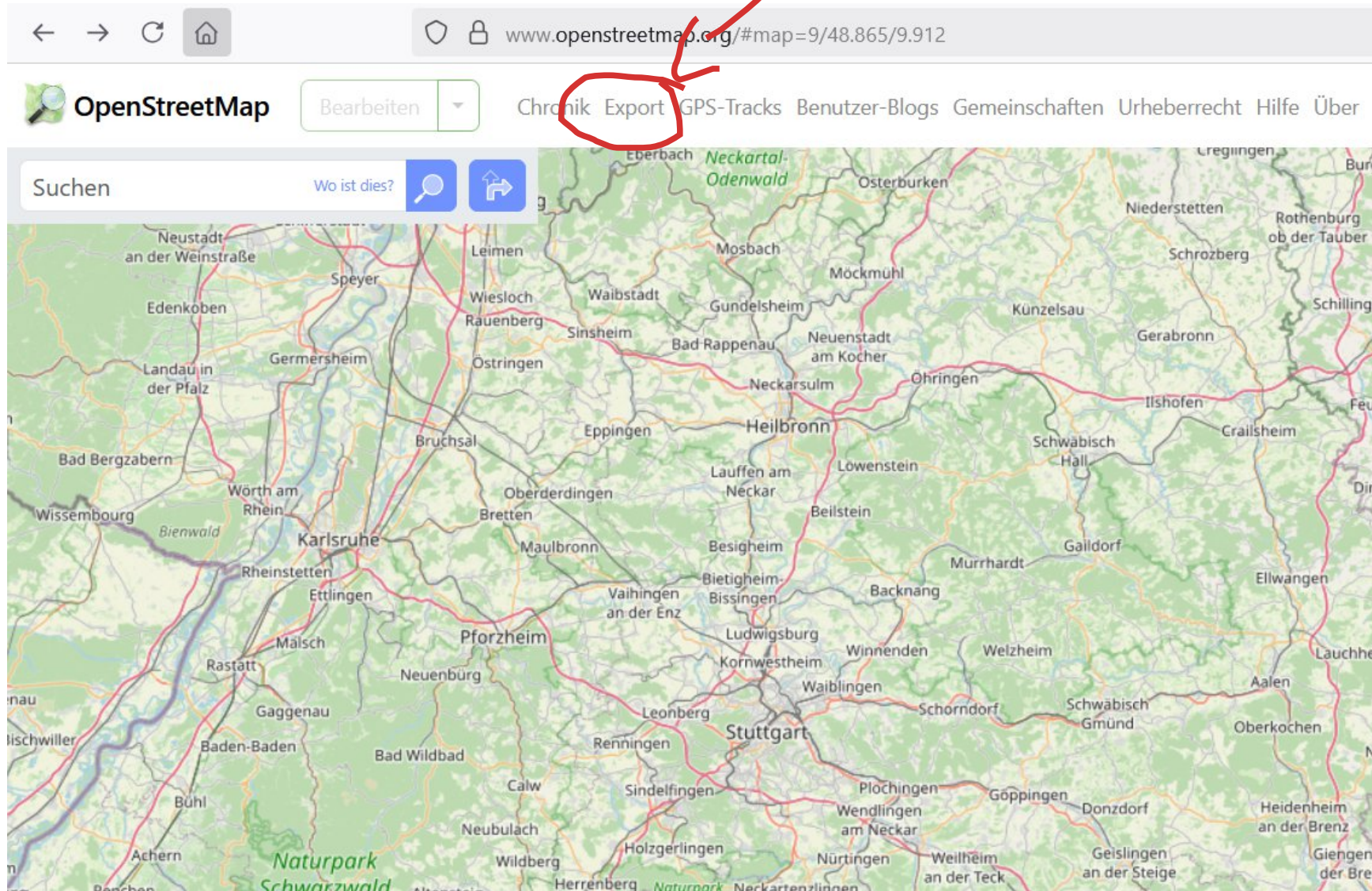
1. Open-Street Map Daten einliest und die Straßendaten in einem Graphen abspeichert
2. den kürzesten Weg zwischen einem Start- und einem Zielpunkt mit Hilfe des Dijkstra Algorithmus findet.
(Der Zielpunkt kann entweder beliebig vom Nutzer bestimmt oder auf die HKA festgelegt sein.)
3. die Karte und die Route mit Hilfe der SDL3 Bibliothek visualisiert.



1. Open Street Map



<https://www.openstreetmap.org/#map=9/48.865/9.912>



<https://www.openstreetmap.org/#map=9/48.865/9.912>

OpenStreetMap Bearbeiten Chronik Export GPS-Tracks Benutzer-Blogs Gemeinschaften Urheberrecht Hilfe Über Anmelden Registrieren

Suchen Wo ist dies?

Exportieren

49.01897
8.38250 8.40795
49.00946

[Einen anderen Bereich manuell auswählen](#)

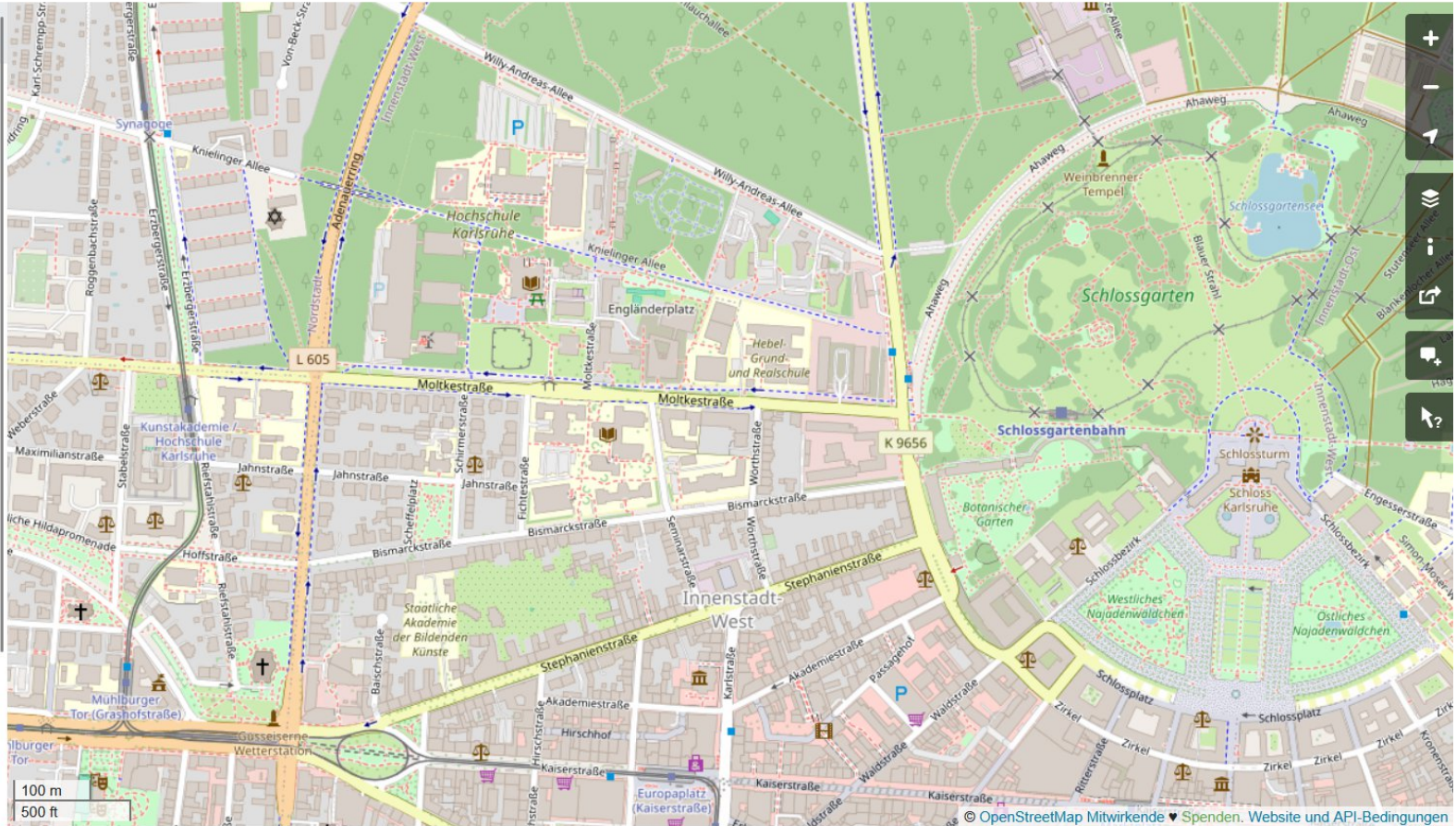
Lizenz

OpenStreetMap-Daten sind unter Open Data Commons Open Database-Lizenz (ODbL) lizenziert.

[Export](#)

Falls der obenstehende Export fehlschlägt, erwäge bitte, eine der unten aufgelisteten Quellen zu verwenden:

- Overpass API**
Diese Bounding Box von einem Mirror der OpenStreetMap-Datenbank herunterladen
- Planet OSM**
Regelmäßig aktualisierte Kopien der kompletten OpenStreetMap-Datenbank
- Geofabrik Downloads**



Lizenz:
OpenStreetMap-Daten sind unter Open Data Commons
Open Database-Lizenz (ODbL) lizenziert.

<https://www.openstreetmap.org/export#map=16/49.01421/8.39523>

- Alle Informationen in einer Textdatei
- Die .osm Datei ist XML-Formatiert
- Import Libraries wie z.B.
 - tinyxml
 - Osmiumexistieren, dürfen in dem Projekt aber nicht verwendet werden.

```
<?xml version="1.0" encoding="UTF-8"?>
<osm version="0.6" generator="openstreetmap-cgimap 2.1.0 (113195 spike-06.openstreetmap.org)" copyright="OpenStreetMap"
license="http://opendatacommons.org/licenses/odbl/1-0/">
  <bounds minlat="49.0136650" minlon="8.3860230" maxlat="49.0184220" maxlon="8.3987470"/>
  <node id="696702" visible="true" version="6" changeset="20540754" timestamp="2014-02-13T16:17:16Z" user="mapper999" />
  <node id="696703" visible="true" version="5" changeset="135531505" timestamp="2023-04-30T10:30:02Z" user="mapper999" />
  <node id="14795133" visible="true" version="11" changeset="20540754" timestamp="2014-02-13T17:16:31Z" user="mapper999" />
    <tag k="highway" v="traffic_signals"/>
  </node>
  <node id="14795418" visible="true" version="7" changeset="14828657" timestamp="2013-01-28T22:10:37Z" user="chewey" />
  <node id="14795723" visible="true" version="10" changeset="143497492" timestamp="2023-11-01T21:23:36Z" user="Scout J" />
  <node id="14795804" visible="true" version="9" changeset="151367222" timestamp="2024-05-15T15:32:35Z" user="Mittelobe" />
  <node id="14890793" visible="true" version="3" changeset="10665379" timestamp="2012-02-12T17:16:56Z" user="mapper999" />
  <node id="14890794" visible="true" version="4" changeset="134148116" timestamp="2023-03-26T18:48:17Z" user="mapper999" />
  <node id="14890795" visible="true" version="4" changeset="134148116" timestamp="2023-03-26T18:48:17Z" user="mapper999" />
  <node id="14959773" visible="true" version="7" changeset="134148116" timestamp="2023-03-26T18:48:17Z" user="mapper999" />
  <node id="14961080" visible="true" version="8" changeset="37348146" timestamp="2016-02-21T14:49:01Z" user="bjoern262" />
    <tag k="bus" v="yes"/>
    <tag k="name" v="Linkenheimer Tor"/>
    <tag k="network" v="Karlsruher Verkehrsverbund"/>
    <tag k="operator" v="Verkehrsbetriebe Karlsruhe GmbH"/>
    <tag k="public_transport" v="stop_position"/>
    <tag k="ref_name" v="Karlsruhe Linkenheimer Tor"/>
    <tag k="wheelchair" v="yes"/>
    <tag k="wheelchair:description" v="Busse haben eine ausfahrbare Rampe"/>
  </node>
  <node id="15357691" visible="true" version="16" changeset="164906044" timestamp="2025-04-13T17:38:02Z" user="EinEngel" />
    <tag k="crossing" v="uncontrolled"/>
    <tag k="crossing:island" v="yes"/>
    <tag k="crossing:markings" v="no"/>
    <tag k="crossing:signals" v="no"/>
    <tag k="highway" v="crossing"/>
    <tag k="tactile_paving" v="no"/>
```

OSM Dateiformat - Knoten



Ein Node (Knoten)

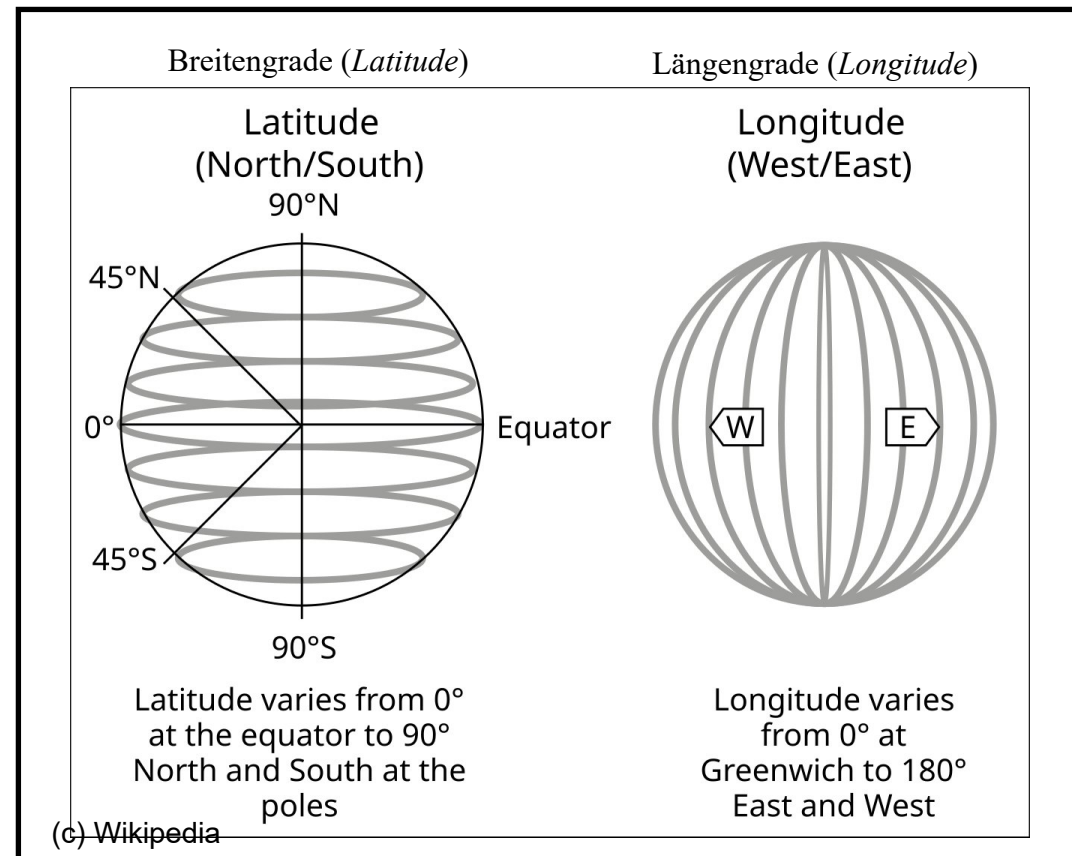
- ist das Grundelement für alles auf der Karte

```
<node id="11850282471" visible="true" version="1" changeset="150536041" timestamp="2024-04-26T12:13:26Z"
user="Nakaner" uid="496201" lat="49.0144049" lon="8.3954825"/>
```

OSM Dateiformat - Knoten

Ein Node (Knoten)

- ist das Grundelement für alles auf der Karte
- ist eine Position auf der Karte.
- hat eine ID, sowie Breiten- und Längengrad
- kann beispielsweise ein Punkt einer Straße oder ein Ort (z.B. Restaurant) sein



```
<node id="11850282471" visible="true" version="1" changeset="150536041" timestamp="2024-04-26T12:13:26Z"
  user="Nakaner" uid="496201" lat="49.0144049" lon="8.3954825">
```



Ein Node (Knoten)

- ist das Grundelement für alles auf der Karte
- ist eine Position auf der Karte.
- hat eine ID, sowie Breiten- und Längengrad
- kann beispielsweise ein Punkt einer Straße oder ein Ort (z.B. Restaurant) sein
- sind optional mit Tags versehen.
Tags beinhalten Informationen wie z.B. Straßentyp, Öffnungszeiten, etc.

```
<node id="643462941" visible="true" version="12" changeset="155336545" timest  
<tag k="amenity" v="library"/>  
<tag k="check_date:opening_hours" v="2024-08-16"/>  
<tag k="description" v="Abweichende Öffnungszeiten während Klausurphase"/>  
<tag k="internet_access" v="wlan"/>  
<tag k="level" v="1"/>  
<tag k="name" v="Fachbibliothek Hochschule Karlsruhe"/>  
<tag k="note" v="Außenstelle der KIT-Bibliothek"/>  
<tag k="opening_hours" v="Mo-Fr 08:00-21:00"/>  
<tag k="ref:isil" v="DE-90-4"/>  
<tag k="service_times" v="Mo-Fr 8:00-16:00"/>  
<tag k="short_name" v="FBH"/>  
<tag k="website" v="https://www.bibliothek.kit.edu/cms/fbh.php"/>  
<tag k="wheelchair" v="yes"/>  
</node>
```


OSM Dateiformat - Knoten



Ein Node (Knoten)

- ist das Grundelement für alles auf der Karte
- ist eine Position auf der Karte.
- hat eine ID, sowie Breiten- und Längengrad
- kann beispielsweise ein Punkt einer Straße oder ein Ort (z.B. Restaurant) sein
- sind optional mit Tags versehen.
Tags beinhalten Informationen wie z.B. Straßentyp, Öffnungszeiten, etc.

```
<node id="643462941" visible="true" version="12" changeset="155336545" timest  
<tag k="amenity" v="library"/>  
<tag k="check_date:opening_hours" v="2024-08-16"/>  
<tag k="description" v="Abweichende Öffnungszeiten während Klausurphase"/>  
<tag k="internet_access" v="wlan"/>  
<tag k="level" v="1"/>  
<tag k="name" v="Fachbibliothek Hochschule Karlsruhe"/>  
<tag k="note" v="Außenstelle der KIT-Bibliothek"/>  
<tag k="opening_hours" v="Mo-Fr 08:00-21:00"/>  
<tag k="ref:isil" v="DE-90-4"/>  
<tag k="service_times" v="Mo-Fr 8:00-16:00"/>  
<tag k="short_name" v="FBH"/>  
<tag k="website" v="https://www.bibliothek.kit.edu/cms/fbh.php"/>  
<tag k="wheelchair" v="yes"/>  
</node>
```

OSM Dateiformat - Straßen



Straßen (way) werde als

- eine geordnete Liste von Node IDs beschrieben
- weitere Informationen (z.B. Straßenname) wird durch Tags hinzugefügt

```
<way id="3100654" visible="true" version="13" changeset="145541440" timestamp="2023-12-26T16:52:51Z" user="Doggy77" uid="20092260">
  <nd ref="25947870"/>
  <nd ref="4935288158"/>
  <nd ref="4935288181"/>
  <nd ref="1719671859"/>
  <nd ref="14795418"/>
  <tag k="cycleway:both" v="no"/>
  <tag k="highway" v="residential"/>
  <tag k="lit" v="yes"/>
  <tag k="maxspeed" v="30"/>
  <tag k="name" v="Schirmerstraße"/>
  <tag k="oneway" v="no"/>
  <tag k="sidewalk" v="both"/>
  <tag k="surface" v="asphalt"/>
  <tag k="width" v="6"/>
</way>
```

OSM Dateiformat - Straßen



Straßen (way) werde als

- eine geordnete Liste von Node IDs beschrieben
- weitere Informationen (z.B. Straßenname) wird durch Tags hinzugefügt

```
<way id="3100654" visible="true" version="13" changeset="145541440" timestamp="2023-12-26T16:52:51Z" user="Doggy77" uid="20092260">
  <nd ref="25947870"/>
  <nd ref="4935288158"/>
  <nd ref="4935288181"/>
  <nd ref="1719671859"/>
  <nd ref="14795418"/>
  <tag k="cycleway:both" v="no"/>
  <tag k="highway" v="residential"/>
  <tag k="lit" v="yes"/>
  <tag k="maxspeed" v="30"/>
  <tag k="name" v="Schirmerstraße"/>
  <tag k="oneway" v="no"/>
  <tag k="sidewalk" v="both"/>
  <tag k="surface" v="asphalt"/>
  <tag k="width" v="6"/>
</way>
```

geordnete Liste von Knoten

OSM Dateiformat - Straßen



Straßen (way) werde als

- eine geordnete Liste von Node IDs beschrieben
- weitere Informationen (z.B. Straßenname) wird durch Tags hinzugefügt

```
<way id="3100654" visible="true" version="13" changeset="145541440" timestamp="2023-12-26T16:52:51Z" user="Doggy77" uid="20092260">
  <nd ref="25947870"/>
  <nd ref="4935288158"/>
  <nd ref="4935288181"/>
  <nd ref="1719671859"/>
  <nd ref="14795418"/>
  <tag k="cycleway:both" v="no"/>
  <tag k="highway" v="residential"/>
  <tag k="lit" v="yes"/>
  <tag k="maxspeed" v="30"/>
  <tag k="name" v="Schirmerstraße"/>
  <tag k="oneway" v="no"/>
  <tag k="sidewalk" v="both"/>
  <tag k="surface" v="asphalt"/>
  <tag k="width" v="6"/>
</way>
```


OSM Dateiformat - Straßen



- Straßen (way) werde als
- eine geordnete Liste von IDs beschrieben
 - weitere Informationen (z.B. Straßenname) wird durch Tags hinzugefügt

```
<way id="3100654" visible="true" version="1"
  <nd ref="25947870"/>
  <nd ref="4935288158"/>
  <nd ref="4935288181"/>
  <nd ref="1719671859"/>
  <nd ref="14795418"/>
  <tag k="cycleway:both" v="no"/>
  <tag k="highway" v="residential"/>
  <tag k="lit" v="yes"/>
  <tag k="maxspeed" v="30"/>
  <tag k="name" v="Schirmerstraße"/>
  <tag k="oneway" v="no"/>
  <tag k="sidewalk" v="both"/>
  <tag k="surface" v="asphalt"/>
  <tag k="width" v="6"/>
</way>
```

Key	Value	Element	Comment	Rendering carto	Examples
Roads					
This group lists the 7 main tags for the road network, from most to least functionally important for motor vehicle traffic.					
highway	motorway		A restricted access major divided highway, normally with 2 or more running lanes plus emergency hard shoulder. Equivalent to the Freeway, Autobahn, etc..		
highway	trunk		The most important roads in a country's system that aren't motorways. (Need not necessarily be a divided highway.)		
highway	primary		The next most important roads in a country's system. (Often link larger towns.)		

<https://wiki.openstreetmap.org/wiki/Key:highway>

OSM Dateiformat - Straßen



Straßen (way) werde als

- eine geordnete Liste von Node IDs beschrieben
- weitere Informationen (z.B. Straßenname) wird durch Tags hinzugefügt

```
<way id="3100654" visible="true" version="13" changeset="145541440" timestamp="2023-12-26T16:52:51Z" user="Doggy77" uid="20092260">
  <nd ref="25947870"/>
  <nd ref="4935288158"/>
  <nd ref="4935288181"/>
  <nd ref="1719671859"/>
  <nd ref="14795418"/>
  <tag k="cycleway:both" v="no"/>
  <tag k="highway" v="residential"/>
  <tag k="lit" v="yes"/>
  <tag k="maxspeed" v="30"/>
  <tag k="name" v="Schirmerstraße"/>
  <tag k="oneway" v="no"/>
  <tag k="sidewalk" v="both"/>
  <tag k="surface" v="asphalt"/>
  <tag k="width" v="6"/>
</way>
```



Kreuzungen

- mehrfache Verwendung von Nodes in mehreren Straßen definieren Kreuzungen
- spezielle (optionale) Tags beschreiben Kreuzungen und fügen Informationen zu Ampeln oder Fußwegen hinzu

```
<node id="15357691" visible="true" version="16" changeset="164906044" timestamp="2025-04-13T17:38:02Z" user="EinEngener" uid=""  
  <tag k="crossing" v="uncontrolled"/>  
  <tag k="crossing:island" v="yes"/>  
  <tag k="crossing:markings" v="no"/>  
  <tag k="crossing:signals" v="no"/>  
  <tag k="highway" v="crossing"/>  
  <tag k="tactile_paving" v="no"/>  
</node>
```



Kreuzungen

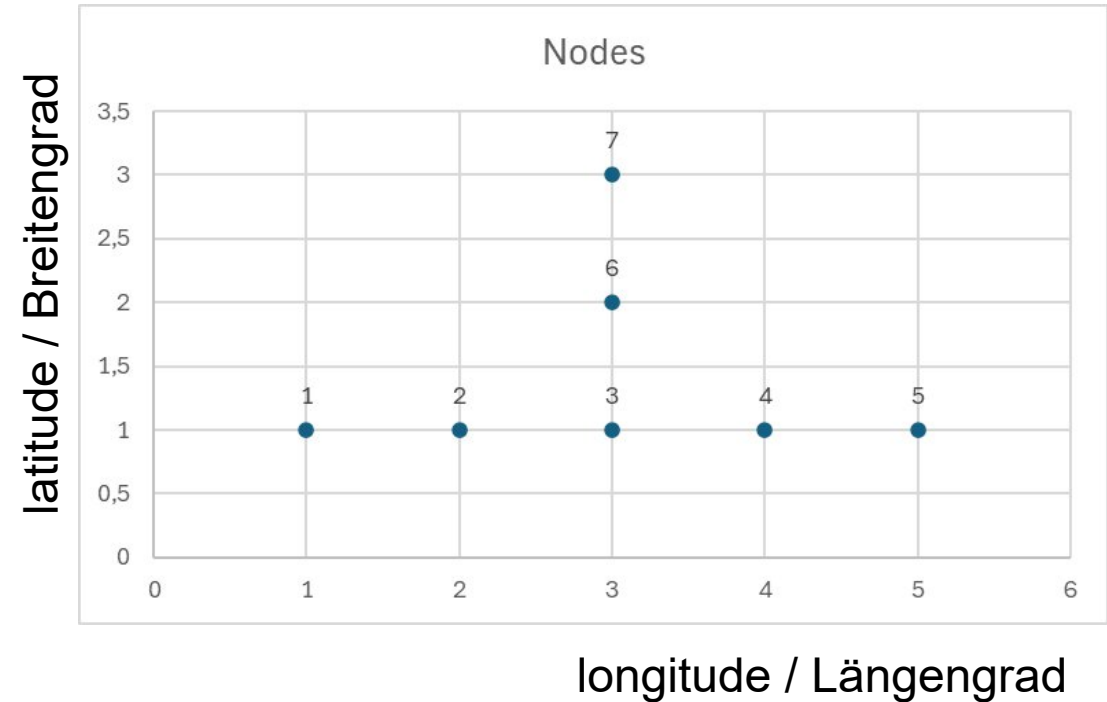
- mehrfache Verwendung von Nodes in mehreren Straßen definieren Kreuzungen
- spezielle (optionale) Tags beschreiben Kreuzungen und fügen Informationen zu Ampeln oder Fußwegen hinzu

```
<node id="15357691" visible="true" version="16" changeset="164906044" timestamp="2025-04-13T17:38:02Z" user="EinEngener" uid="15357691">  
  <tag k="crossing" v="uncontrolled"/>  
  <tag k="crossing:island" v="yes"/>  
  <tag k="crossing:markings" v="no"/>  
  <tag k="crossing:signals" v="no"/>  
  <tag k="highway" v="crossing"/>  
  <tag k="tactile_paving" v="no"/>  
</node>
```


OSM Beispiel: Nodes/Way/Intersection



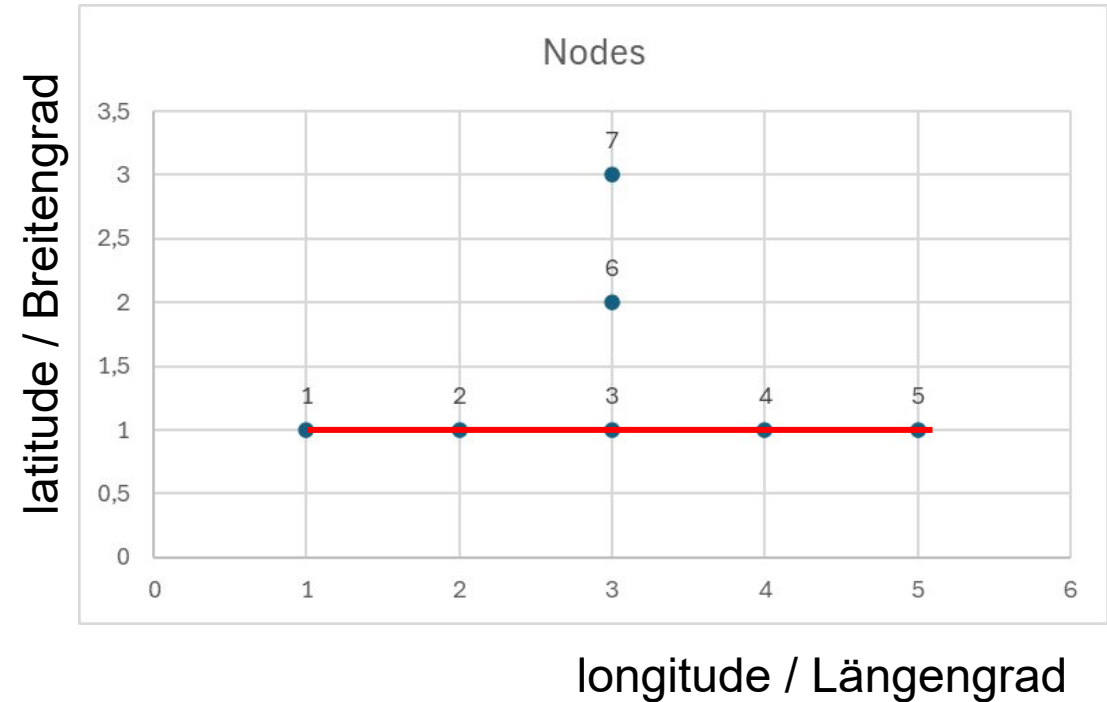
```
<node id="1" [..] lat="1" lon="1"/>  
<node id="2" [..] lat="1" lon="2"/>  
<node id="3" [..] lat="1" lon="3"/>  
<node id="4" [..] lat="1" lon="4"/>  
<node id="5" [..] lat="1" lon="5"/>  
<node id="6" [..] lat="2" lon="3"/>  
<node id="7" [..] lat="3" lon="3"/>
```



OSM Beispiel: Nodes/Way/Intersection



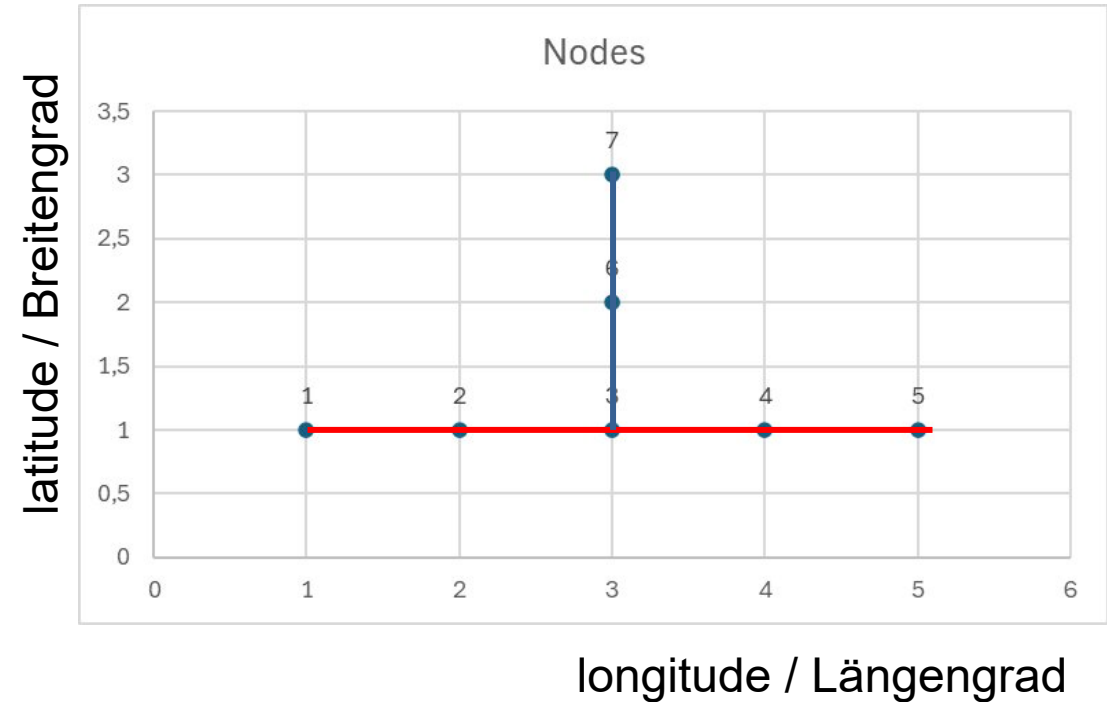
```
<way id="10" >  
  <nd ref="1"/>  
  <nd ref="2"/>  
  <nd ref="3"/>  
  <nd ref="4"/>  
  <nd ref="5"/>  
  <tag k="highway" v=[..]>  
</way>
```



OSM Beispiel: Nodes/Way/Intersection



```
<way id="11" >  
  <nd ref="7"/>  
  <nd ref="6"/>  
  <nd ref="3"/>  
  <tag k="highway" v=[..]>  
</way>
```



OSM Beispiel: Weg: Knielinger Allee (684886008)



Tags

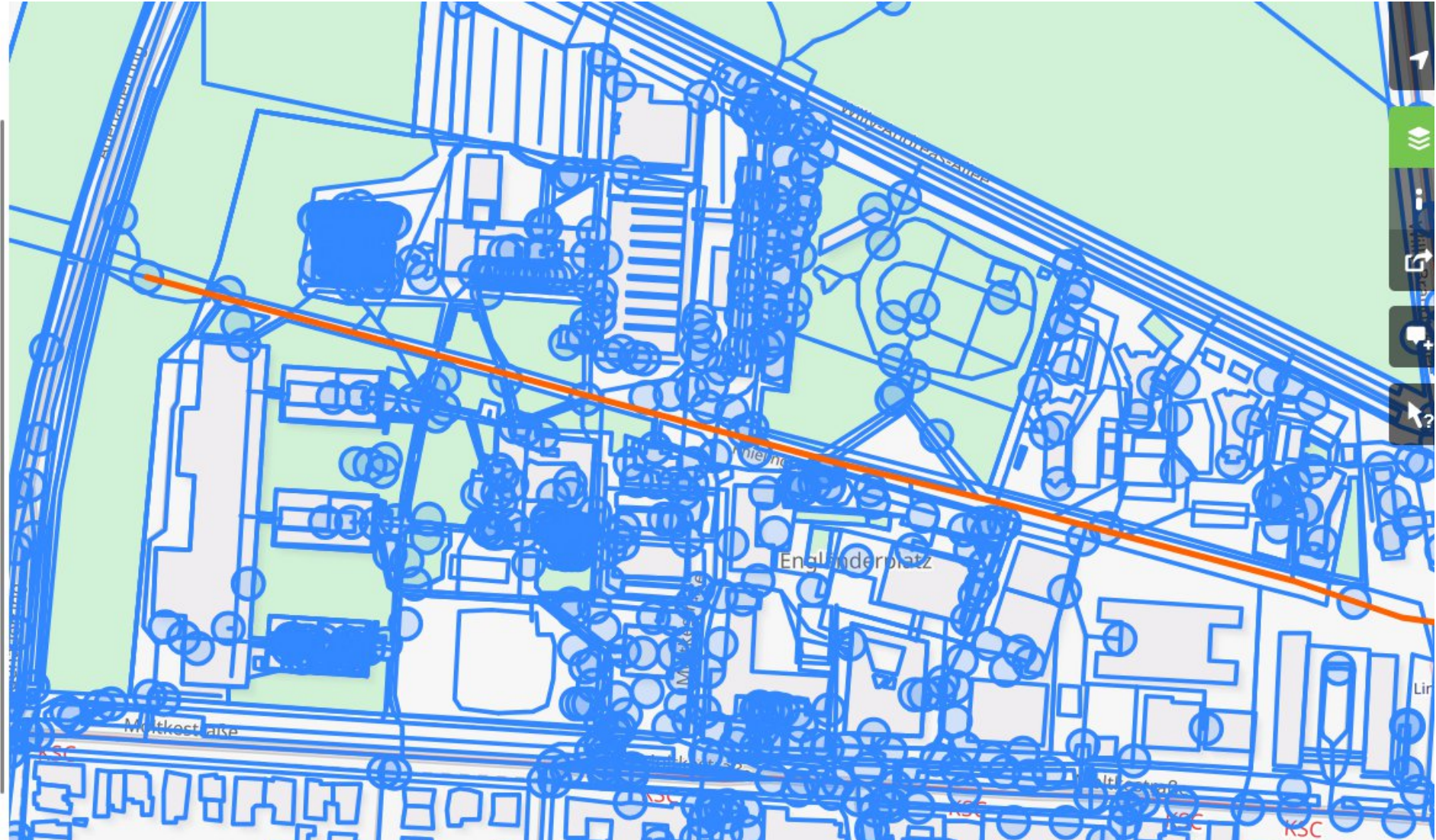
bicycle	designated
foot	designated
highway	cycleway
lit	yes
name	Knielinger Allee
oneway	no
segregated	no
smoothness	excellent
surface	asphalt
width	3

Teil von

- ▼ 1 Relation
- Fächerstadt Karlsruhe (9391038)

Knoten

- 29 Knoten



OSM Beispiel: Knielinger-Allee Kreuzung Willy-Brandt-Allee



Knoten: 15357691

Version #16

Ergänzungen bei Karlsruhe beim Schlosspark

Bearbeitet vor 6 Monaten von EinEngener

Änderungssatz #164906044

Standort: 49,0151513, 8,3981449

Tags

crossing	uncontrolled
crossing:island	yes
crossing:markings	no
crossing:signals	no
highway	crossing
tactile_paving	no

Teil von

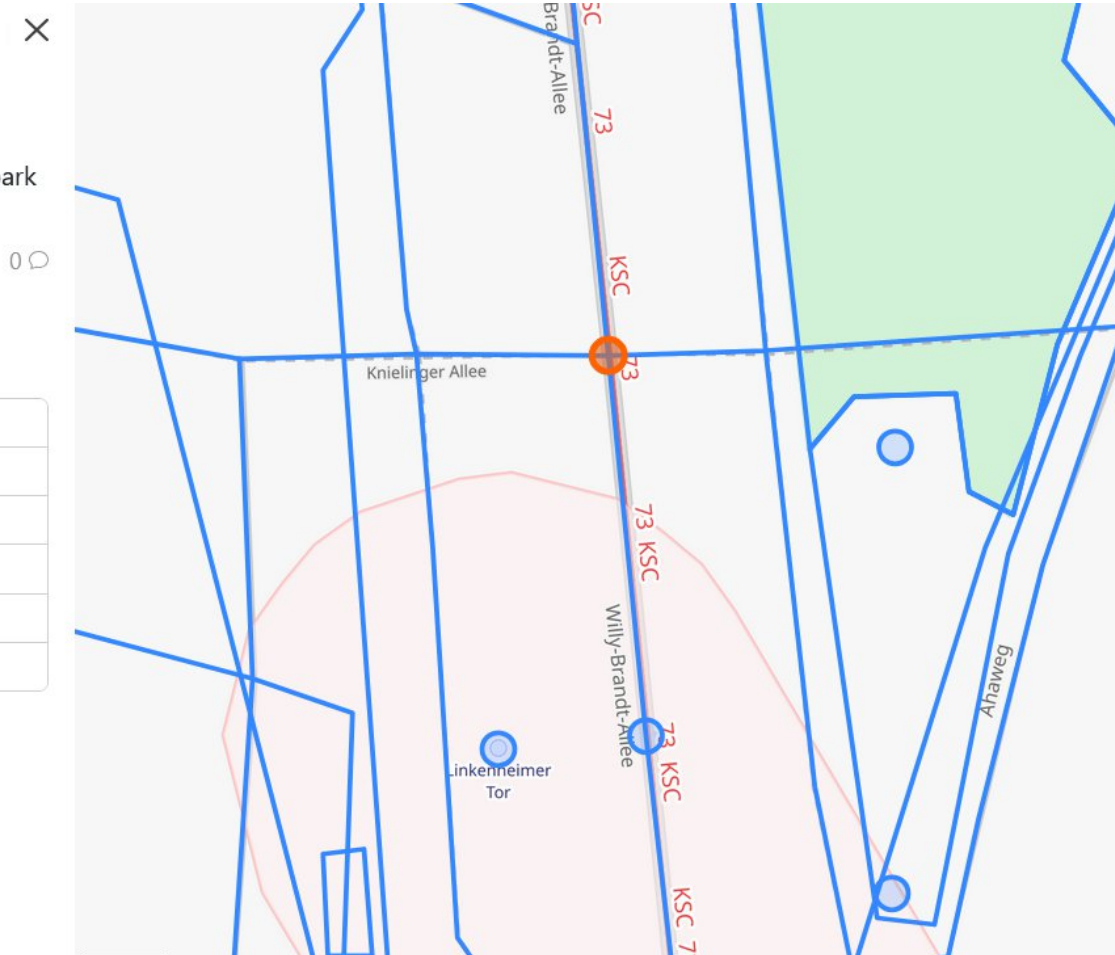
▼ 4 Wege

--- Knielinger Allee (3454244)

--- 686429169

— Willy-Brandt-Allee (261418596)

— Willy-Brandt-Allee (261418601)



OSM Beispiel: Knielinger-Allee Kreuzung Willy-Brandt-Allee



Weg: 686429169

Version #6

Ergänzungen bei Karlsruhe beim Schlosspark

Bearbeitet vor 6 Monaten von EinEngener

Änderungssatz #164906044

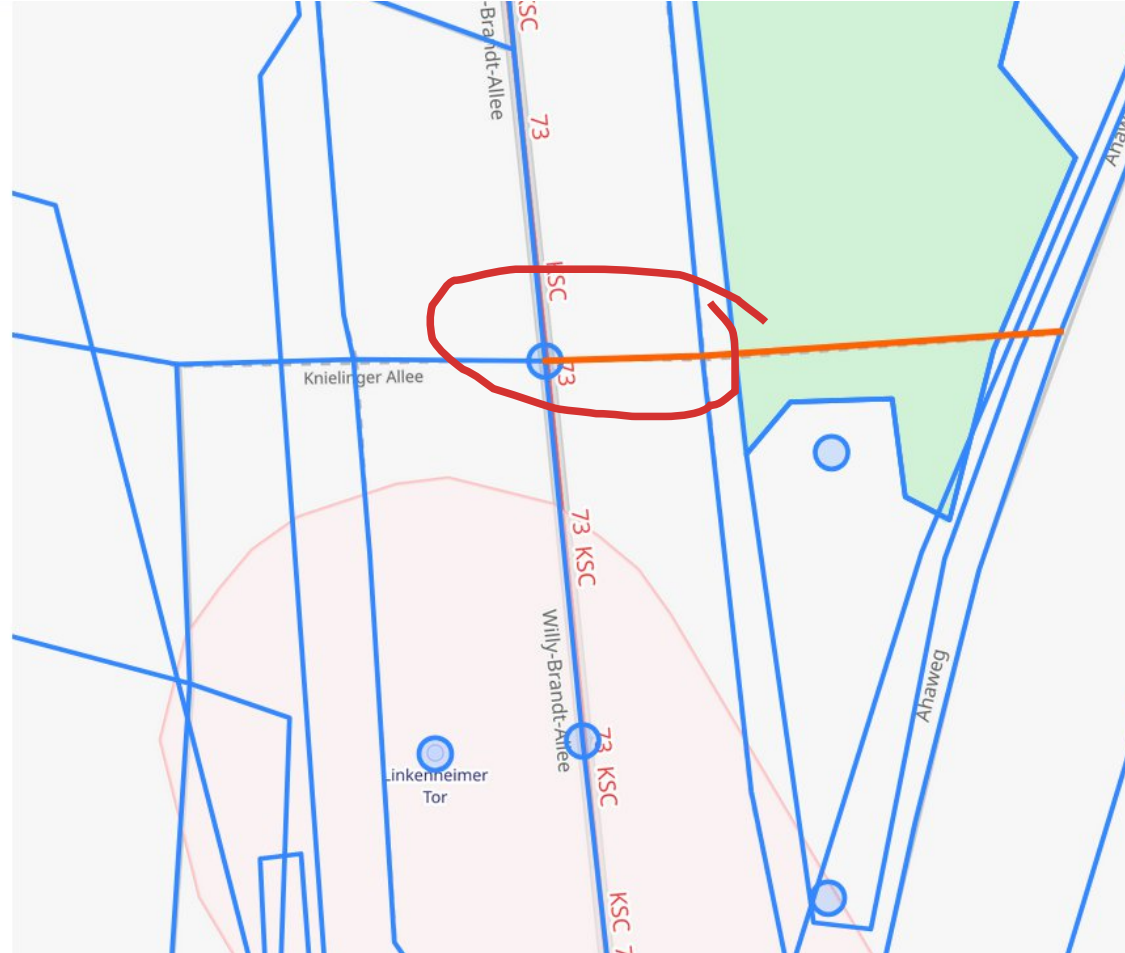
Tags

bicycle	no
highway	path
informal	yes
lit	no
surface	ground

Knoten

▼ 3 Knoten

- 1685844628 (Teil des Wags — Ahaweg (193461845))
- 55474029 (Teil des Wags 711366517)
- 15357691 (Teile der Wege Knielinger Allee (3454244), — Willy-Brandt-Allee (261418596) und — Willy-Brandt-Allee (261418601))



Nicht alle Wege (way) sind Strassen



Suchen

Wo ist dies?

Weg: M (28245327)

Version #17

Updated 4 university buildings and 2 unknown objects

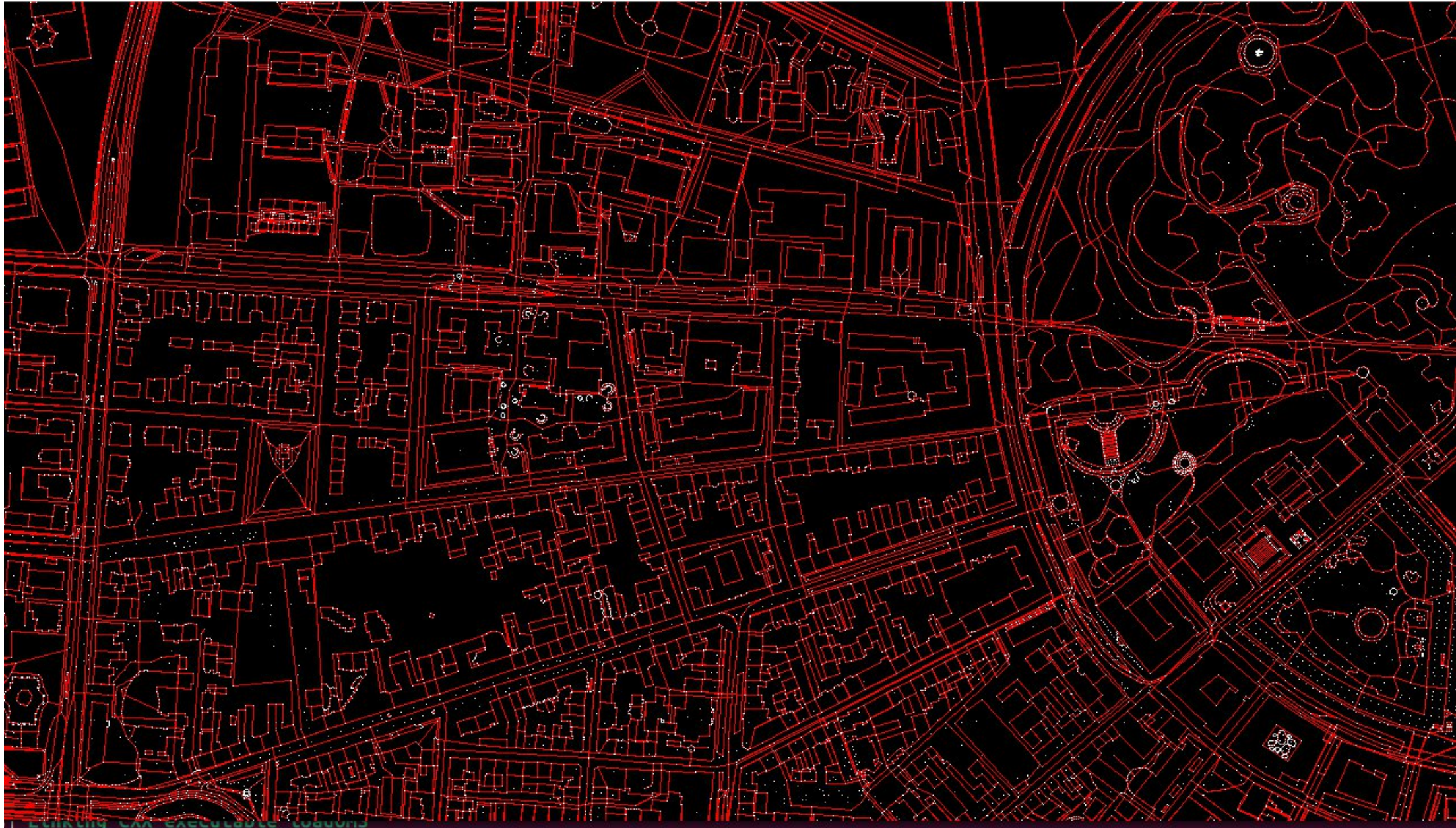
Bearbeitet vor etwa 2 Jahren von SenorHombre

Änderungssatz #142362310

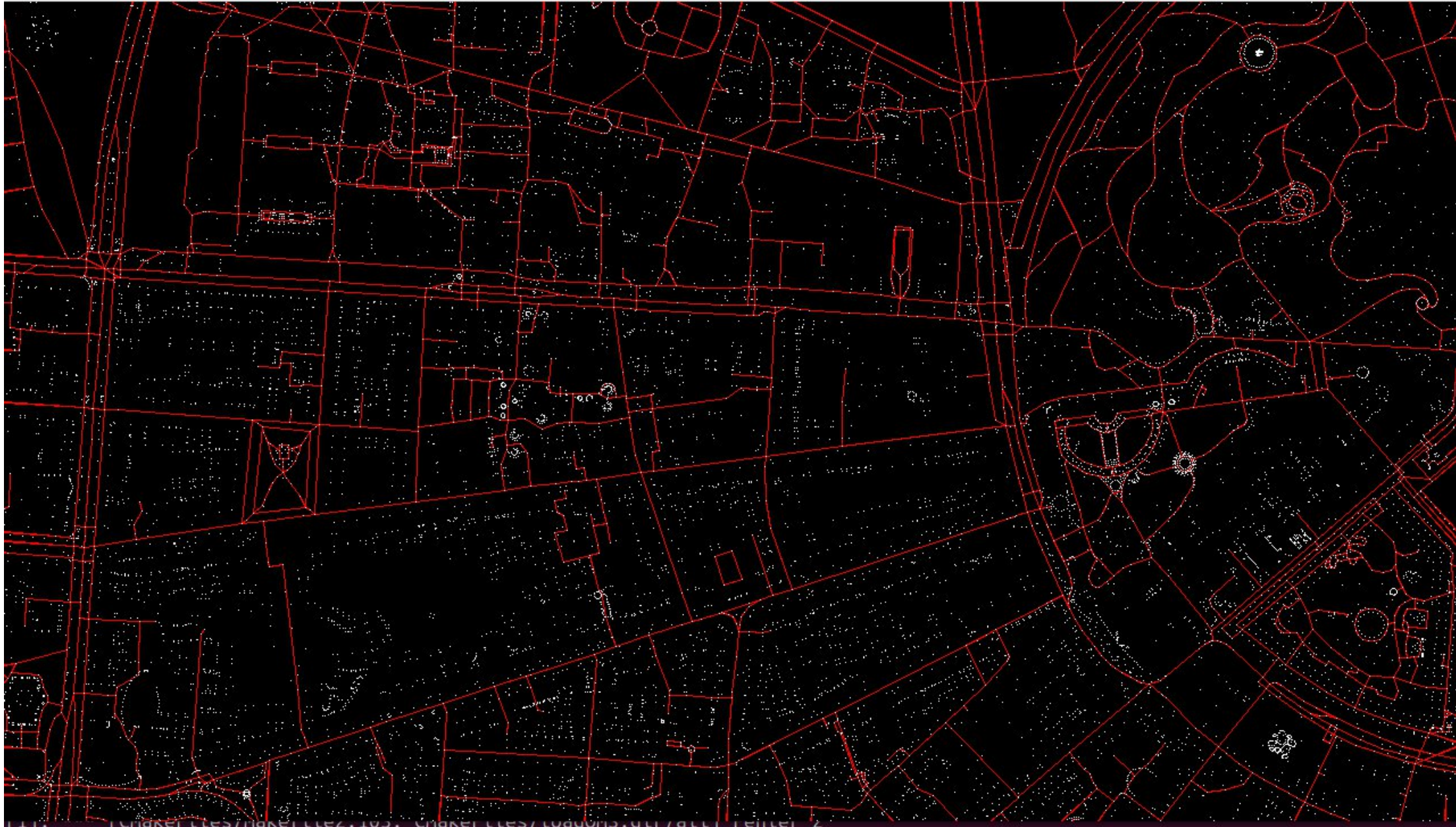
Tags

addr:housename	M
addr:housenumber	30
addr:street	Moltkestraße
building	university
building:levels	4
elevator	yes
height	18
name	M
operator	Hochschule Karlsruhe
roof:shape	flat
source	Maps4BW
wheelchair	no

Darstellung aller Ways



Darstellung aller Ways mit Tag <tag k="highway" v=[..]>



K
A

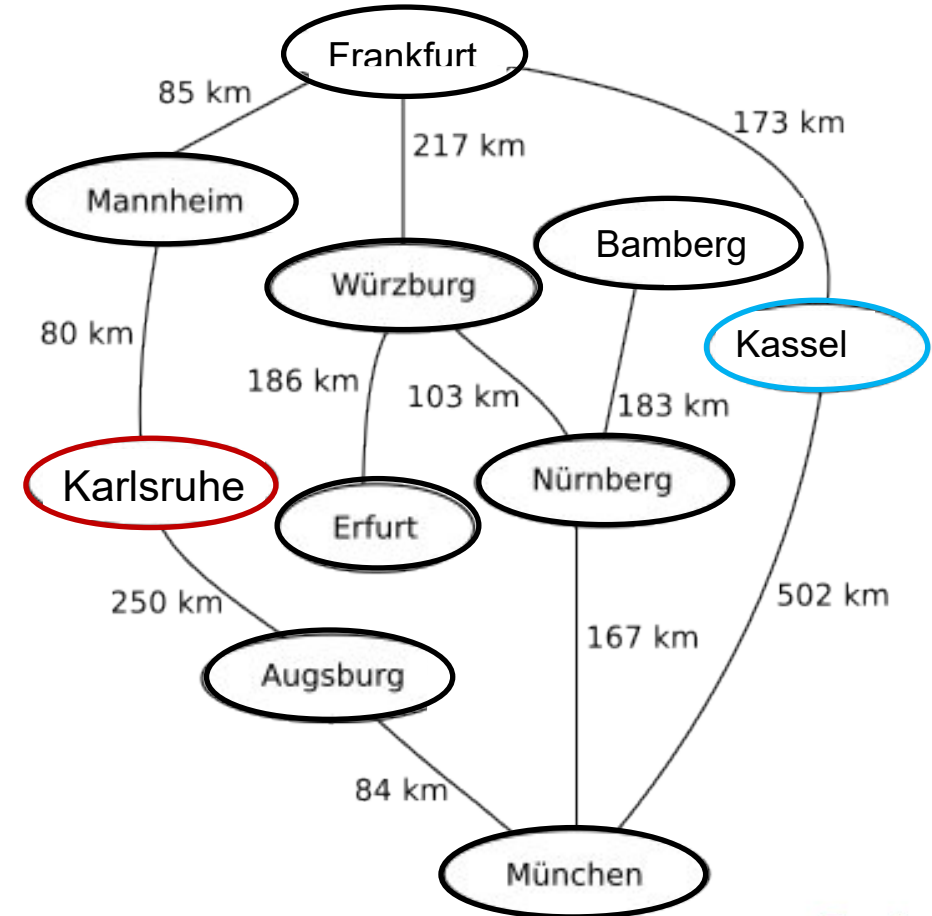
2. Dijkstra's algorithm

Dijkstra's algorithm



- löst das Problem der kürzesten Pfade
- zwischen Startknoten und einem Zielknoten
- kantengewichtete Graph

z.B. von **Karlsruhe** nach **Kassel**

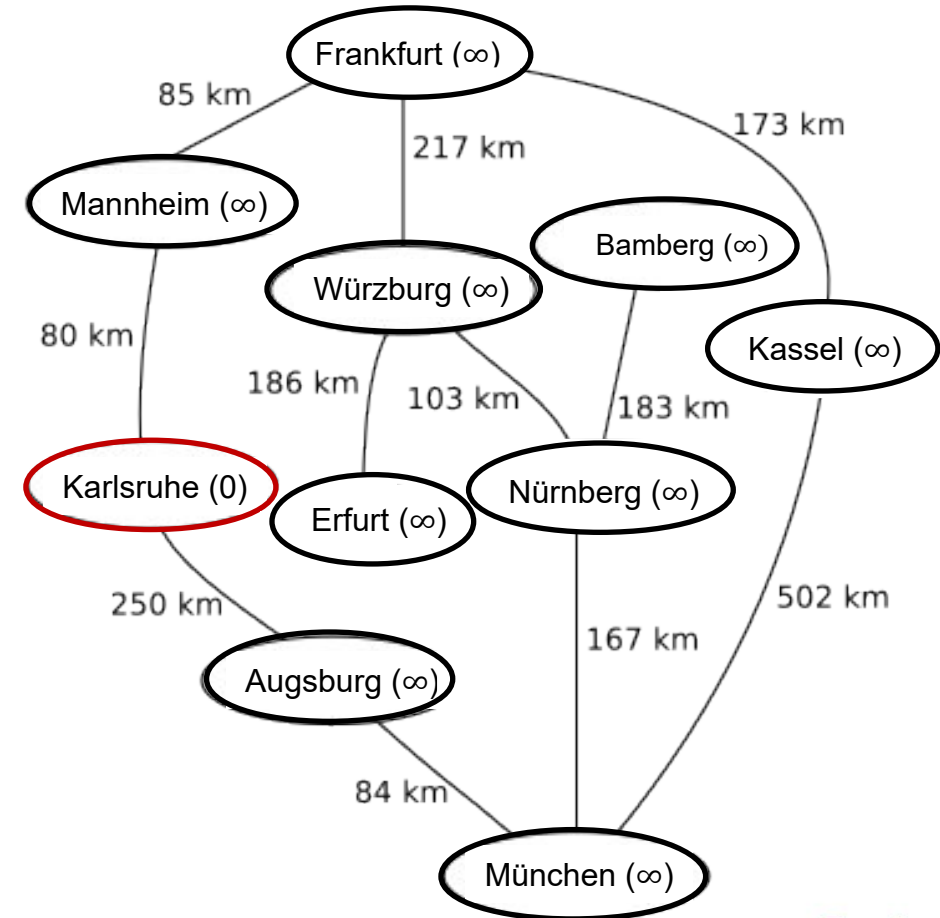


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die **Eigenschaften *Distanz* & *Vorgänger***.
Initialisiere: Distanz (**Startknoten**=0, ∞ alle anderen).

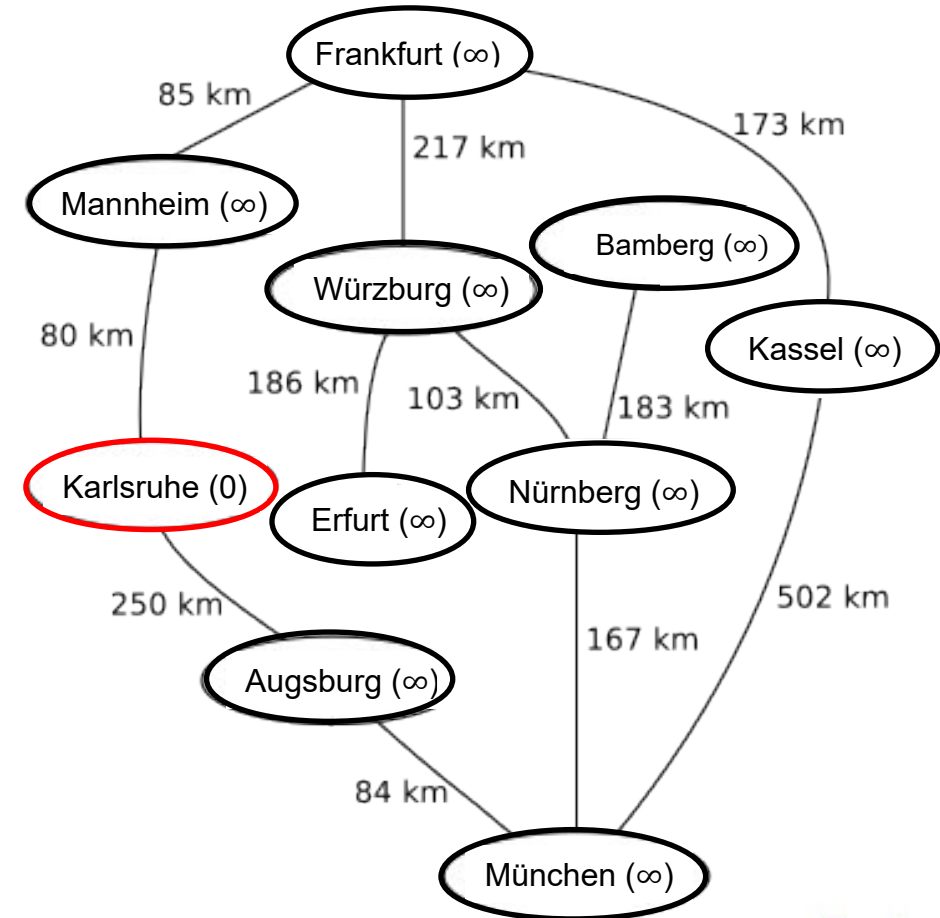


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus

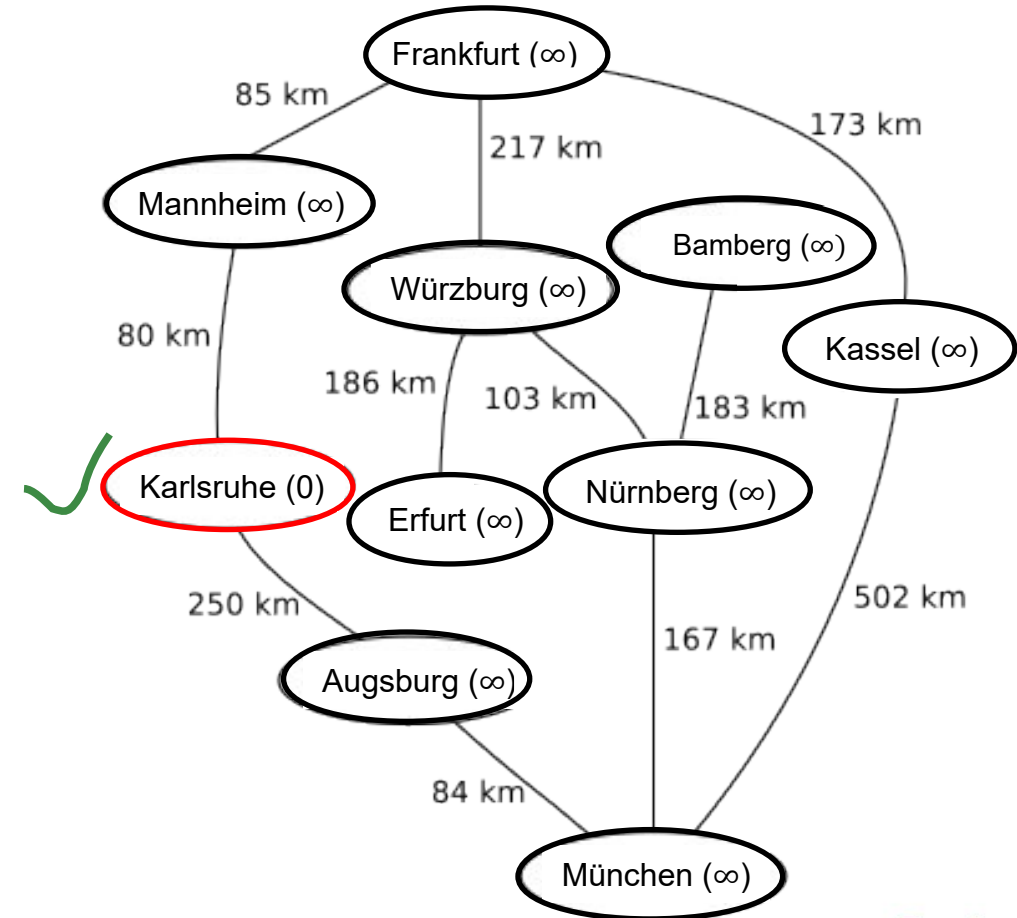


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
1. Markiere diesen Knoten **als besucht**.

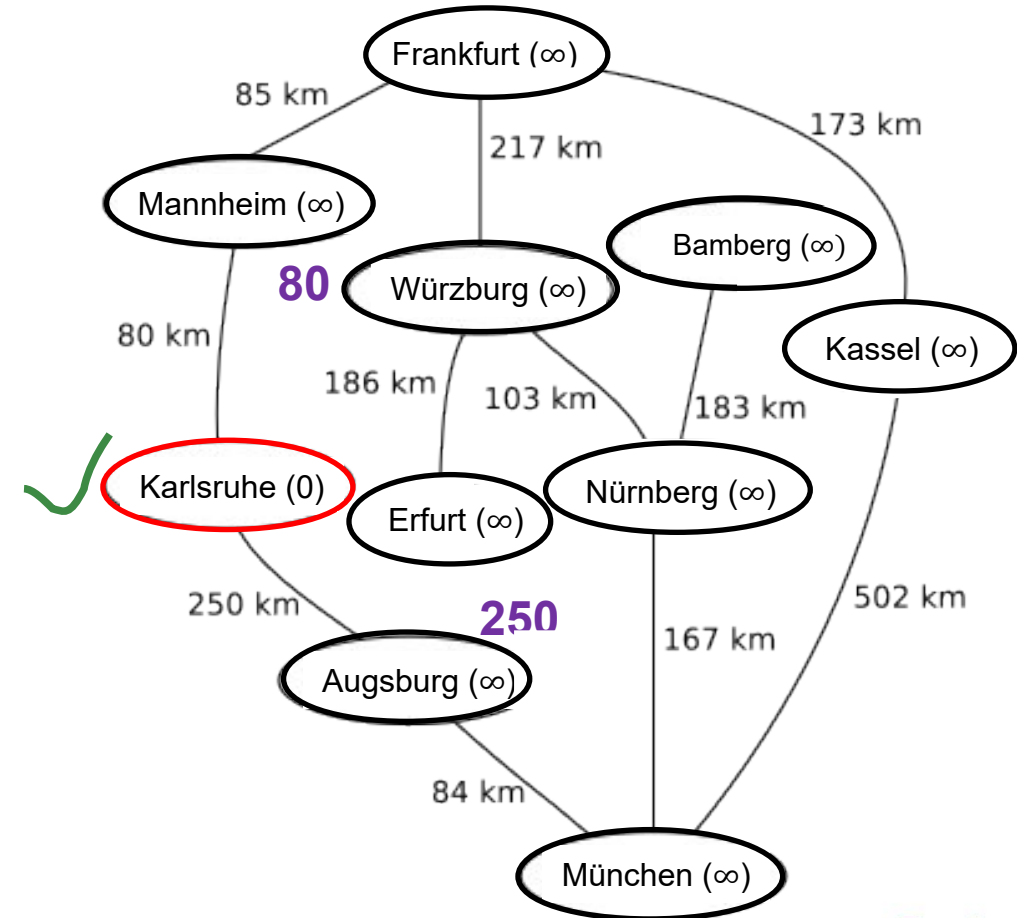


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).

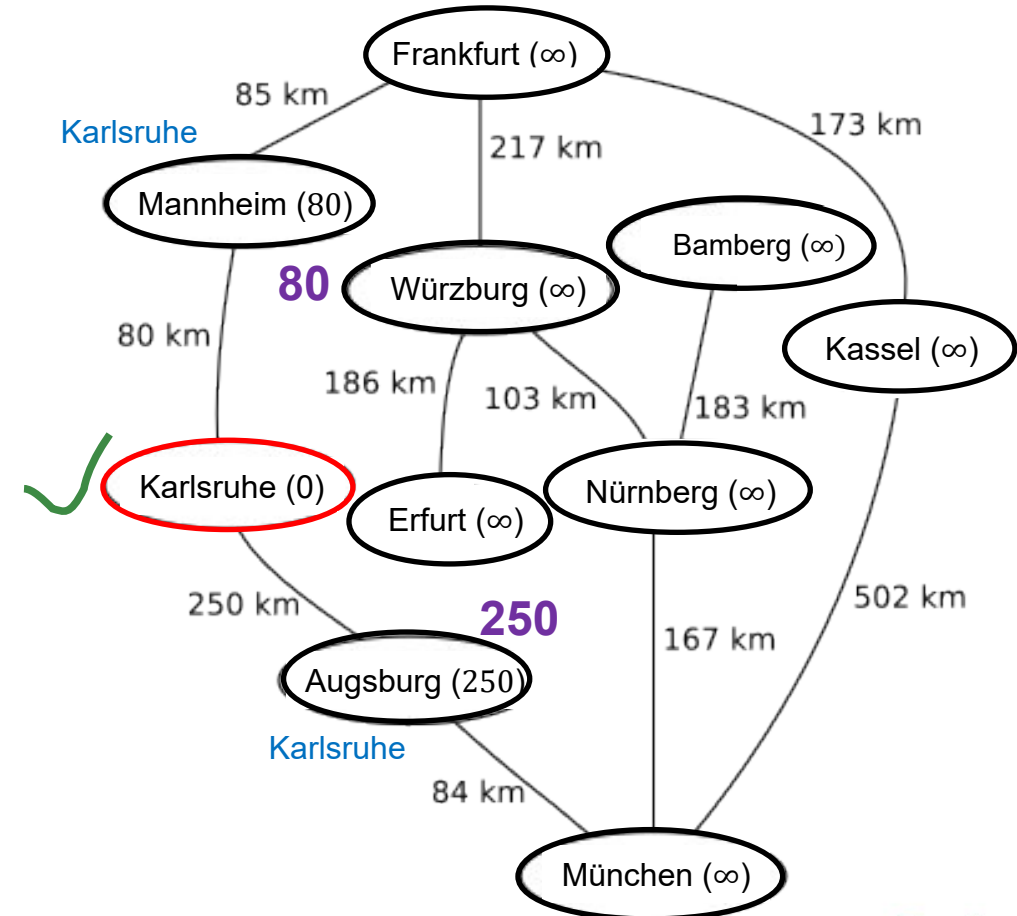


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. Berechne für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. **Relaxation**: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

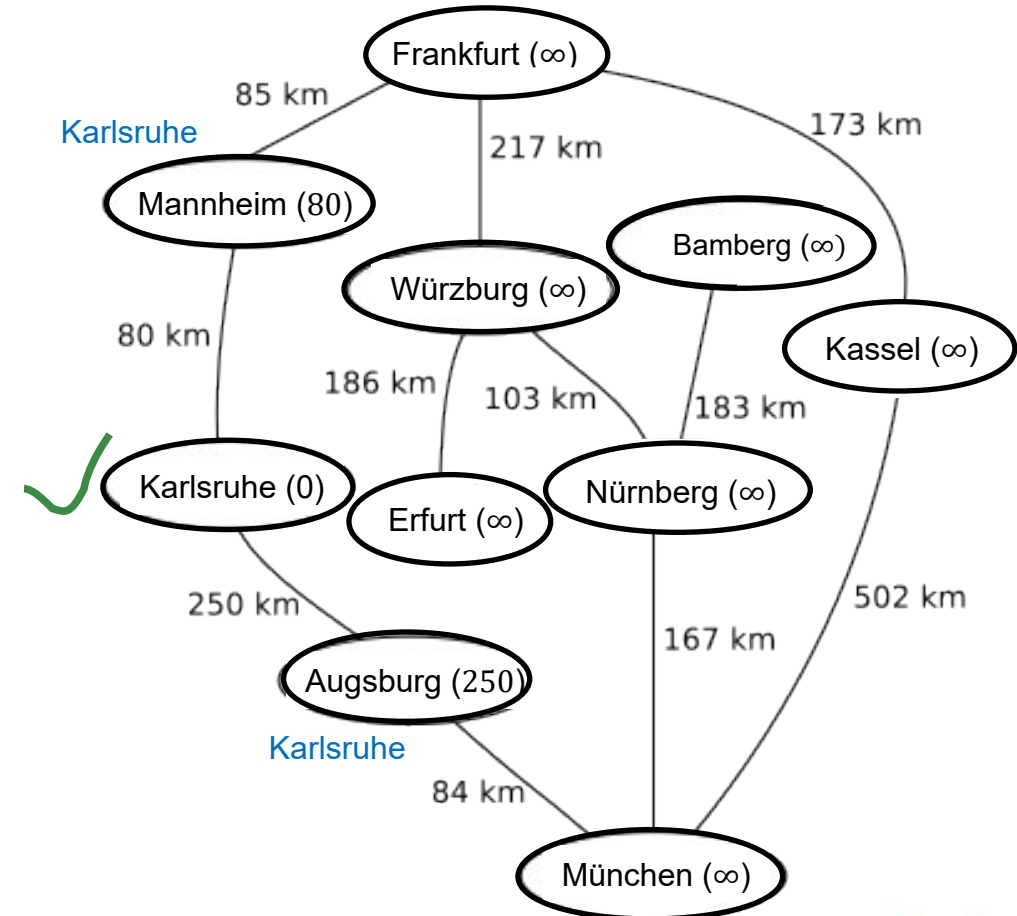


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. Berechne für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. **Relaxation**: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

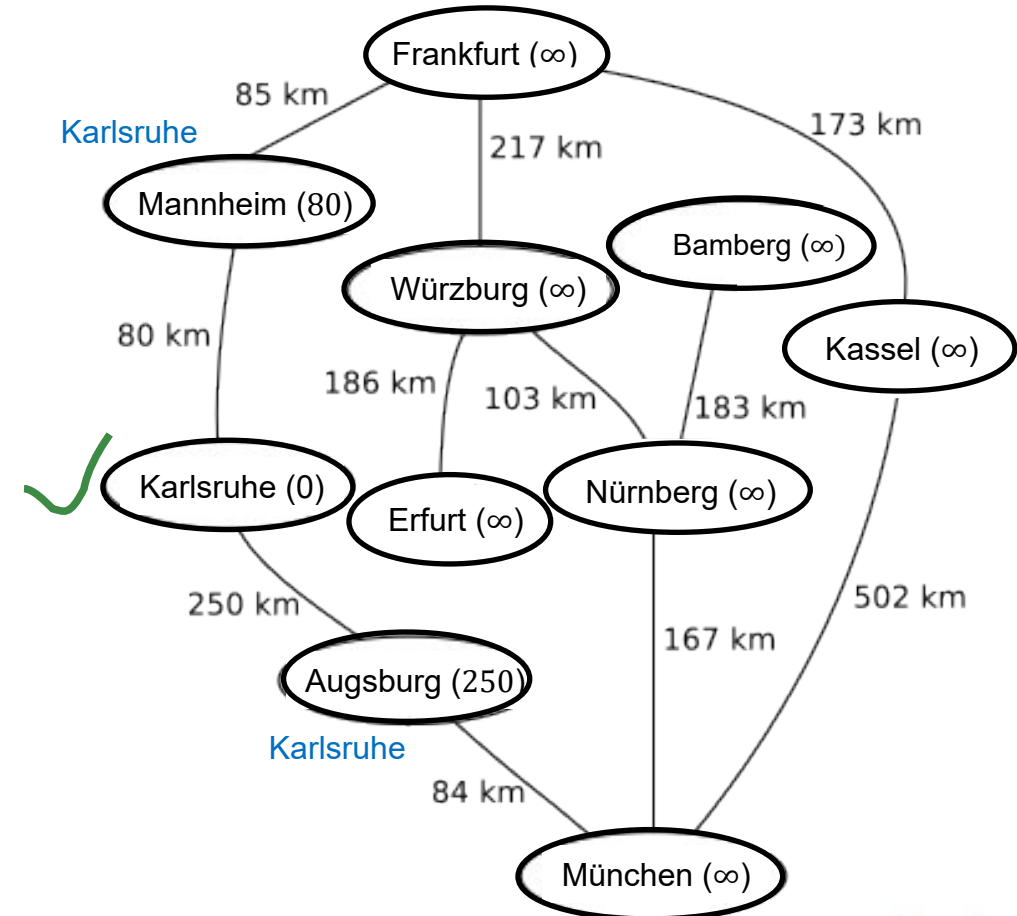


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
 3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

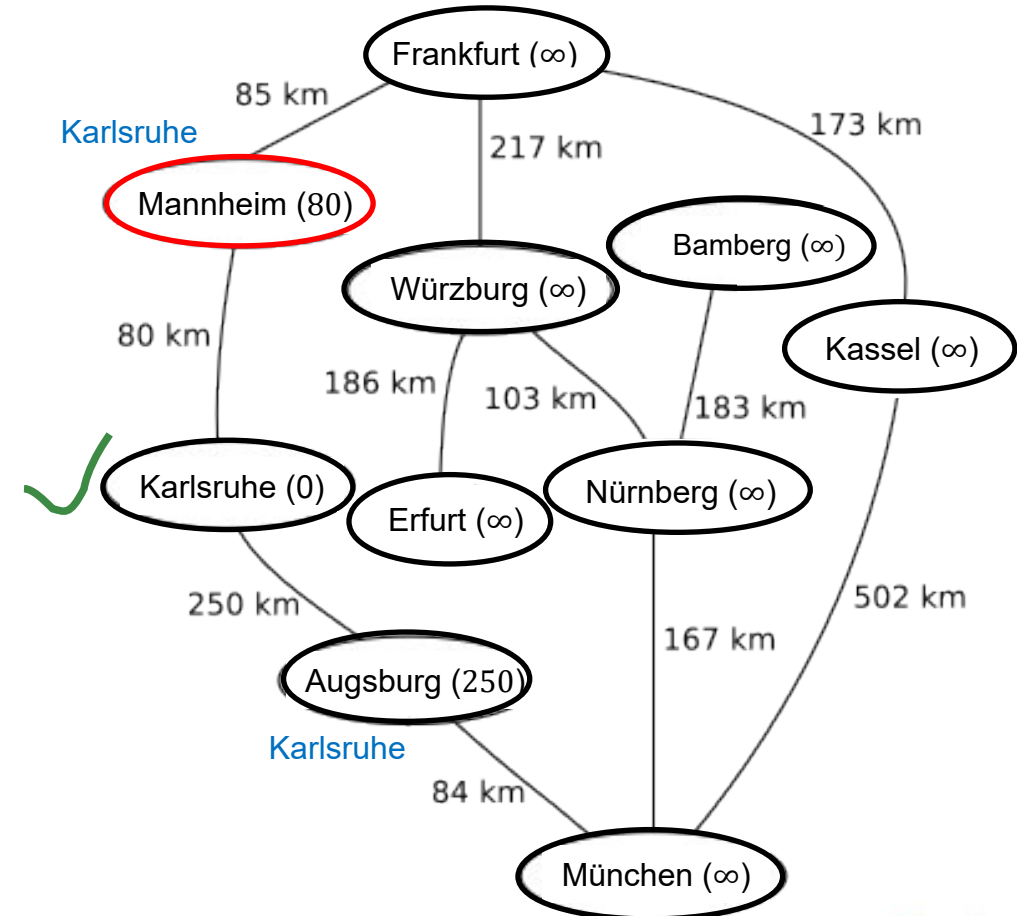


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus

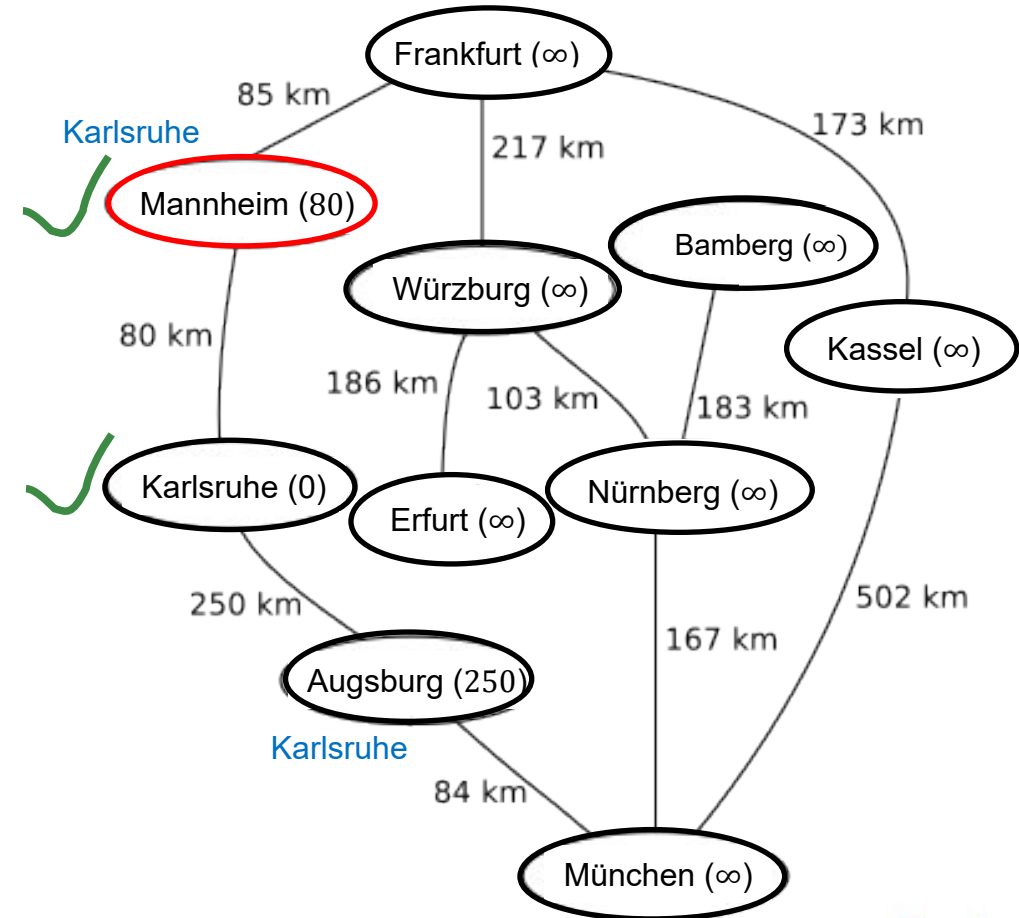


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
1. Markiere diesen Knoten **als besucht**.

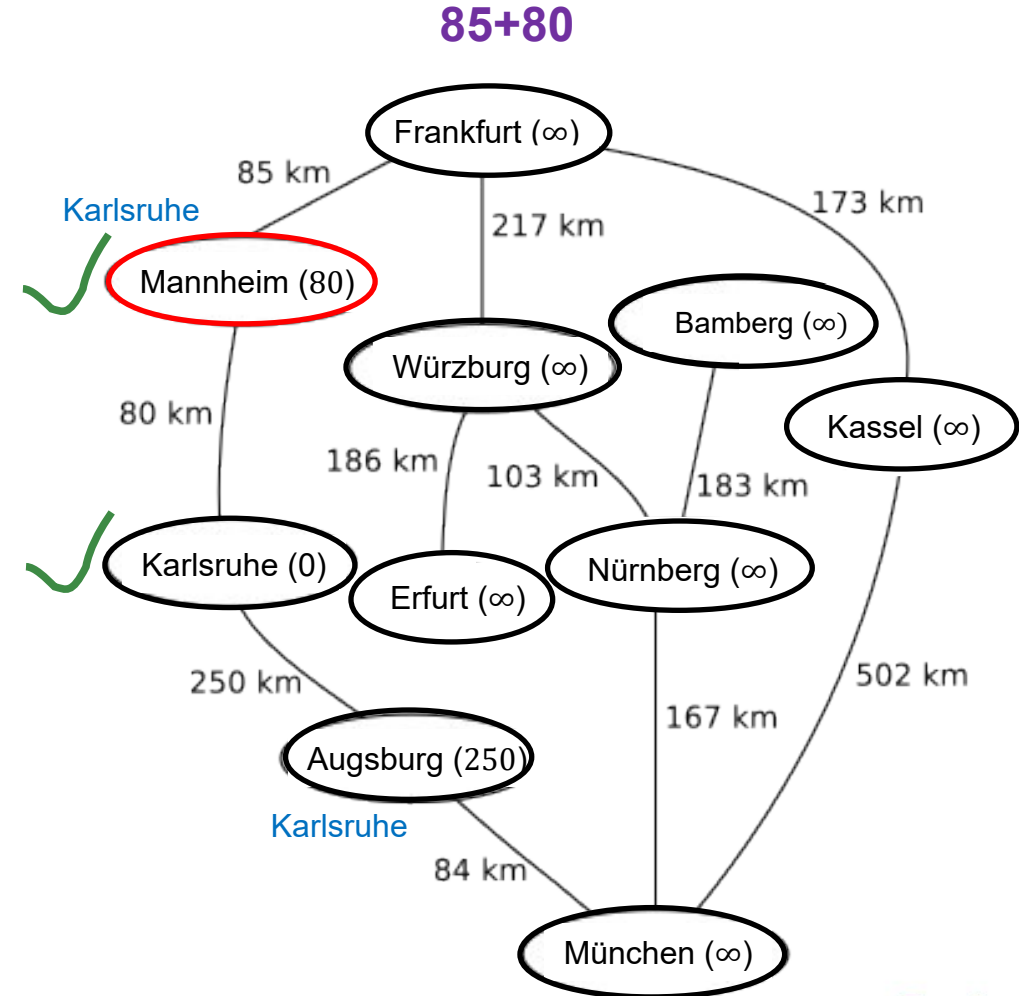


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).

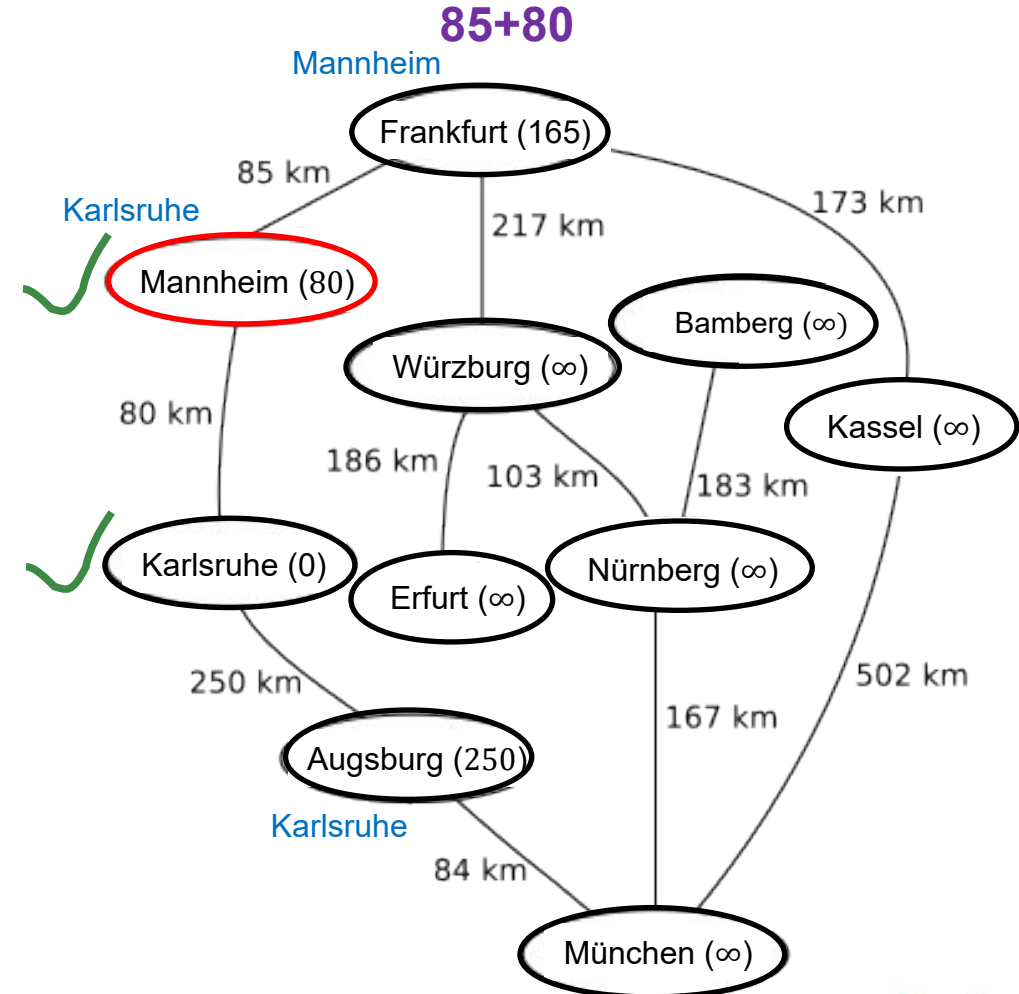


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. **Relaxation**: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

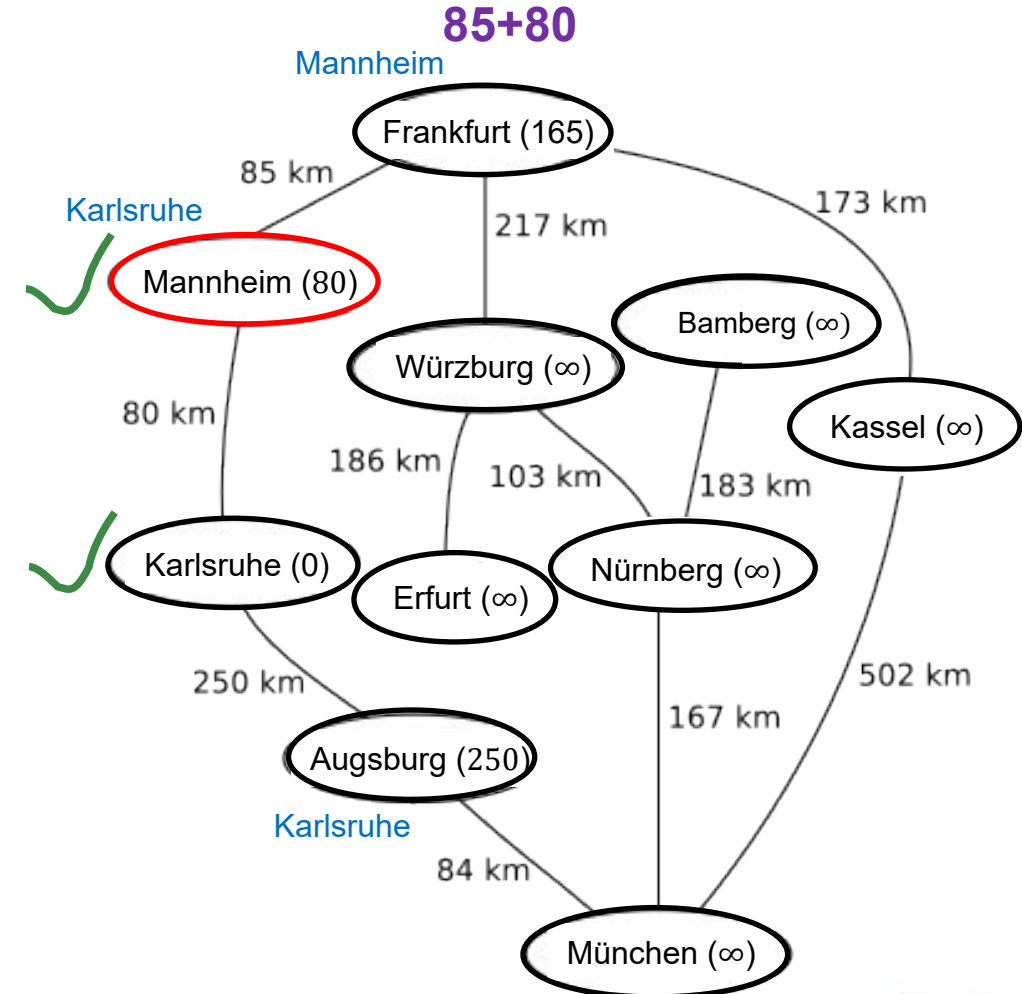


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
 3. **Relaxation**: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

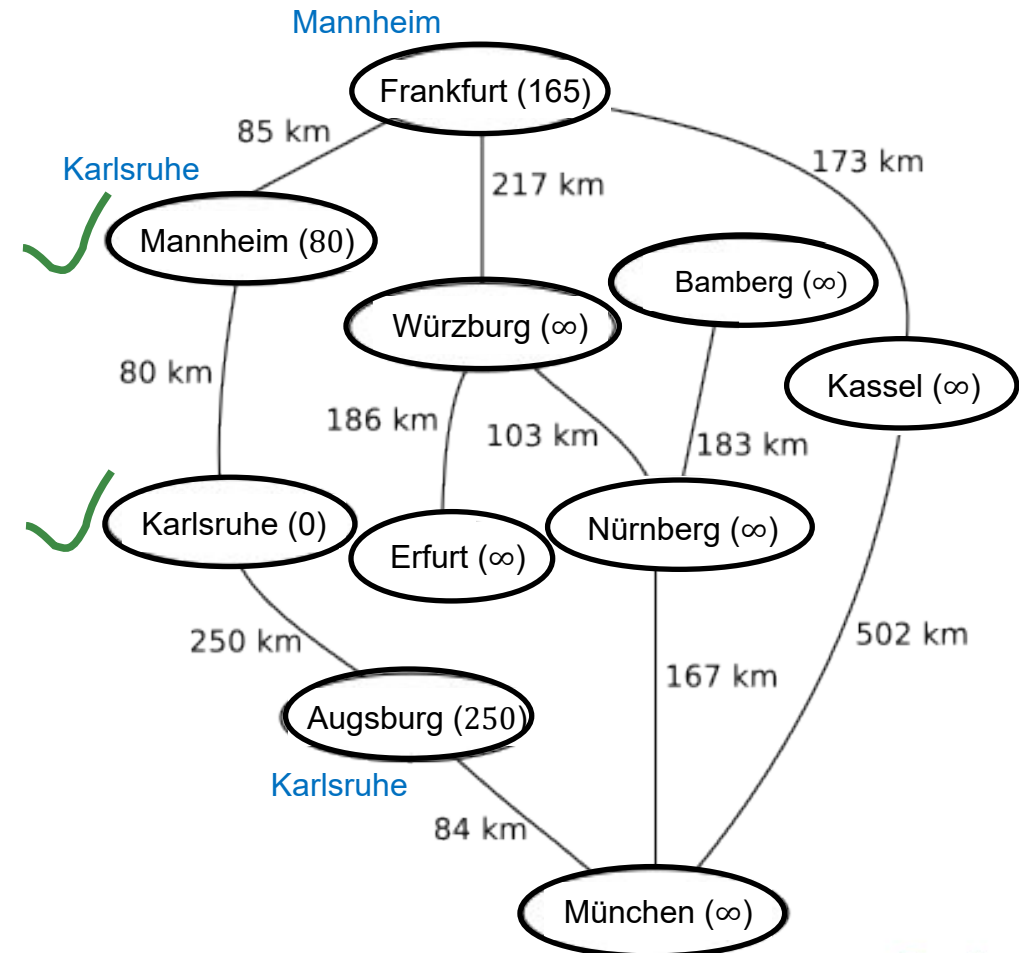


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

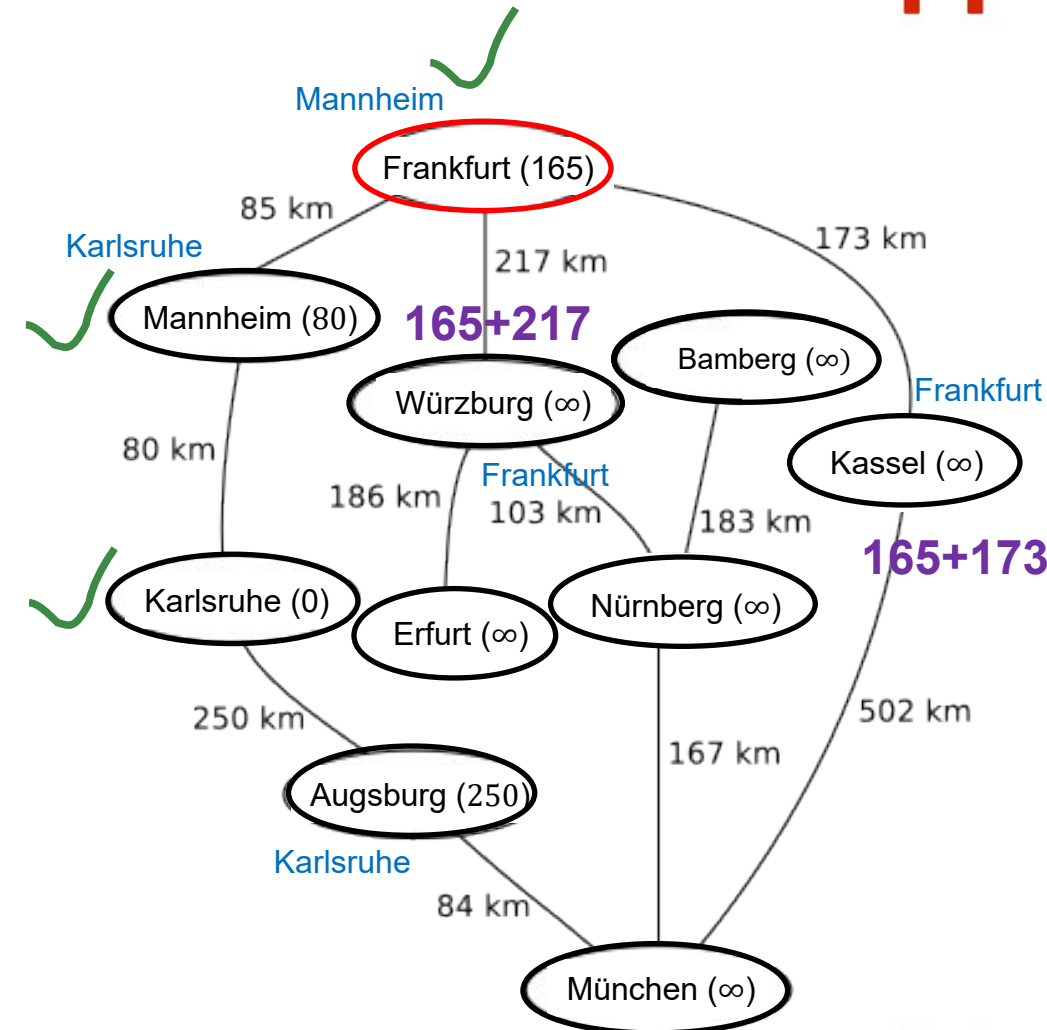


Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.



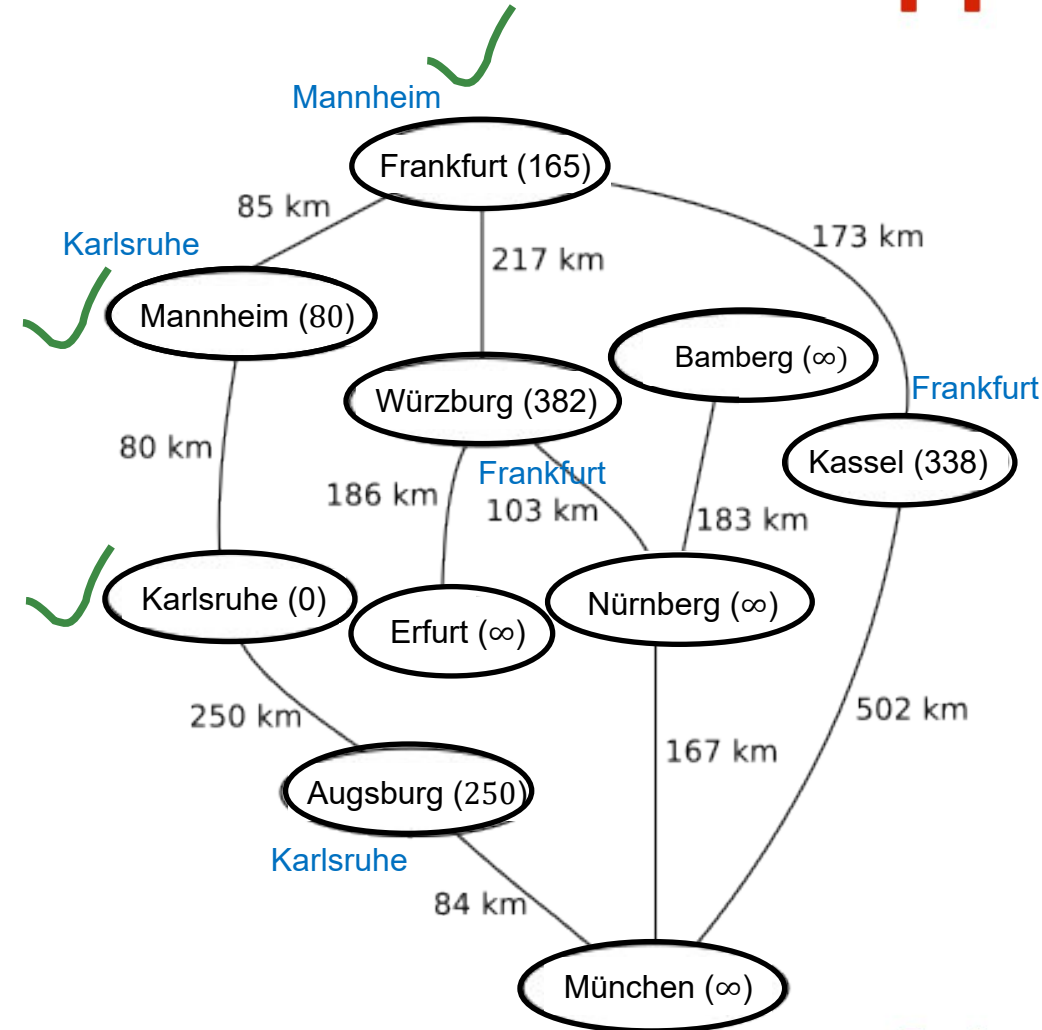
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



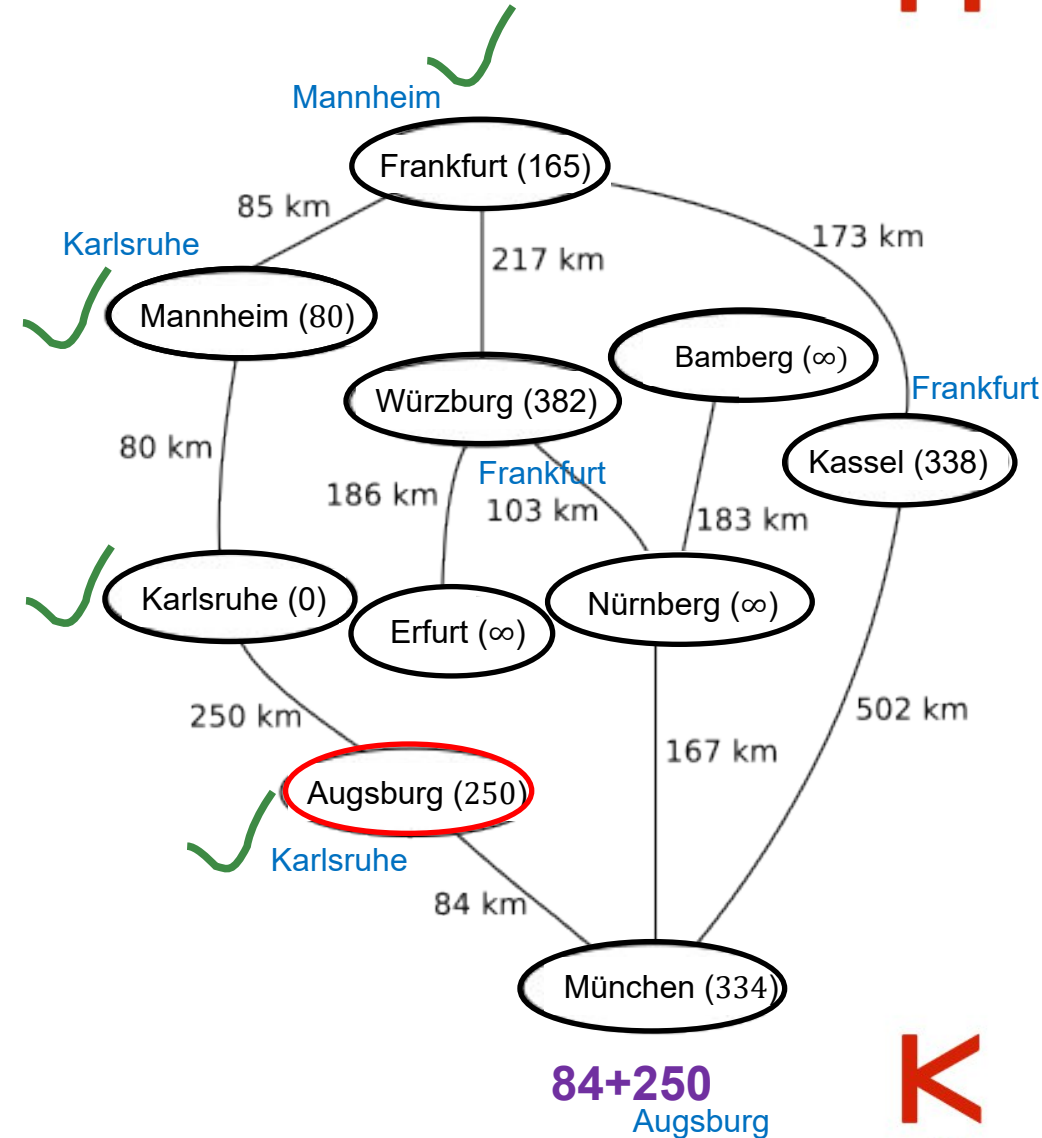
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



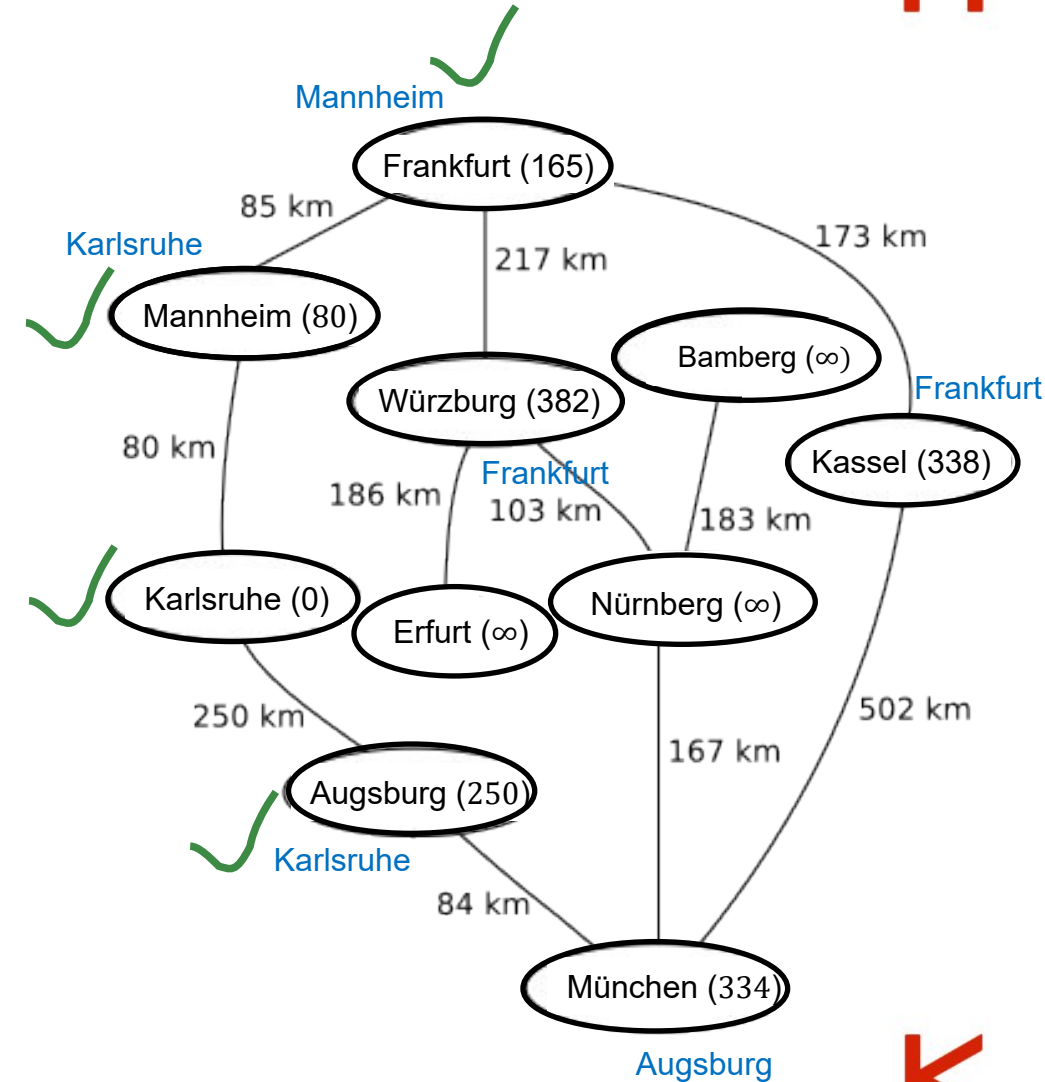
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



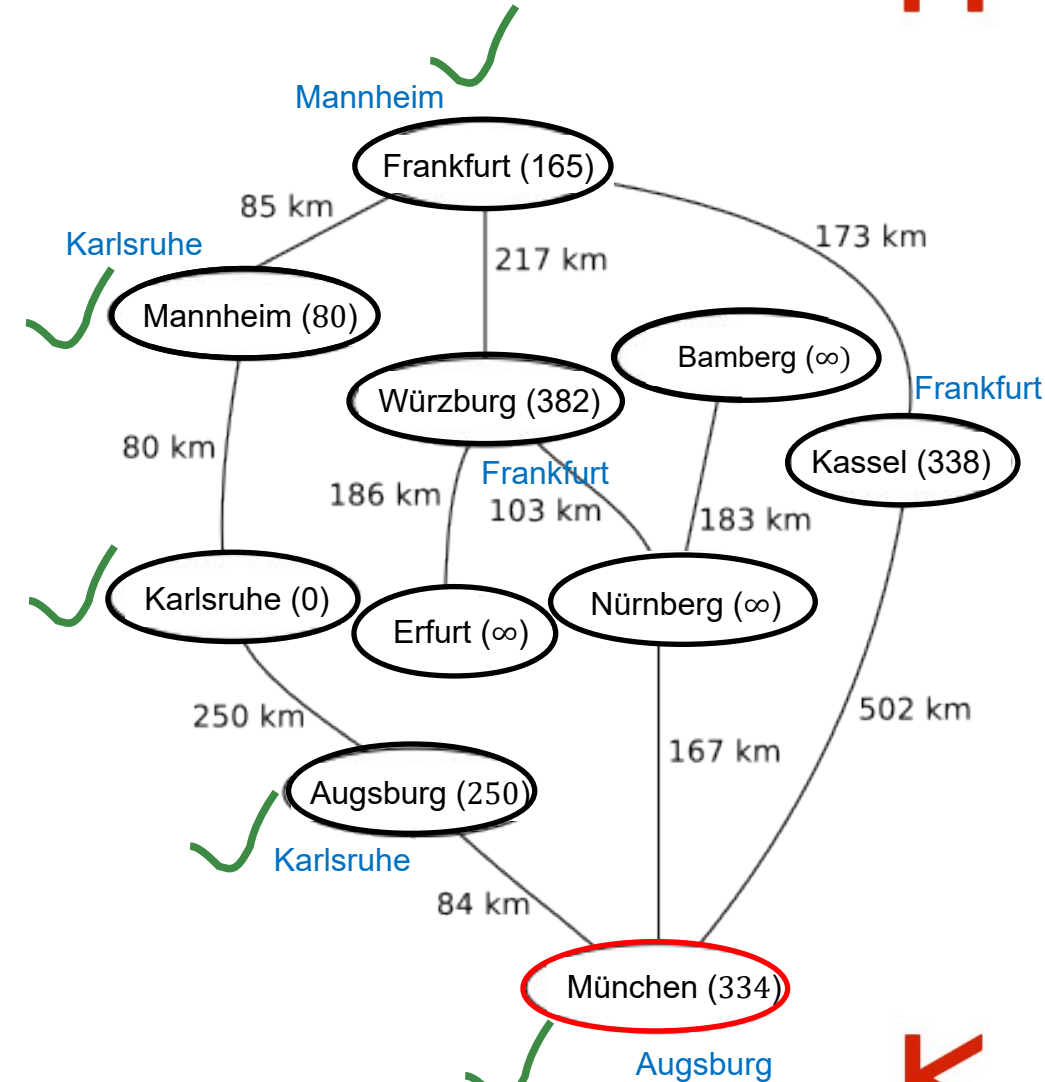
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



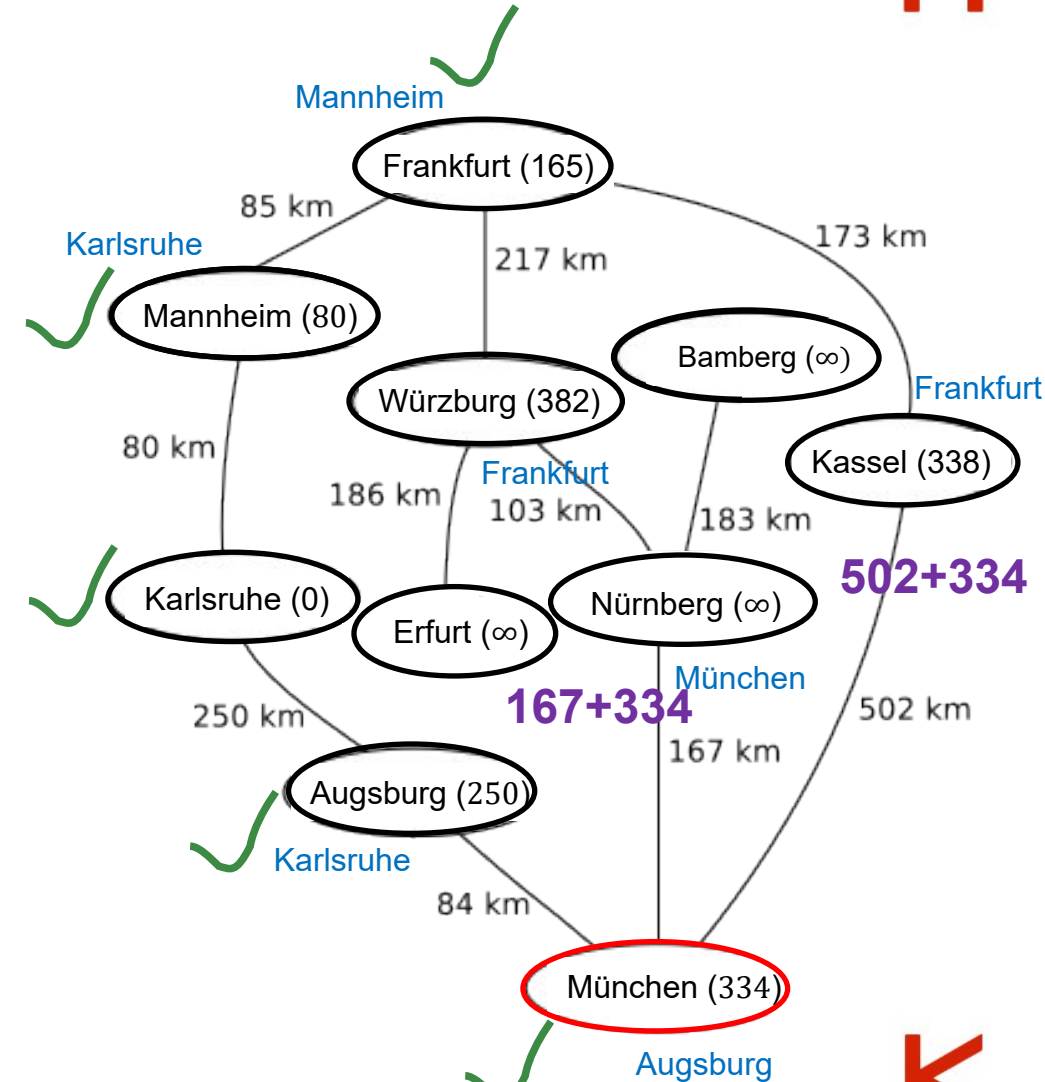
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.





- 502+334

KA



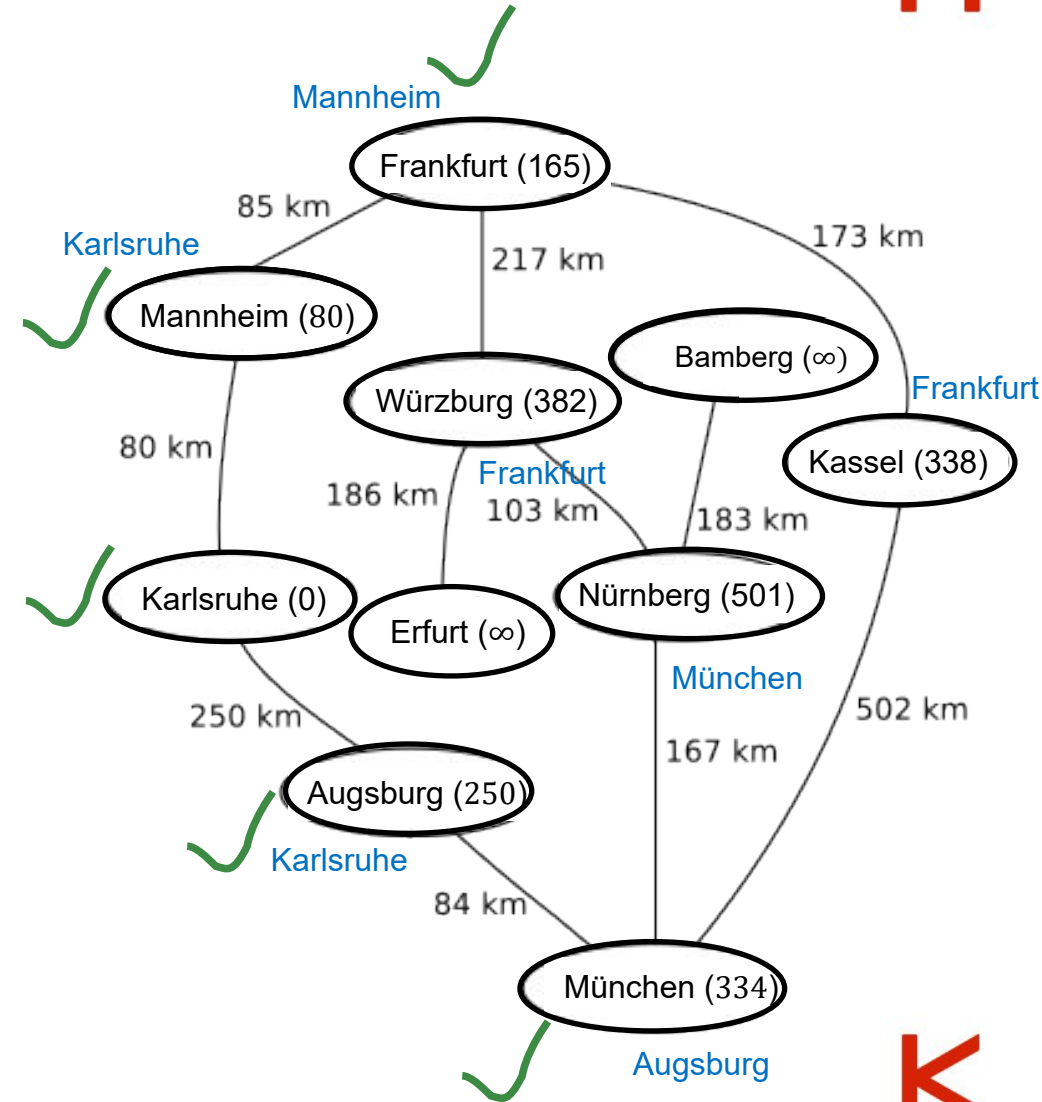
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach Kassel

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
 3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



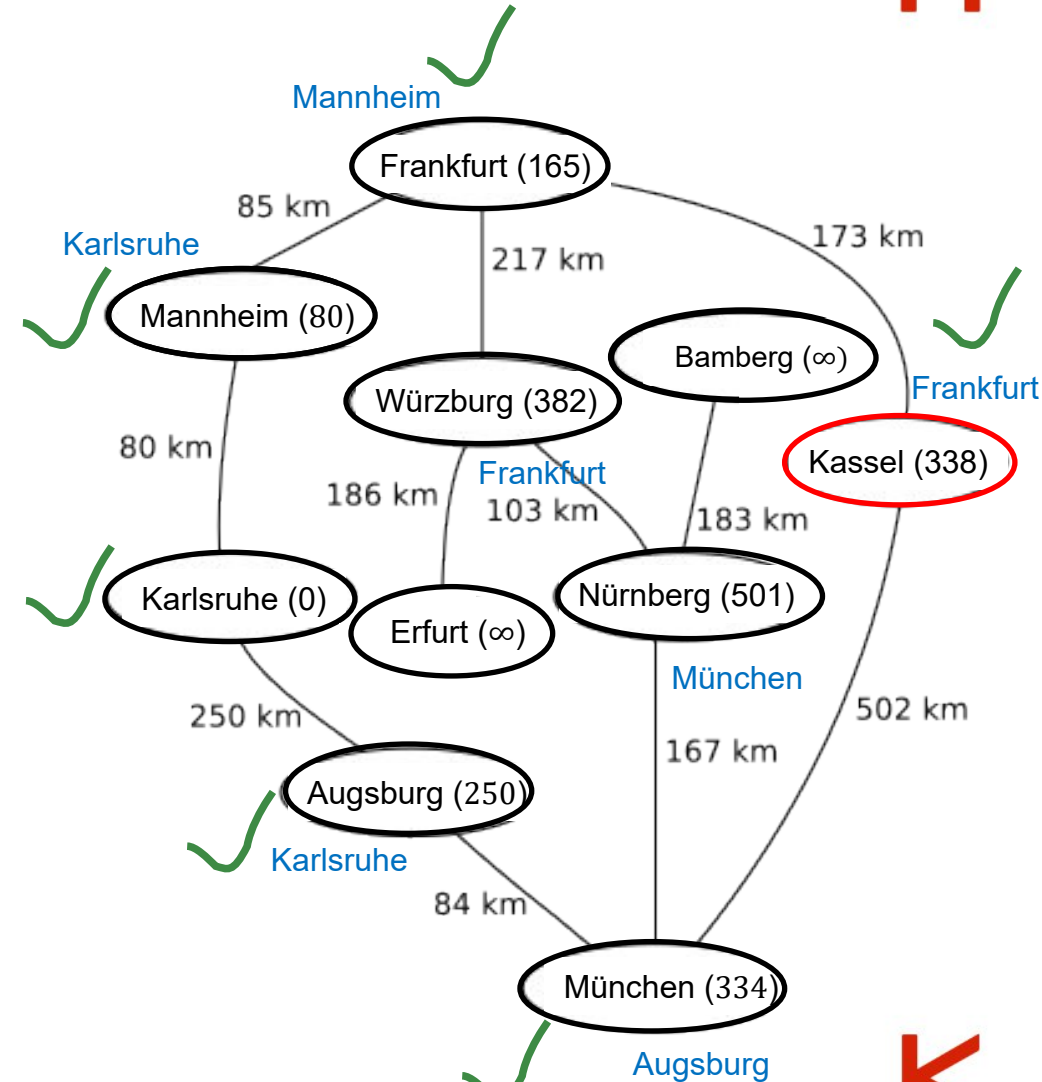
Dijkstra's algorithm (single-pair shortest path)



z.B. von Karlsruhe nach **Kassel**

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, **wähle denjenigen mit minimaler (aufsummierter) Distanz** aus
 1. Markiere diesen Knoten **als besucht**.
 2. **Berechne** für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als **Vorgänger**.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man **in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.**



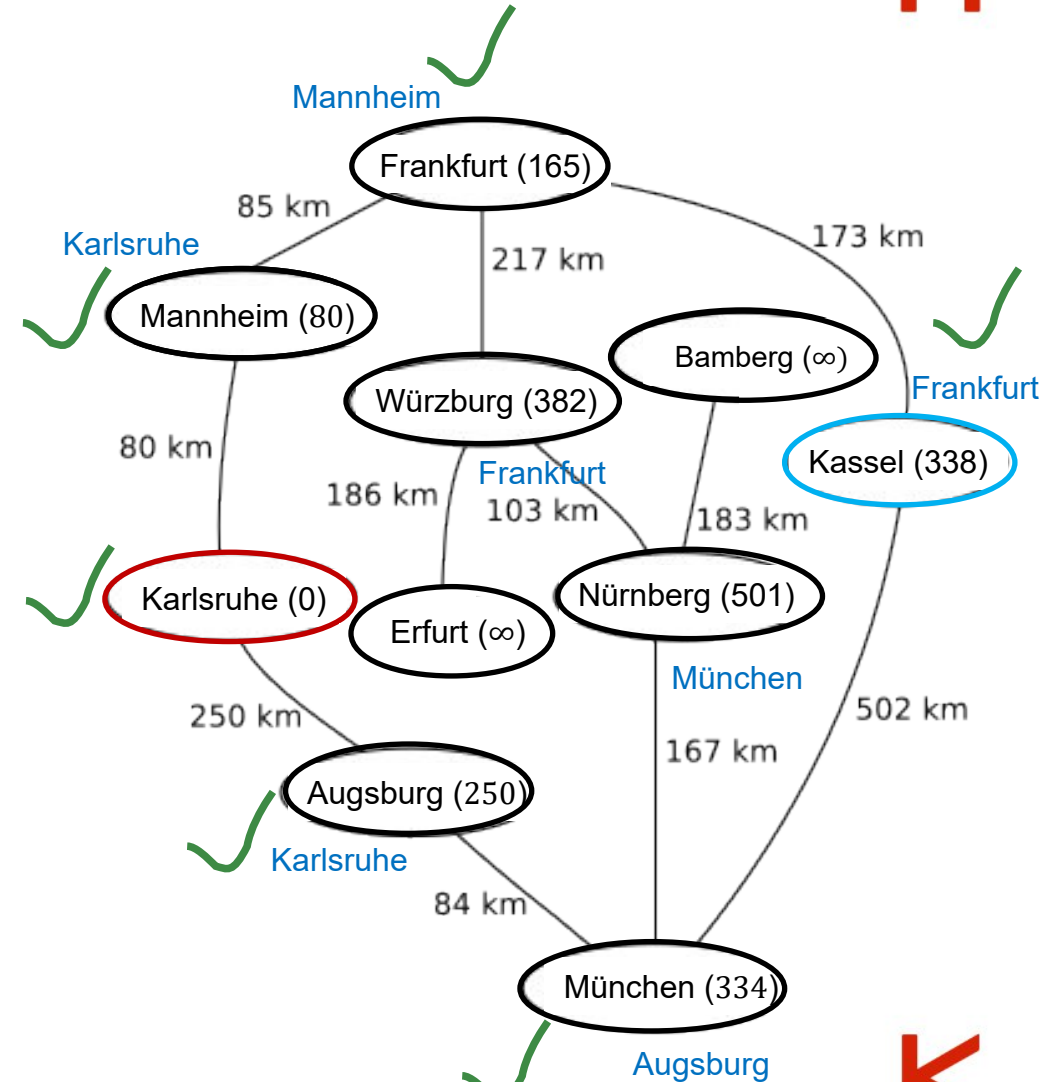
Dijkstra's algorithm (single-pair shortest path)



z.B. von **Karlsruhe** nach **Kassel**

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, wähle denjenigen mit minimaler (aufsummierter) Distanz aus
 1. Markiere diesen Knoten als besucht.
 2. Berechne für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als Vorgänger.

Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.



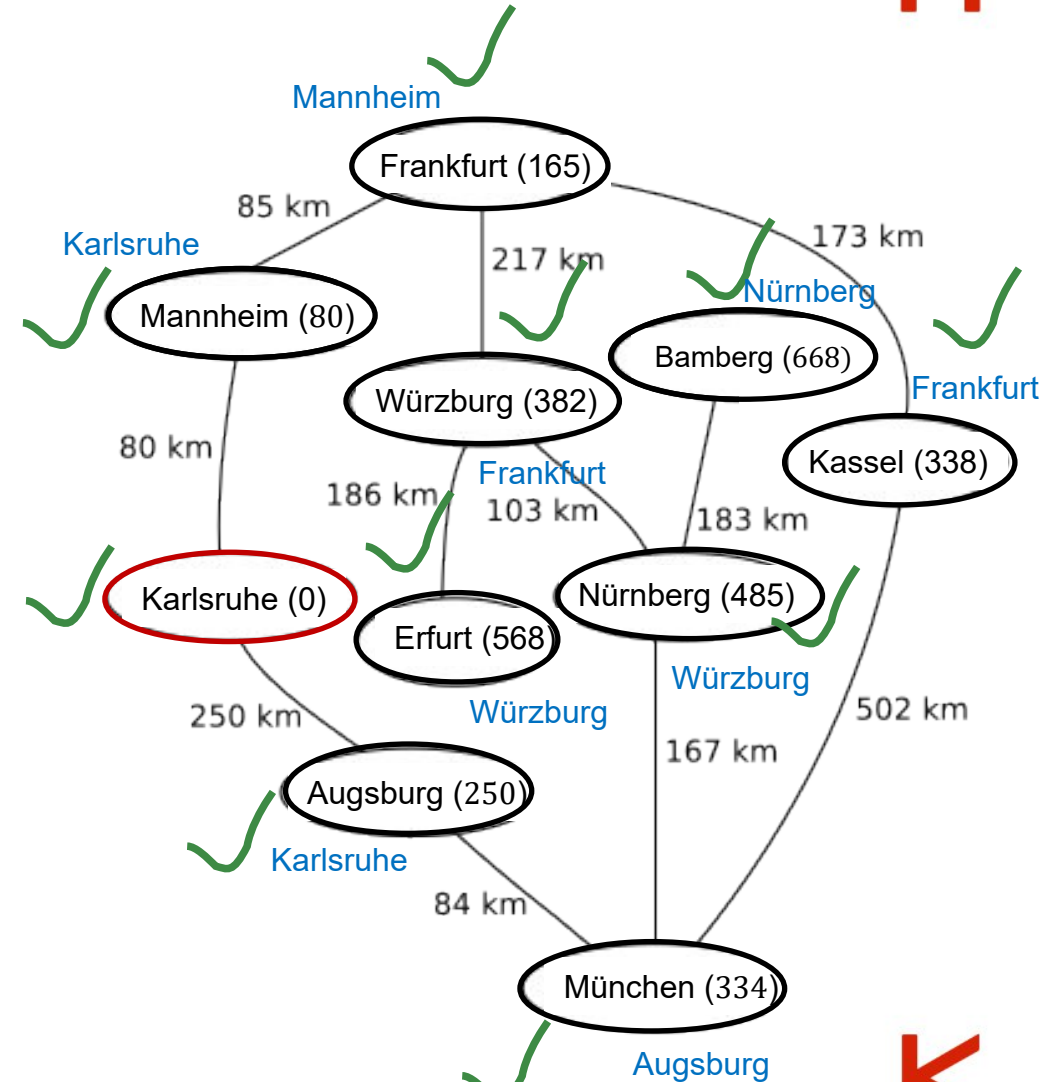
Dijkstra's algorithm (single-source shortest path)



z.B. von **Karlsruhe** zu allen anderen Orten

1. Die Knoten besitzen die Eigenschaften *Distanz* & *Vorgänger*.
Initialisiere: Distanz (Startknoten=0, ∞ alle anderen).
2. Solange es noch unbesuchte Knoten gibt, wähle denjenigen mit minimaler (aufsummierter) Distanz aus
 1. Markiere diesen Knoten als besucht.
 2. Berechne für alle noch unbesuchten Nachbarknoten die Gesamtdistanz des Pfades (Kantengewichtes + Distanz vom Startknoten zum aktuellen Knoten).
3. Relaxation: Ist dieser Wert für einen Knoten kleiner als die dort gespeicherte bisherige aufsummierte Distanz aktualisiere sie und setze den aktuellen Knoten als Vorgänger.

~~Ist man dagegen nur an dem Weg zu einem ganz bestimmten Zielknoten interessiert, so kann man in Schritt (2) abbrechen, wenn der gesuchte Knoten der aktive ist.~~





3. SDL Visualisierung

Visualisierung der Karte & Route



SDL: Initialisierung



```
if (!SDL_Init(SDL_INIT_VIDEO)) {  
    SDL_Log("Unable to initialize SDL: %s", SDL_GetError());  
}
```

- **SDL_Init(flags)** startet SDL-Subsysteme
- **SDL_INIT_VIDEO**: “video subsystem; automatically initializes the events subsystem, should be initialized on the main thread.”
- Am Ende des Programms **SDL_Quit()** ausführen.

SDL: Fenster erstellen



```
//Parameter für Fenster
const char* title = "My SDL Window";
int width = 800;
int height = 600;
Uint32 flags = SDL_WINDOW_RESIZABLE;

//Fenster erstellen
SDL_Window* window = SDL_CreateWindow(title, width, height, flags);
if (NULL == window) {
    SDL_Log("Unable to create window: %s", SDL_GetError());
}

//Fenster am Ende wieder schließen
SDL_DestroyWindow(window);
```




```
//Renderer erstellen
SDL_Renderer* renderer = SDL_CreateRenderer(window, nullptr);
if (NULL == renderer) {
    SDL_Log("Unable to create renderer: %s", SDL_GetError());
}

//Am Ende Renderer wieder löschen
SDL_DestroyRenderer(renderer);
```

- Der Renderer wird für 2D-Grafikausgabe genutzt
- Nutzt Grafikkarte über OpenGL, Direct3D oder Vulkan (sonst Software-Implementation)

SDL: Renderer Funktionen

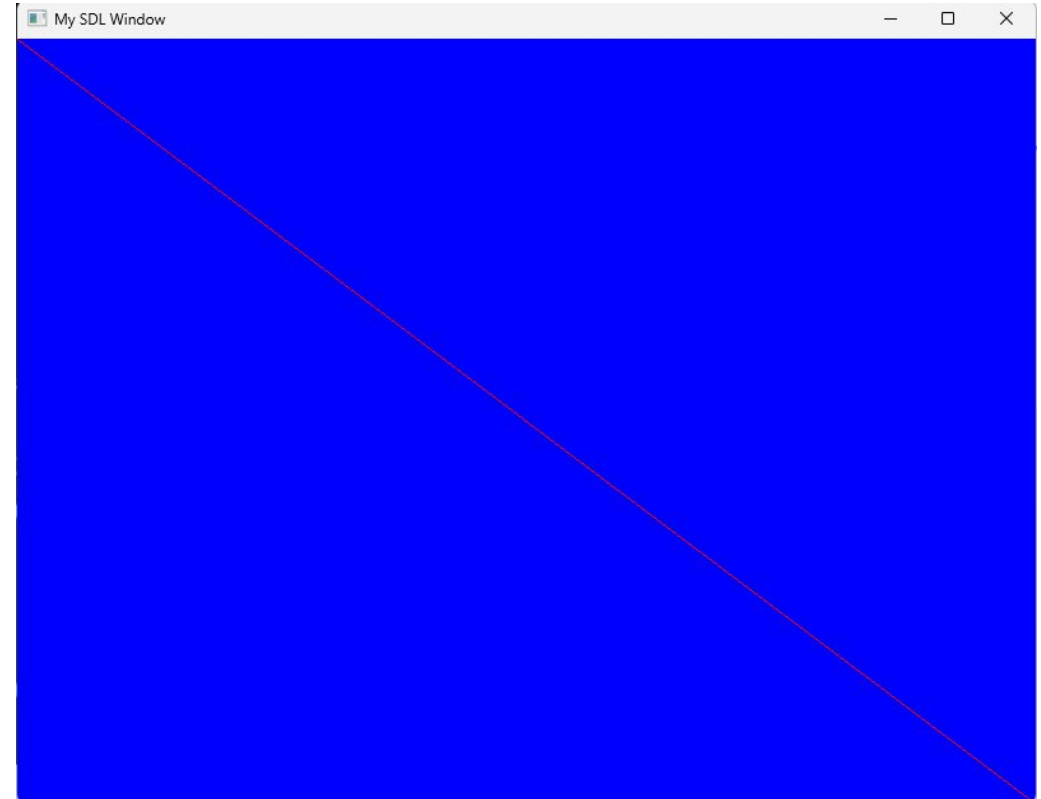


```
//Farbe für den Renderer setzen
SDL_SetRenderDrawColor(renderer, 0, 0, 255, 255); //RGBA -> Blau

//Fläche mit der Farbe füllen
SDL_RenderClear(renderer);

//Linie zeichnen (oben links nach unten rechts)
SDL_SetRenderDrawColor(renderer, 255, 0, 0, 255); //RGBA -> Rot
SDL_RenderLine(renderer, 0, 0, width, height);

//Darstellen
SDL_RenderPresent(renderer);
```



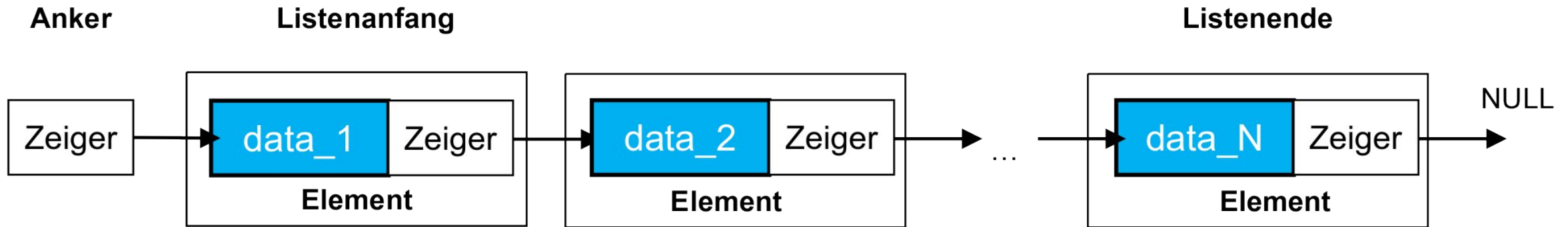


C++

Wiederholung: Listenstruktur – Dynamisch!



Eine **verkettete Liste** ist eine **dynamische** Datenstruktur, in der Datenelemente vorzugsweise **geordnet** gespeichert sind.



Wiederholung: dynamische Arrays



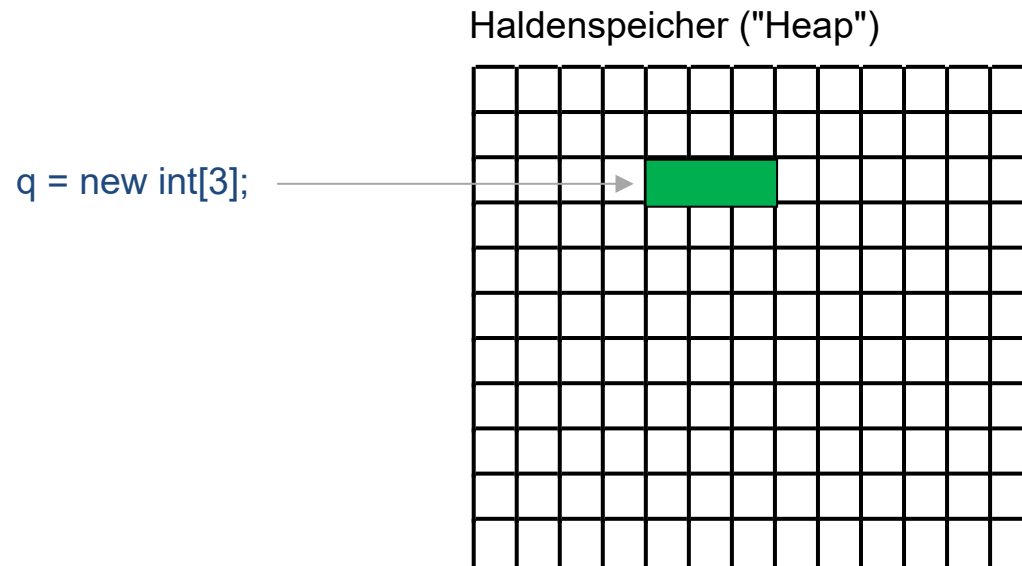
Dynamische Datenstruktur für Felder, bei denen die Größe zur Laufzeit des Programms festgelegt wird.

```
// Einfacher Zeiger
int* p = NULL;
p = new int;
*p = 42;
cout << "p = " << *p << endl;

// Zeiger / Feld Dualitaet
int* q = new int[3];
q[0] = 4; q[1] = 2; q[2] = 1;
cout << "sum of q = "
    << q[0] + q[1] + q[2]
    << endl;

// Freigabe
delete p;
delete[] q;

// Fehler!
cout << "(free'd) p = " << *p << endl;
```



C++ Standardtyp vector



Bei C++ gibt es (im Gegensatz zu C) einen dynamischen Datentyp für eine eindimensionale Tabelle

```
vector<int> v(10);           // runde Klammern

vector<int> v1 {};           // leerer Vektor
vector<int> v2 {7, 0, 9};    // Vektor mit den Elementen 7, 0, 9
```



C++ Standardtyp vector



Bei C++ gibt es (im Gegensatz zu C) einen dynamischen Datentyp für eine eindimensionale Tabelle

```
vector<int> v(10);           // runde Klammern

vector<int> v1 {};           // leerer Vektor
vector<int> v2 {7, 0, 9};    // Vektor mit den Elementen 7, 0, 9
```

Größe über size bestimmbar:

```
cout << v.size() << '\n';    // 10
cout << v1.size() << '\n';    // 0
cout << v2.size() << '\n';    // 3
```



C++ Standardtyp vector



Bei C++ gibt es (im Gegensatz zu C) einen dynamischen Datentyp für eine eindimensionale Tabelle

```
vector<int> v(10); // runde Klammern

vector<int> v1 {}; // leerer Vektor
vector<int> v2 {7, 0, 9}; // Vektor mit den Elementen 7, 0, 9
```

Größe über size bestimmbar:

```
cout << v.size() << '\n'; // 10
cout << v1.size() << '\n'; // 0
cout << v2.size() << '\n'; // 3
```

Werte:

```
cout << v[0] << '\n'; // Zählung beginnt bei 0
cout << v.at(0) << '\n'; // alles bestens, dasselbe wie v[0]
```



C++ Standardtyp vector



Bei C++ gibt es (im Gegensatz zu C) einen dynamischen Datentyp für eine eindimensionale Tabelle

```
vector<int> v(10);           // runde Klammern

vector<int> v1 {};           // leerer Vektor
vector<int> v2 {7, 0, 9};    // Vektor mit den Elementen 7, 0, 9
```

Größe über size bestimmbar:

```
cout << v.size() << '\n';    // 10
cout << v1.size() << '\n';   // 0
cout << v2.size() << '\n';   // 3
```

Werte:

Überprüfung der Bereichsüber- oder -unterschreitung!

```
cout << v[0] << '\n';        // Zählung beginnt bei 0
cout << v.at(0) << '\n';     // alles bestens, dasselbe wie v[0]
```



C++ Standardtyp vector



Bei C++ gibt es (im Gegensatz zu C) einen **dynamischen** Datentyp für eine eindimensionale Tabelle

```
vector<int> v(10); // runde Klammern
```

dynamisches Anhängen von Elementen:

```
v.push_back(99);  
cout << v.size() << endl; // 11
```



C++ Standardtyp vector



Bei C++ gibt es (im Gegensatz zu C) einen **dynamischen** Datentyp für eine eindimensionale Tabelle

```
vector<int> v(10); // runde Klammern
```

dynamisches Anhängen von Elementen:

```
v.push_back(99);  
cout << v.size() << endl; // 11
```

dynamisches Löschen von Elementen

```
v.pop_back(); // löschen des letzten Elements  
cout << v.size() << endl; // 10
```



C++ Standardtyp vector



Bei C++ gibt es (im Gegensatz zu C) einen **dynamischen** Datentyp für eine eindimensionale Tabelle

```
vector<int> v(10); // runde Klammern
```

dynamisches Anhängen von Elementen:

```
v.push_back(99);  
cout << v.size() << endl; // 11
```

dynamisches Löschen von Elementen

```
v.pop_back(); // löschen des letzten Elements  
cout << v.size() << endl; // 10  
  
v.erase(v.begin()); // löschen des ersten Elements  
v.erase(v.begin()+5); // löschen des sechsten Elements  
v.erase(v.begin(),v.begin()+3); // löschen der ersten 3 Element
```



Code aus der Vorlesung



```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> MeineListe;
    cout << "Anzahl Elemente meiner Liste: " << MeineListe.size() << endl;
    MeineListe.push_back(10);
    MeineListe.push_back(11);
    MeineListe.at(0) = 1;
    cout << "Anzahl Elemente meiner Liste: " << MeineListe.size() << endl;
    for (int jj = 0; jj < MeineListe.size(); jj++) cout << "Element " << jj
        << " ist " << MeineListe.at(jj) << endl;
    MeineListe.push_back(13);
    MeineListe.push_back(14);
    int n;
    cin >> n;
    for (int jj = 0; jj < n; jj++) MeineListe.push_back(jj);
    cout << "Anzahl Elemente meiner Liste: " << MeineListe.size() << endl;
    MeineListe.erase(MeineListe.begin()+5);
    cout << "Anzahl Elemente meiner Liste: " << MeineListe.size() << endl;
    for (int jj = 0; jj < MeineListe.size(); jj++) cout << "Element " << jj
        << " ist " << MeineListe.at(jj) << endl;
}
```



Code aus der Vorlesung



```
#include <iostream>
#include <vector>
using namespace std;

struct Node {
    unsigned long long id;
    float lon, lat;
    // verkettete Liste von Nachfolgeknoten
};

int main() {
    vector<Node*> MeineListevonKnoten;
}
```

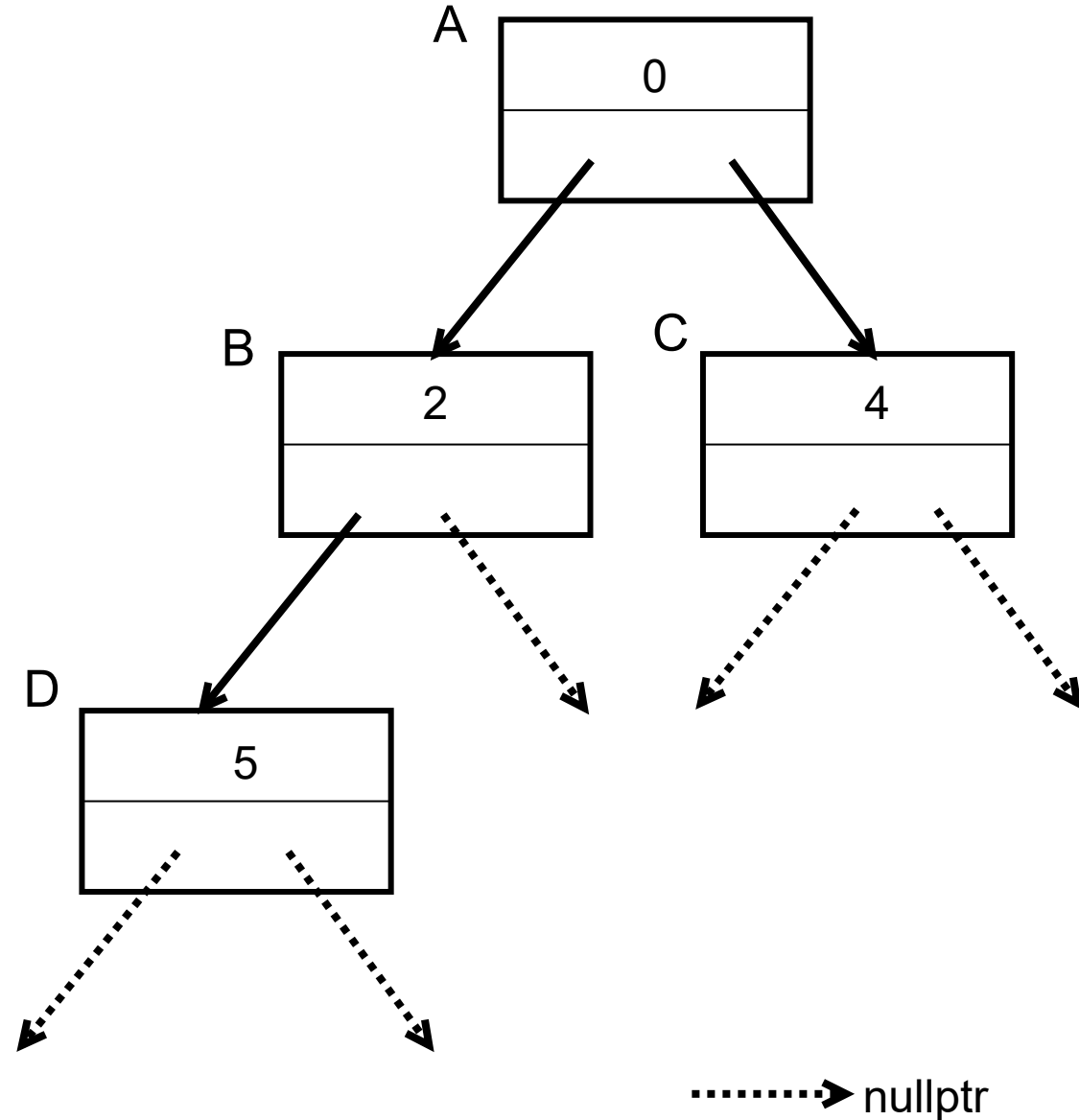
Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;

struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
}
```



Rekursive Baumsuche/-ausgabe



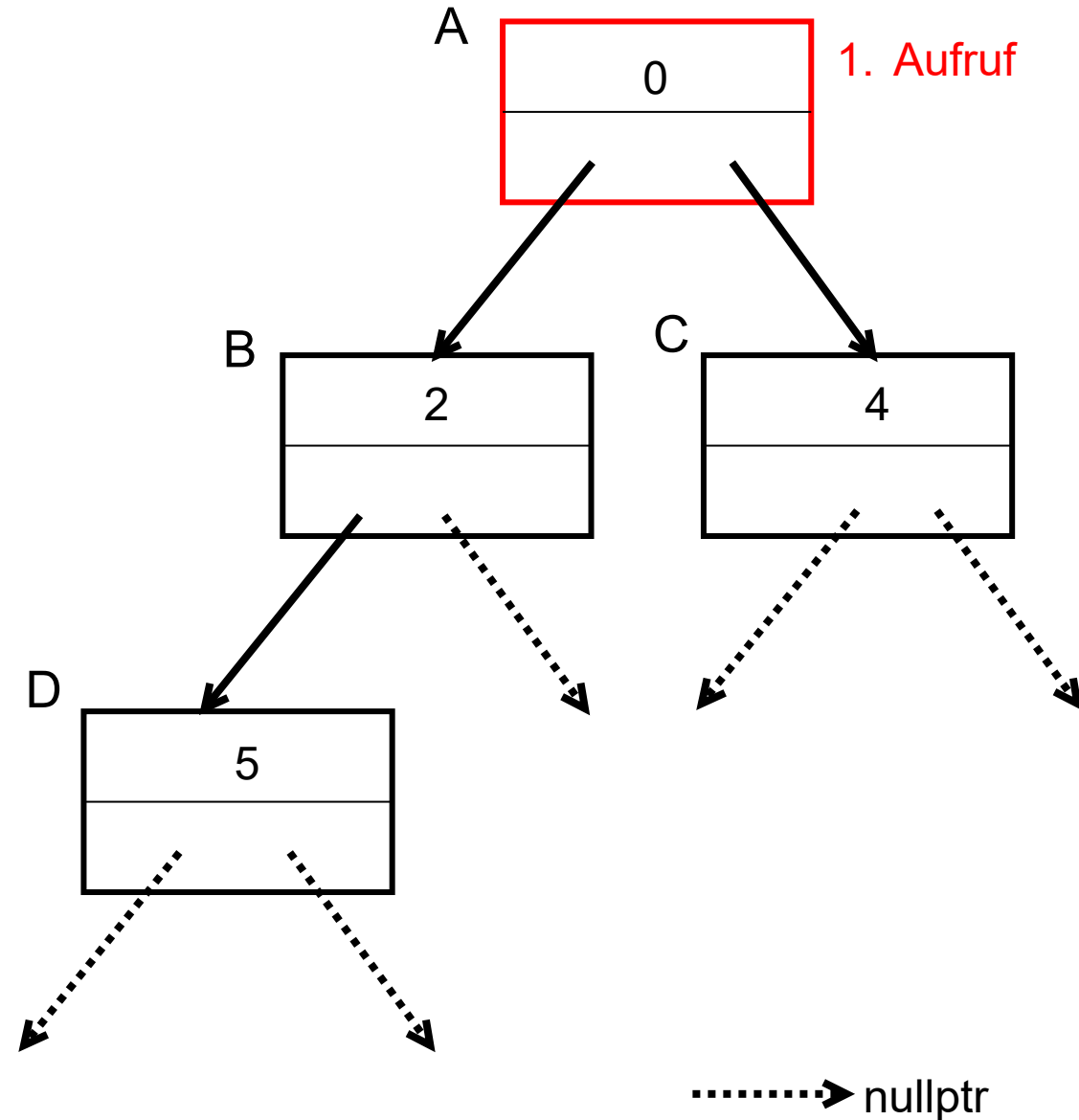
```
#include <iostream>
using namespace std;

struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

void Baumausgabe(Node* wobinich) {

}

int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

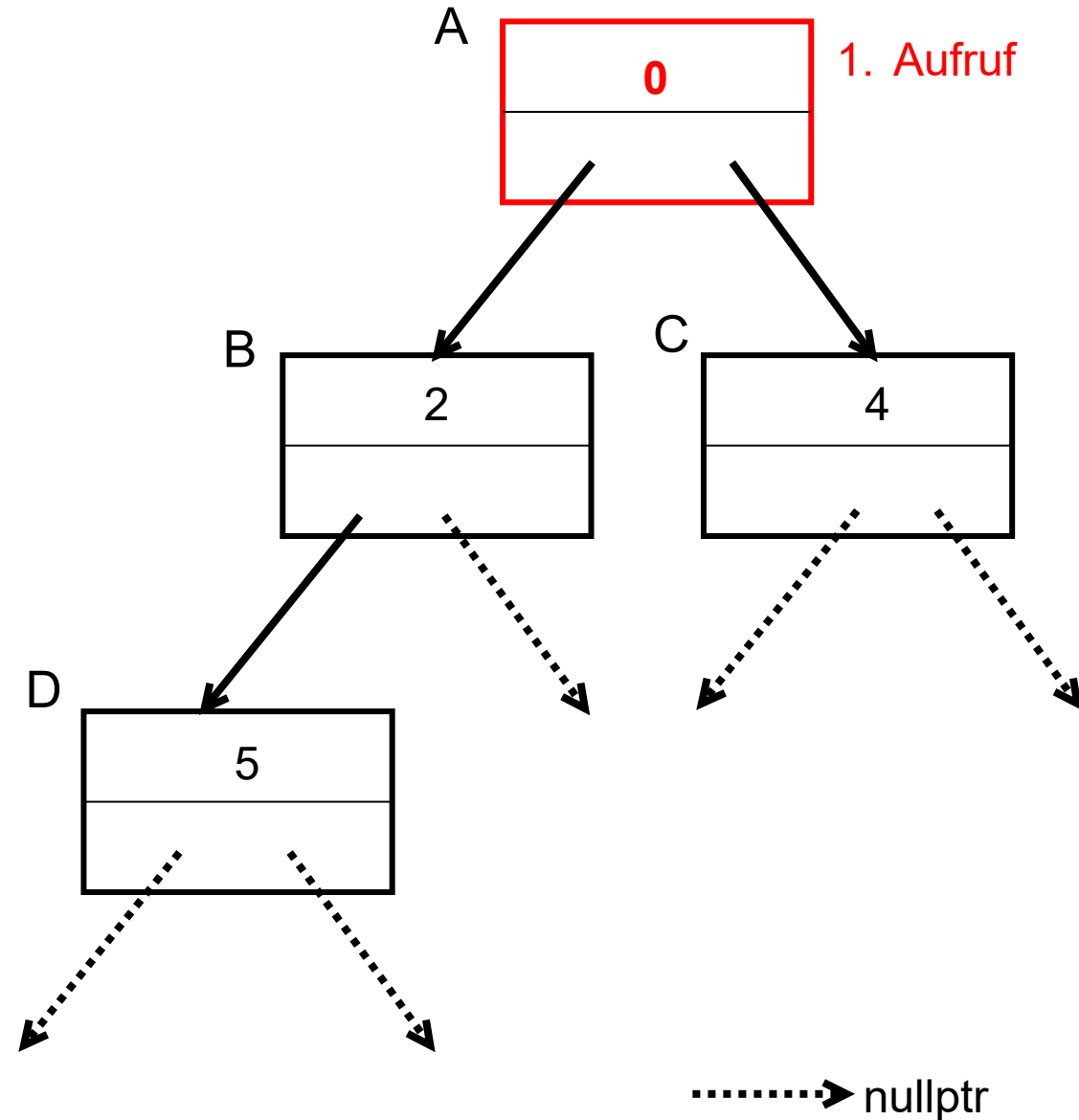


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
    }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

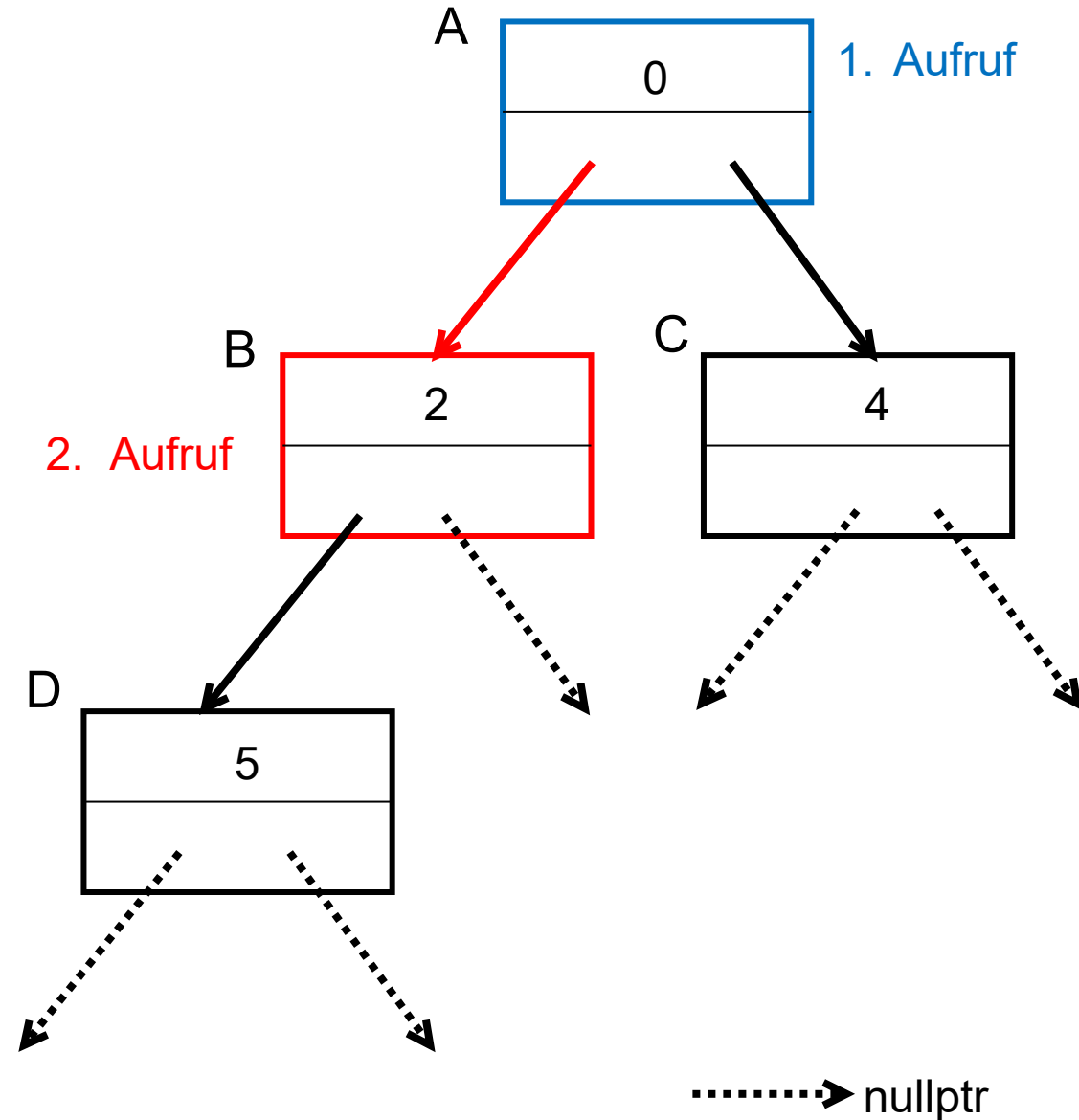


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

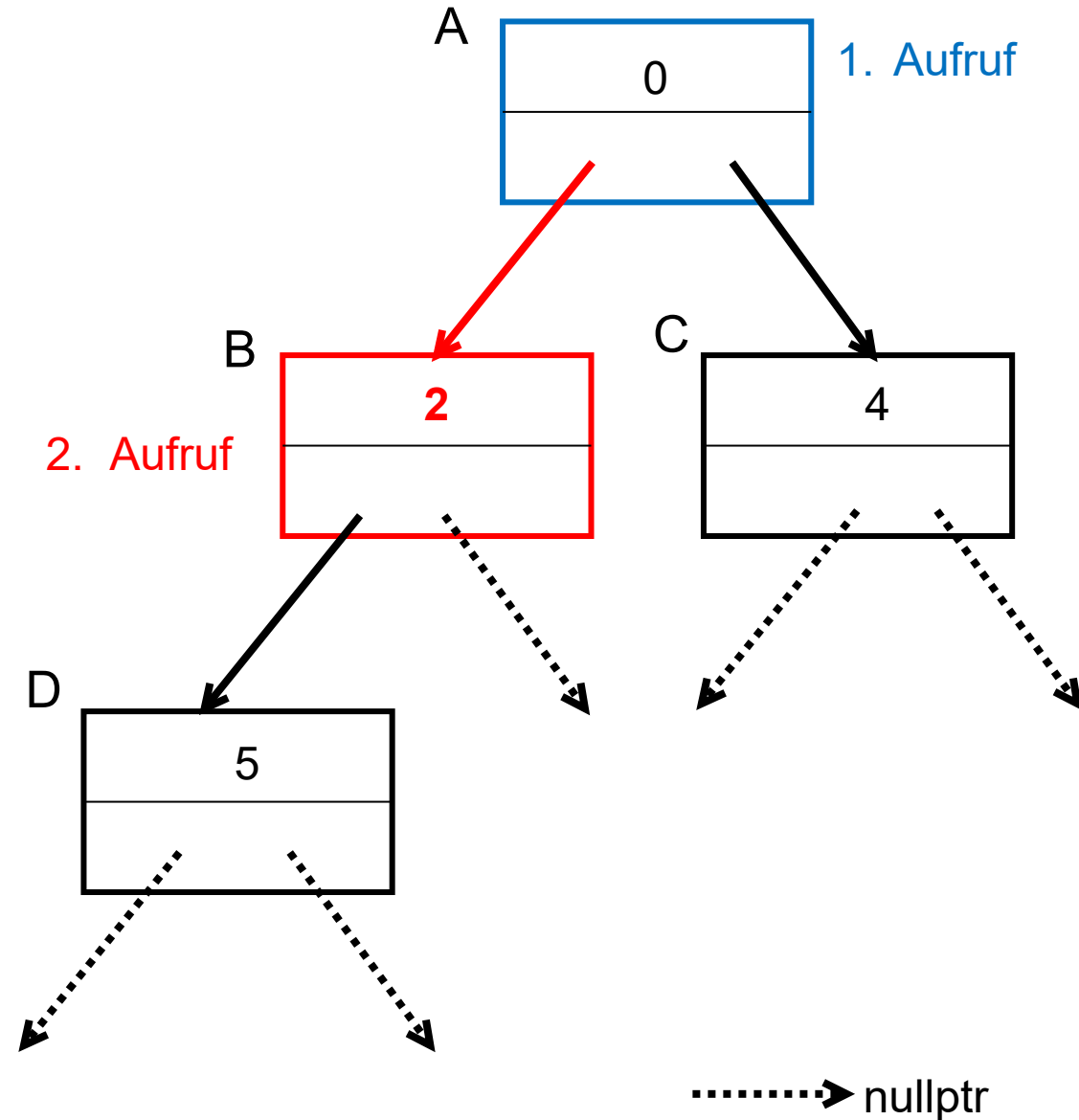


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

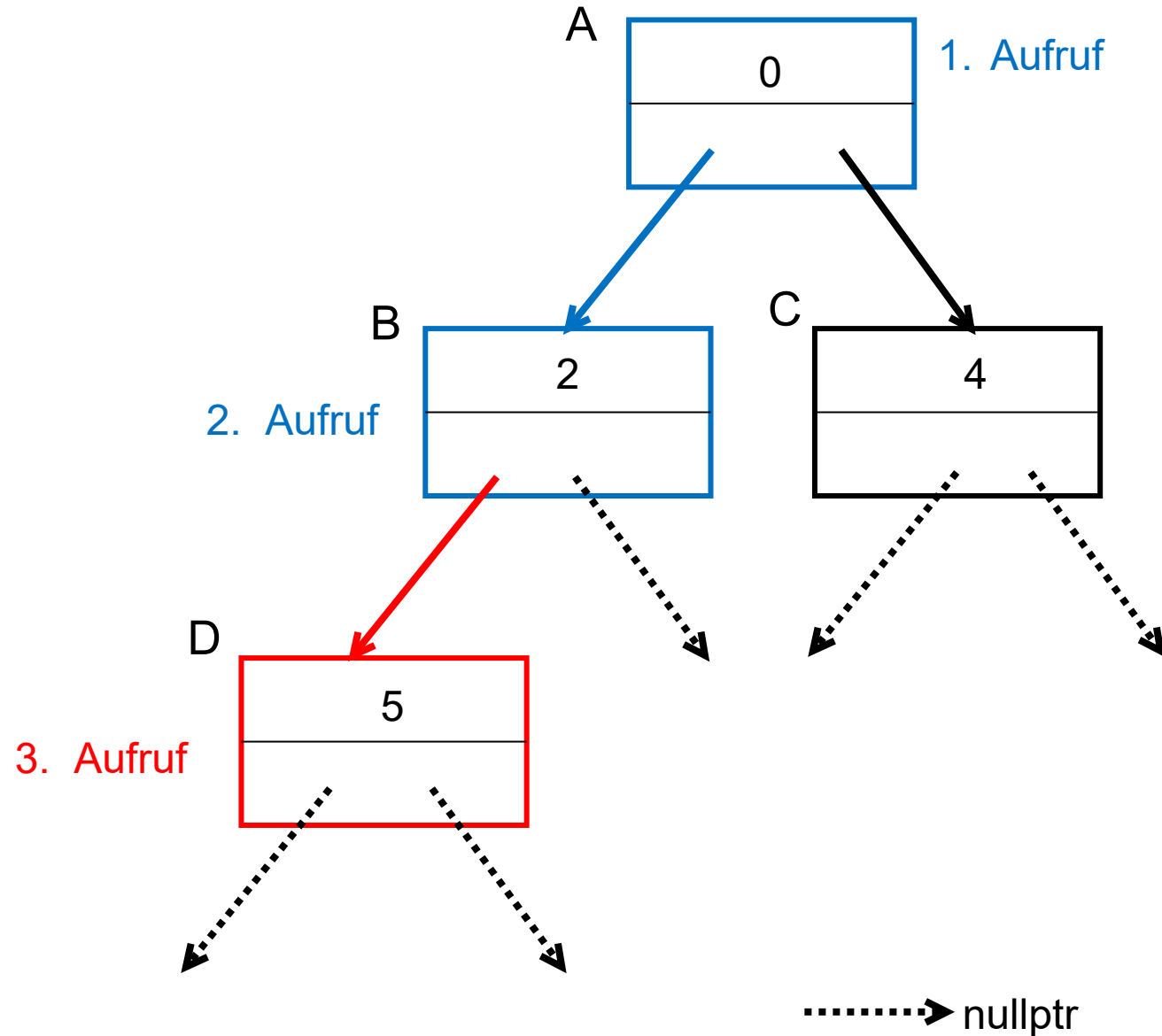


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

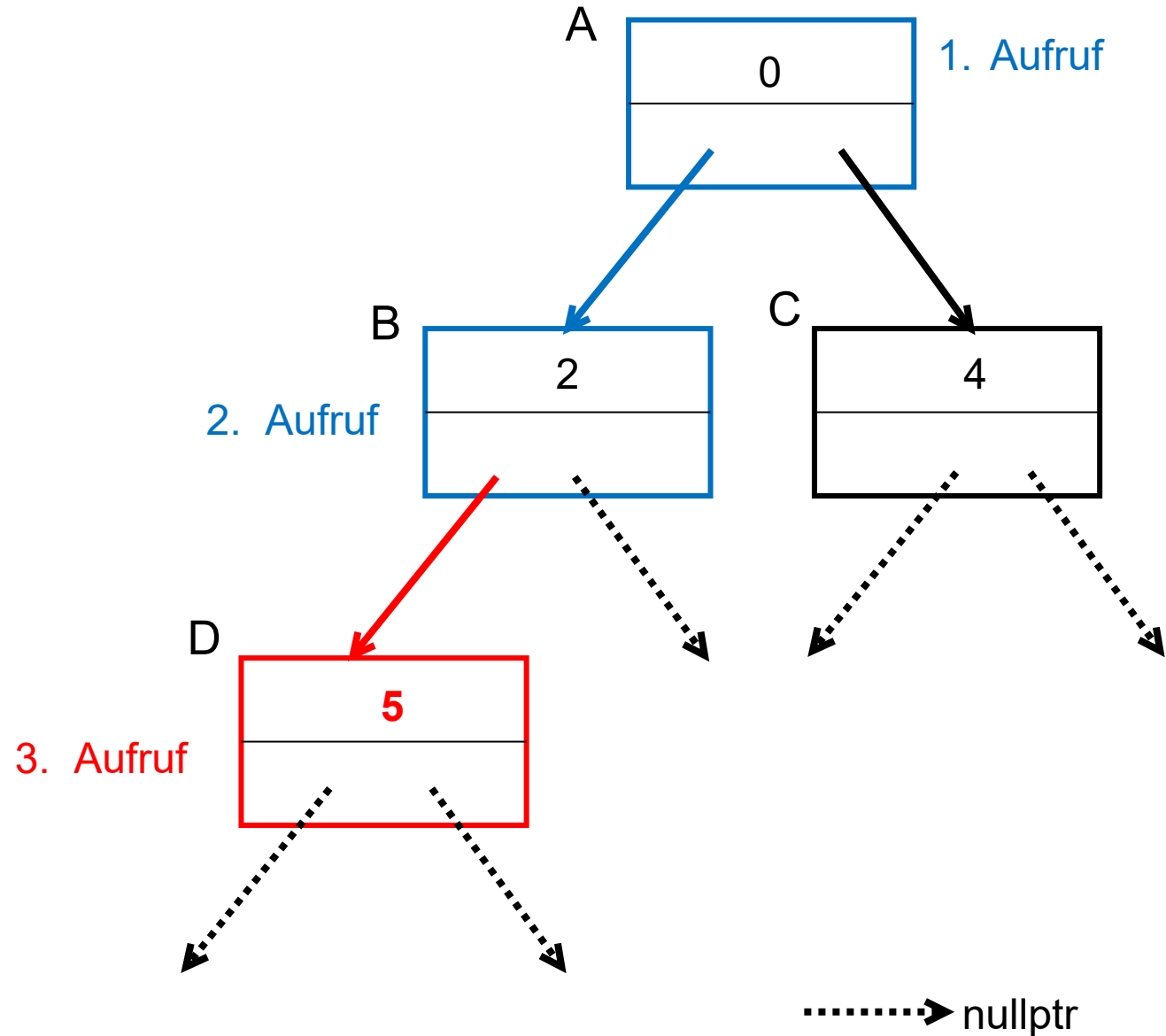


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

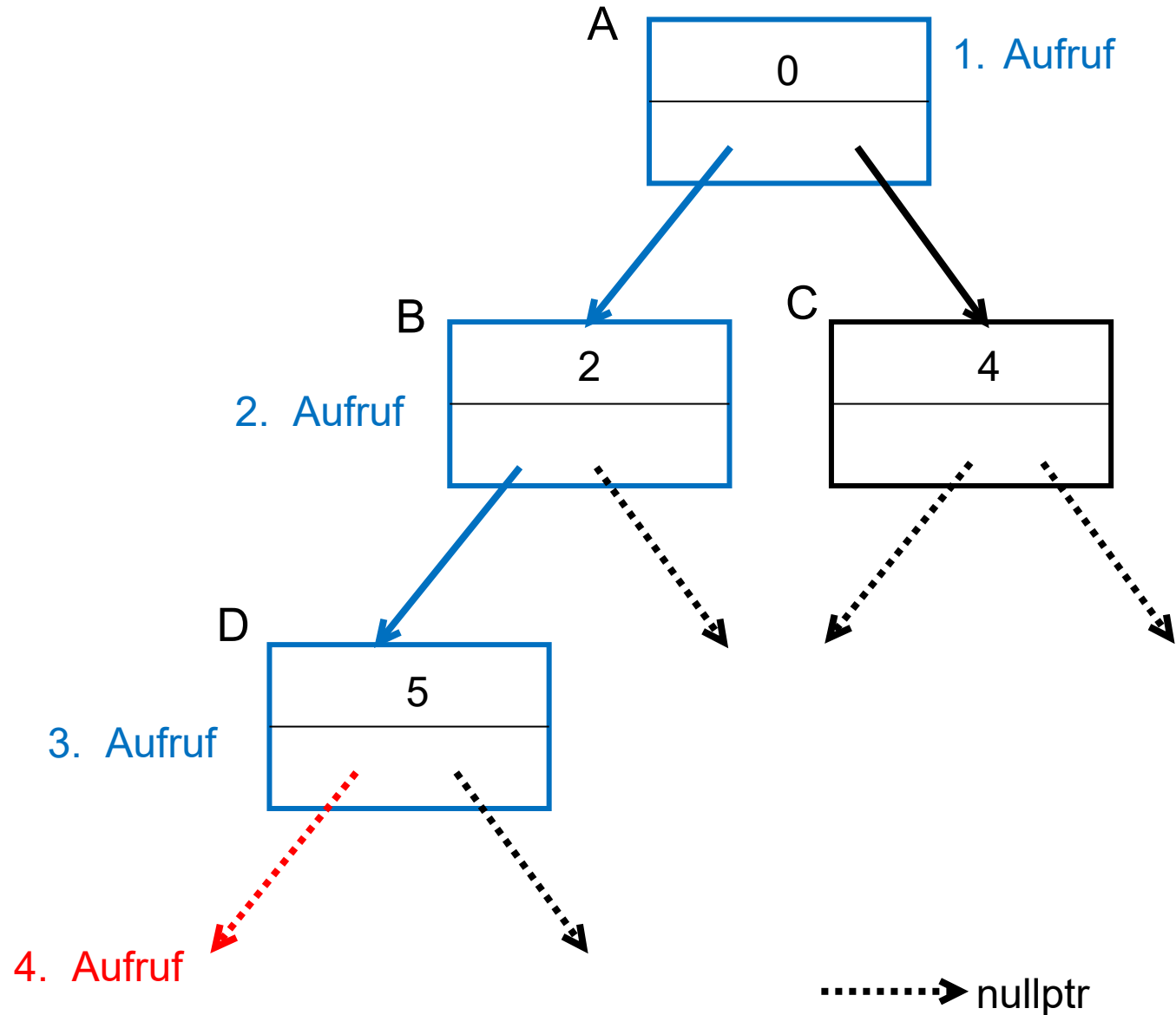


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

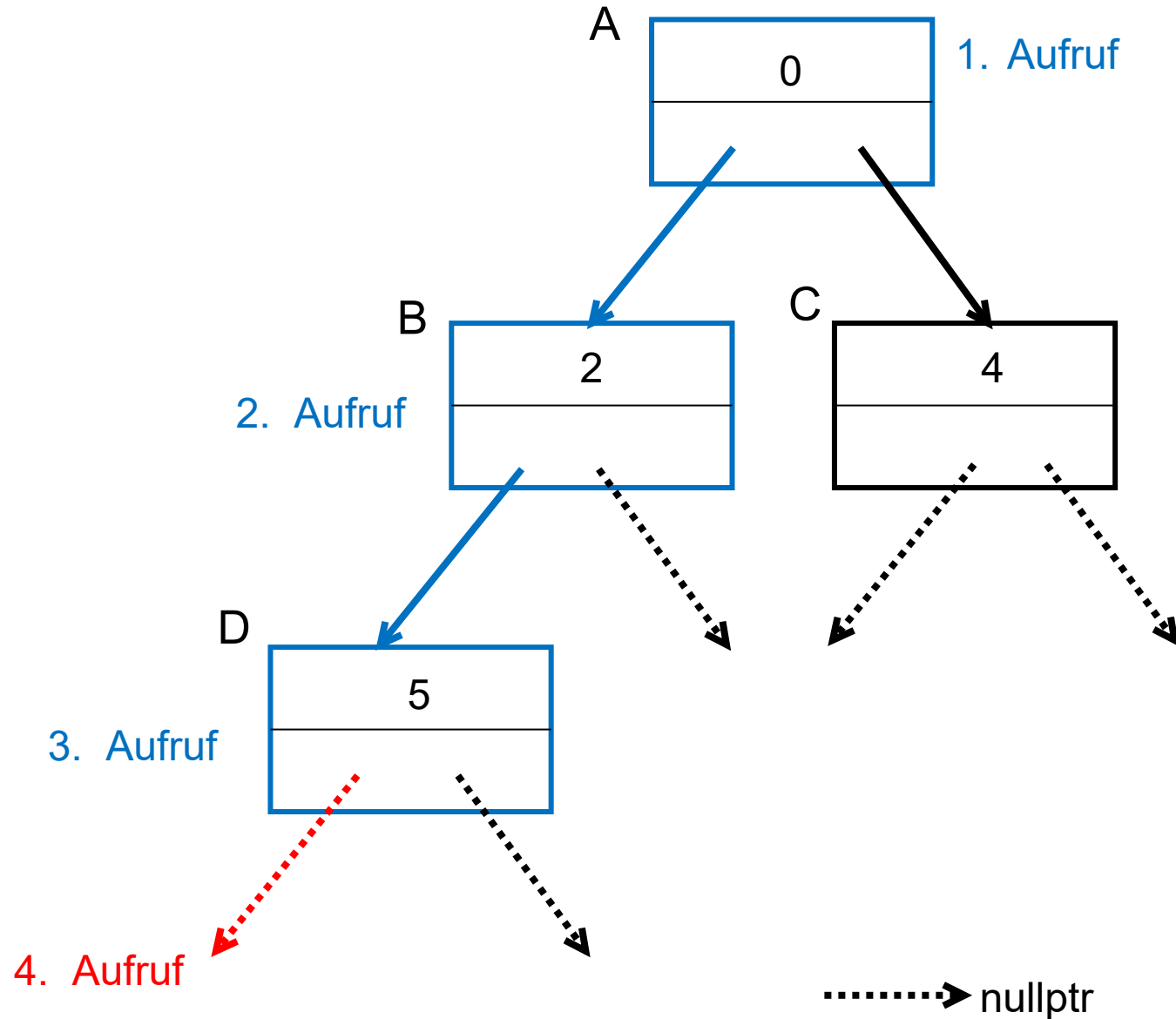
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) { ←
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}

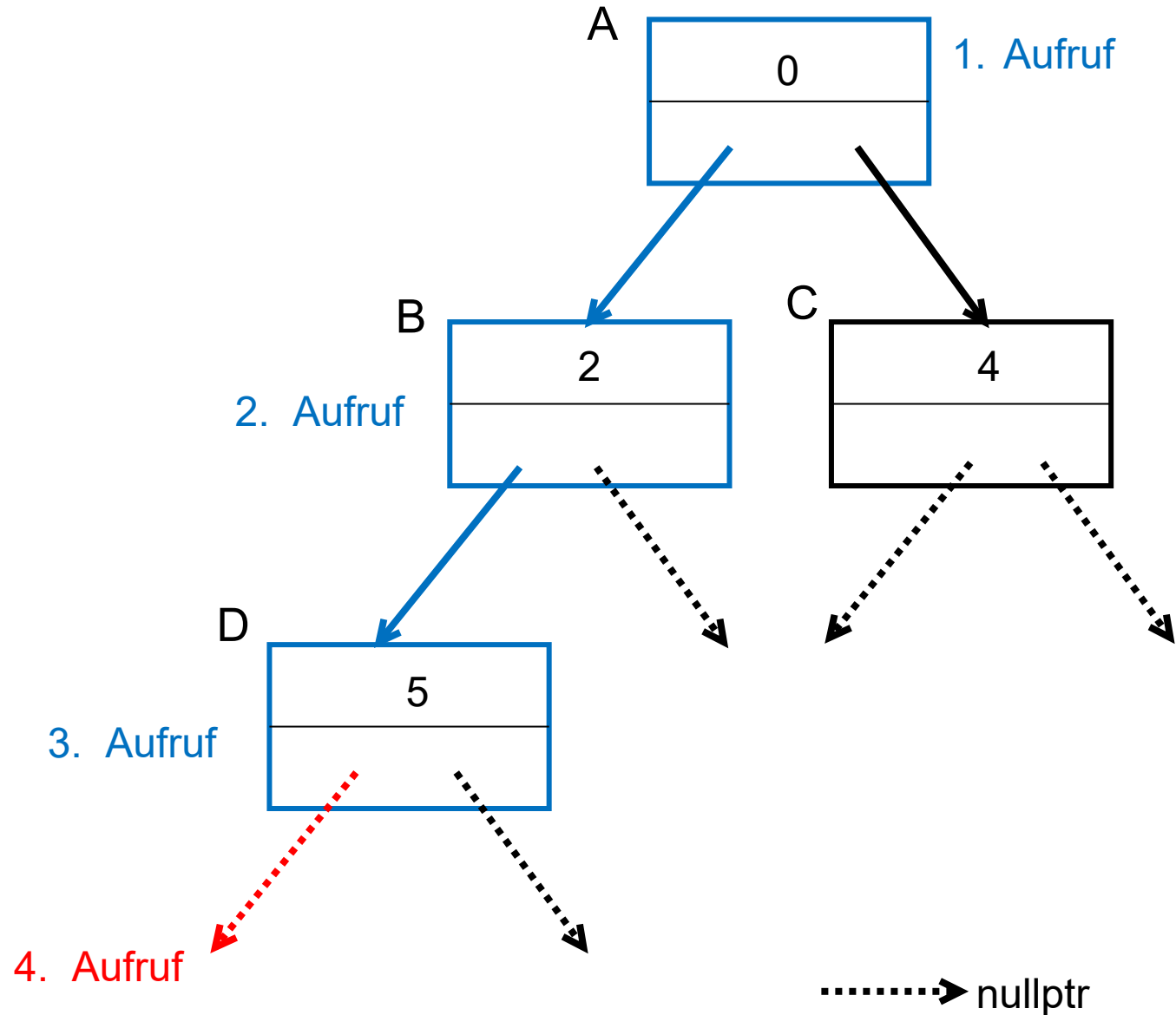
```

else { return; } ←

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

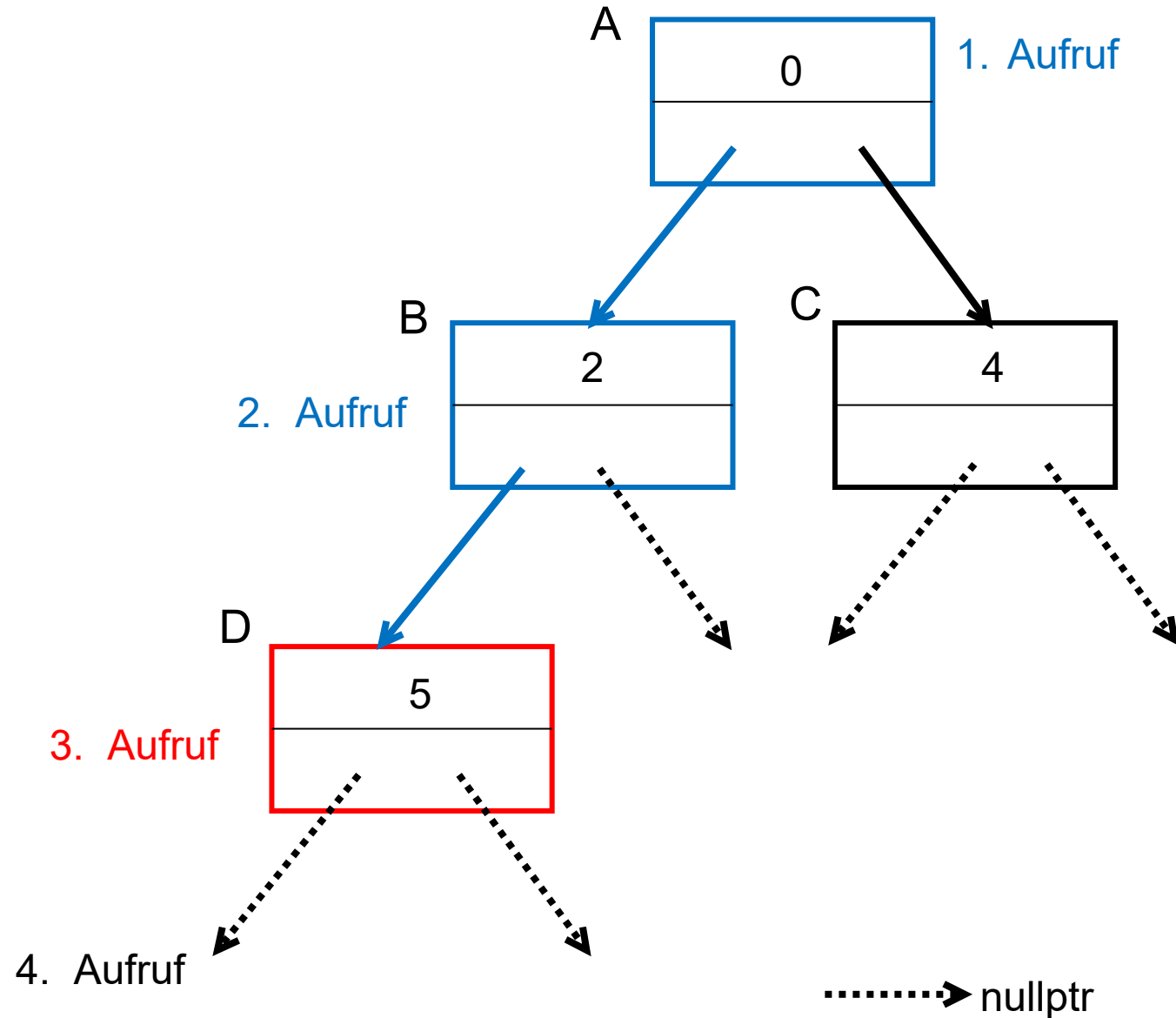
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}

```

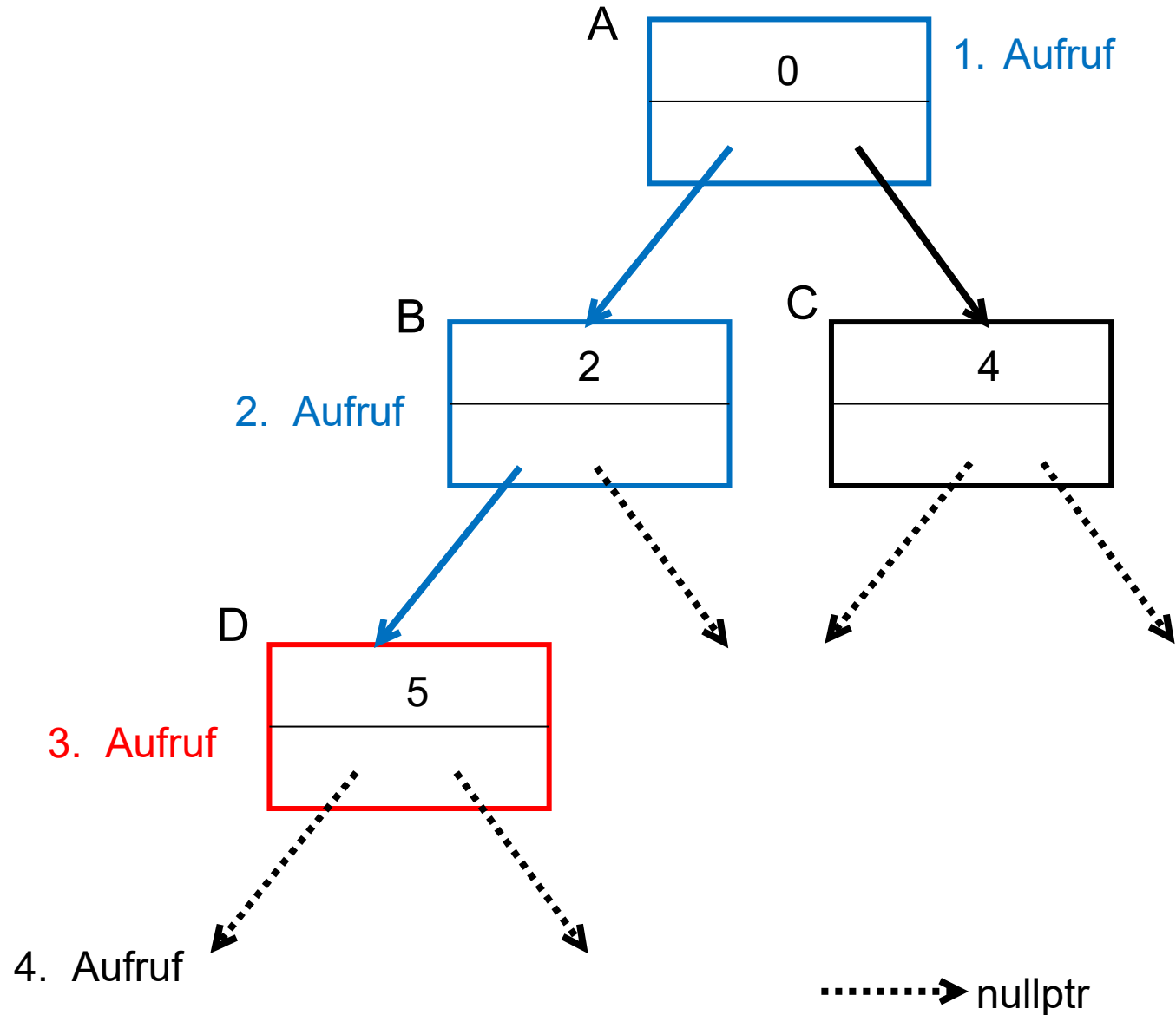
```
else { return; }

```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}

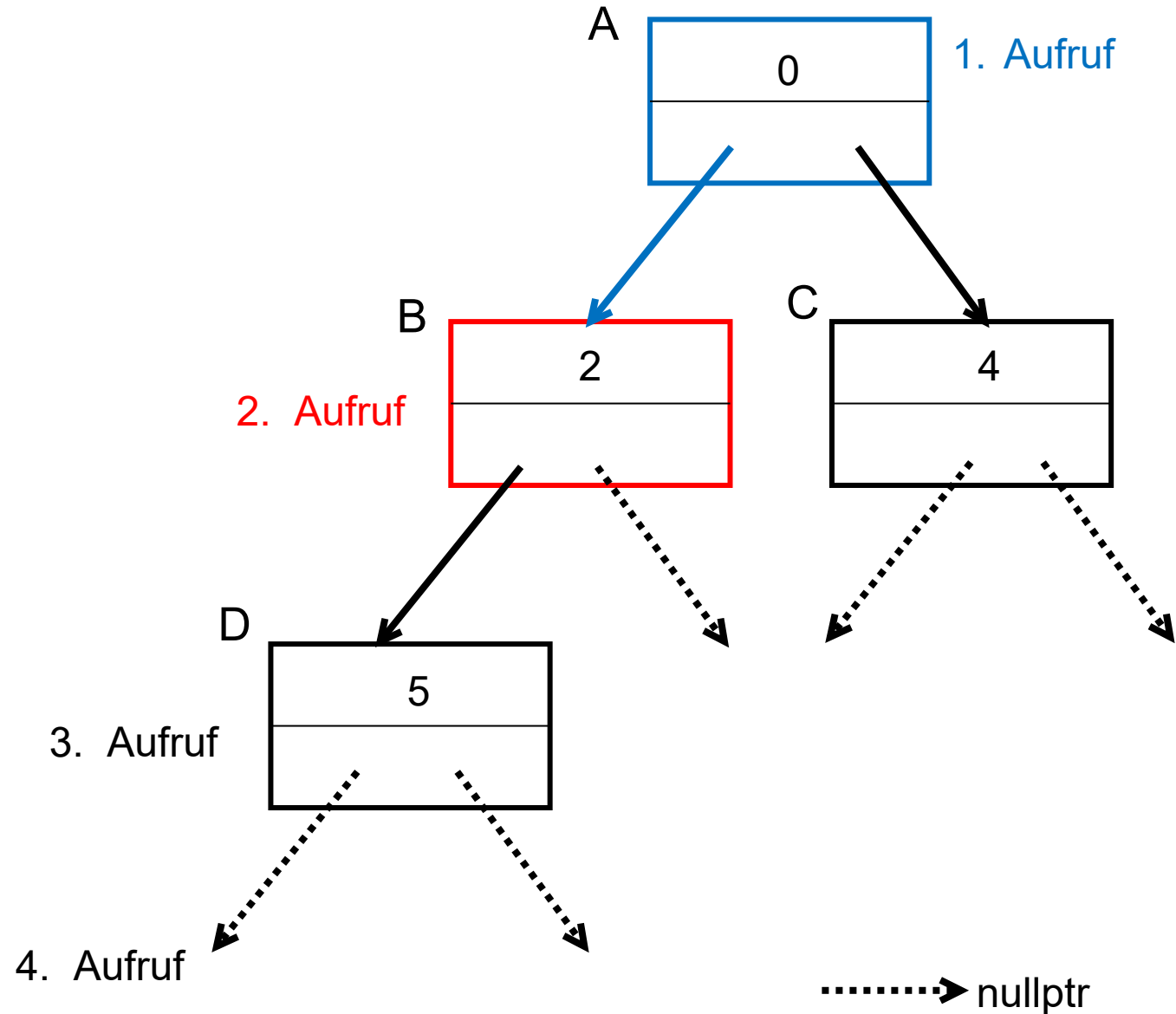
```

```
else { return; }
```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
}

```

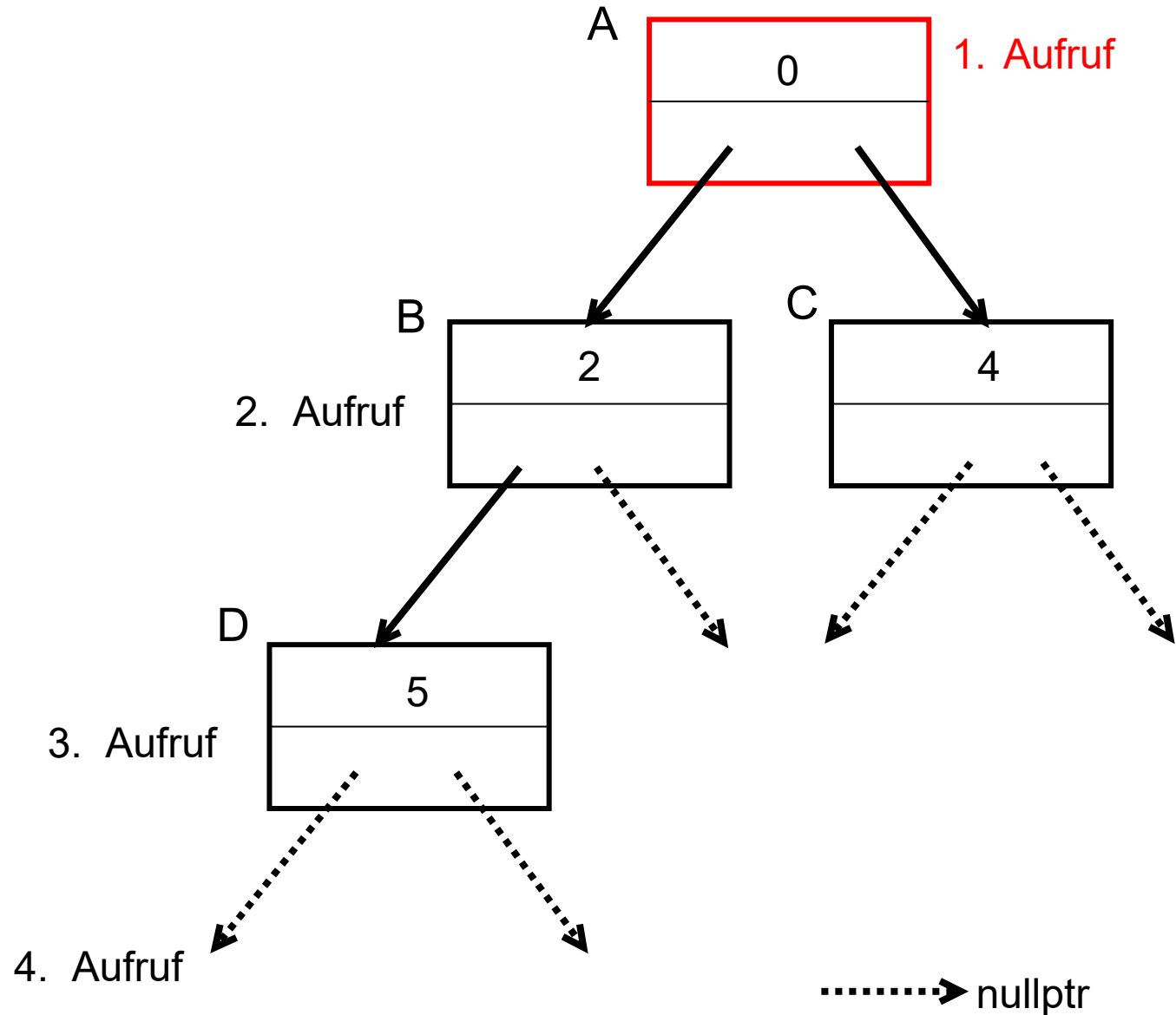
```
else { return; }

```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

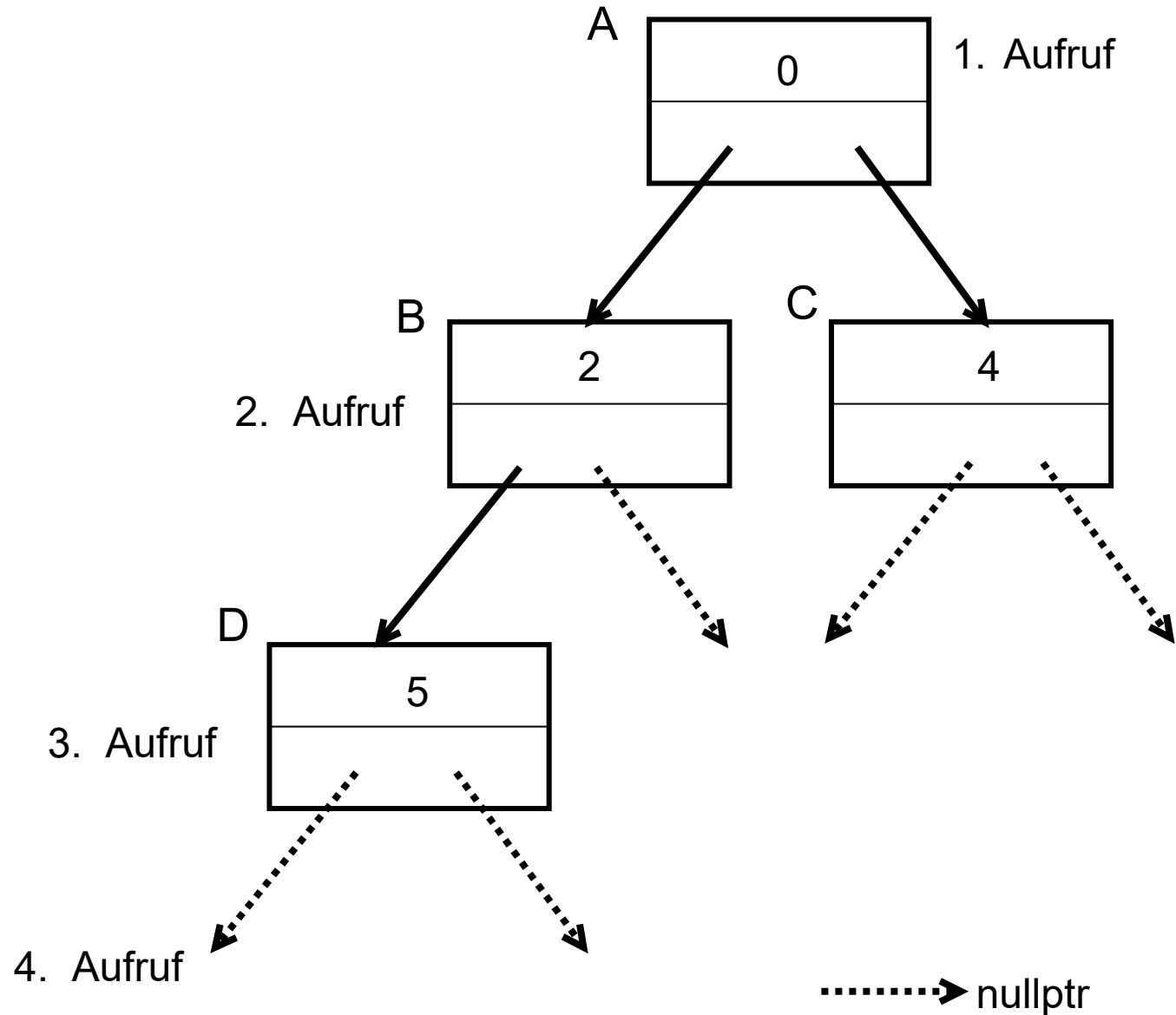
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

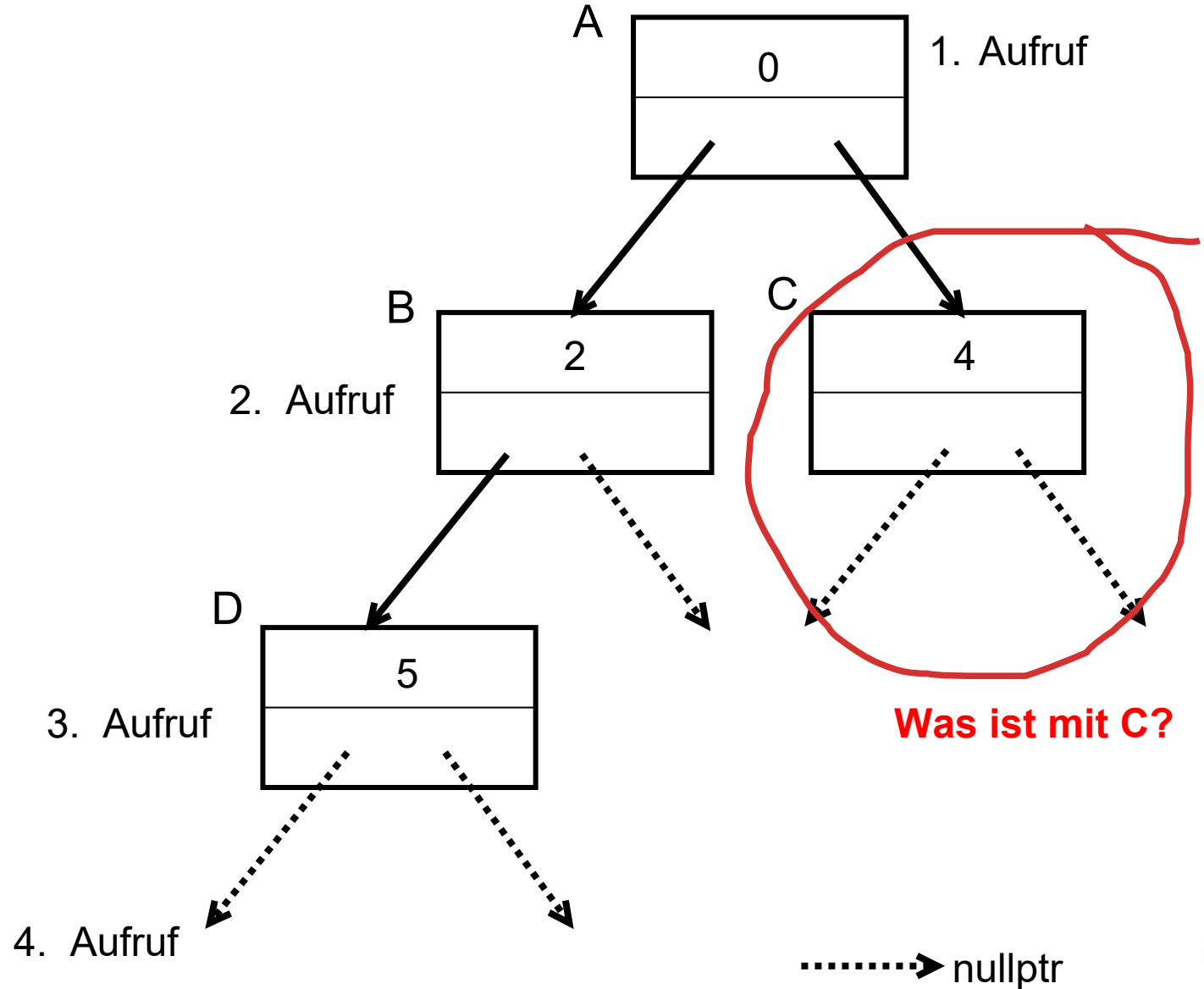
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
    }
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe

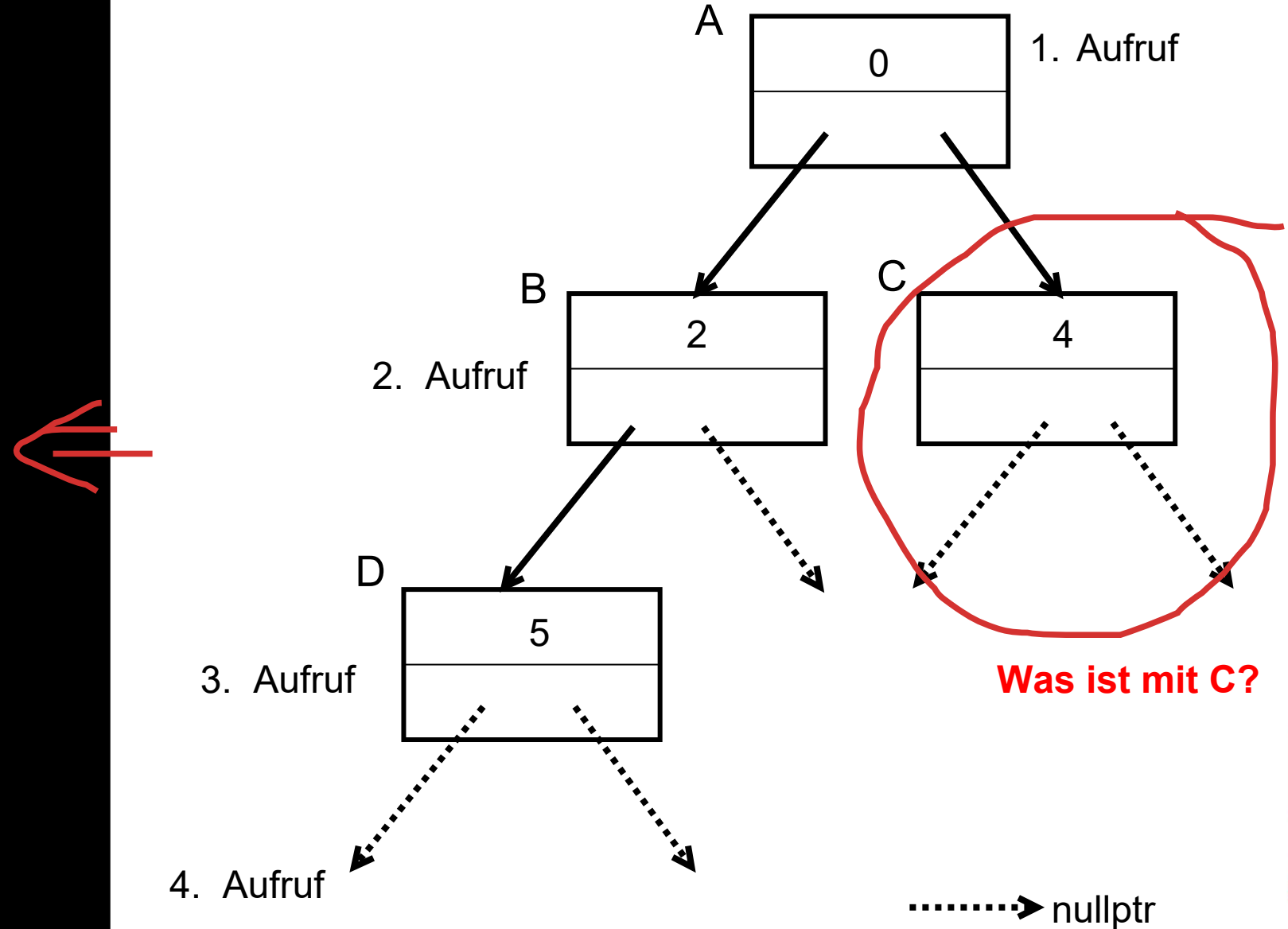


```
#include <iostream>
using namespace std;

struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

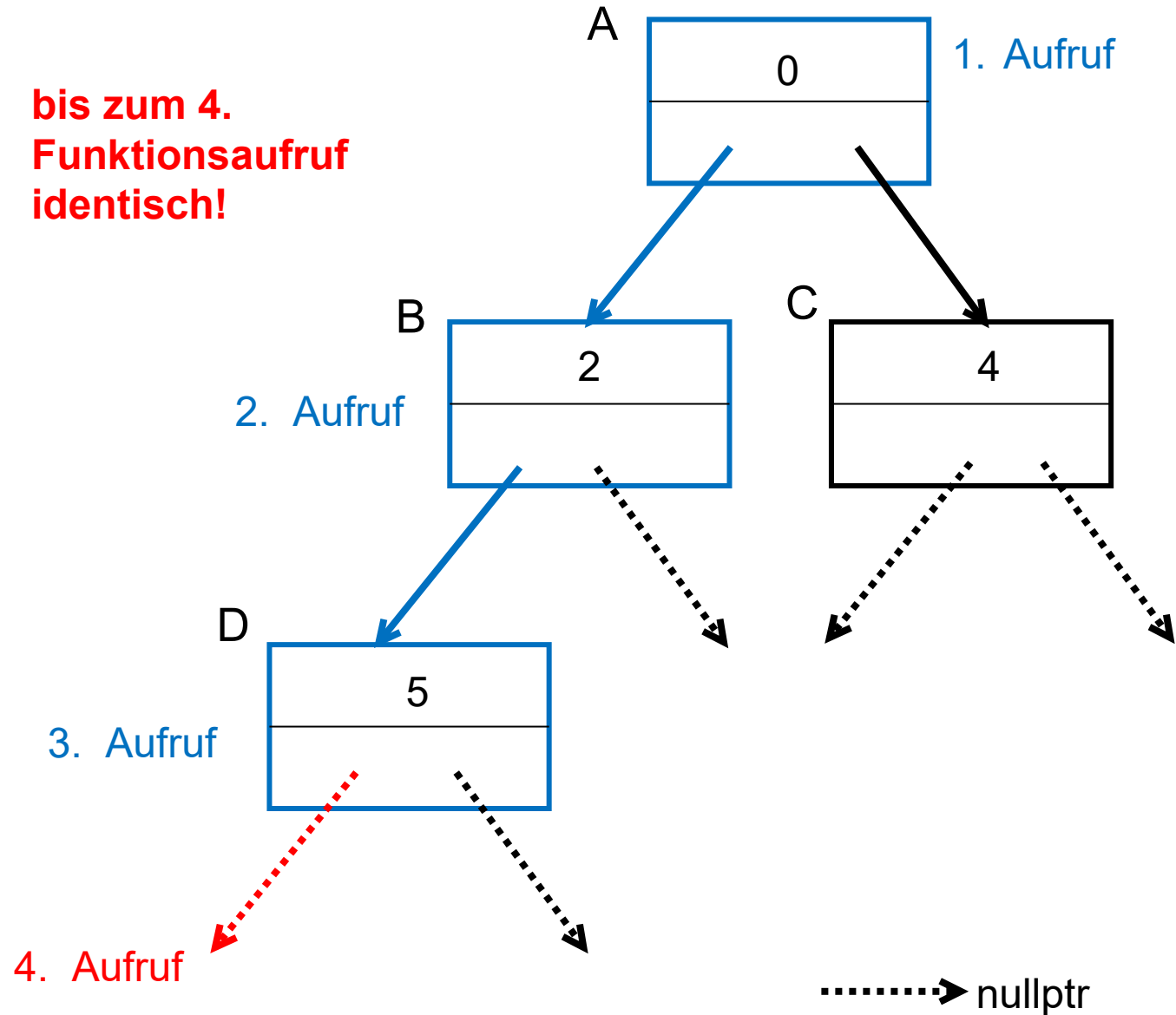
```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) { ←
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```

**bis zum 4.
Funktionsaufruf
identisch!**



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

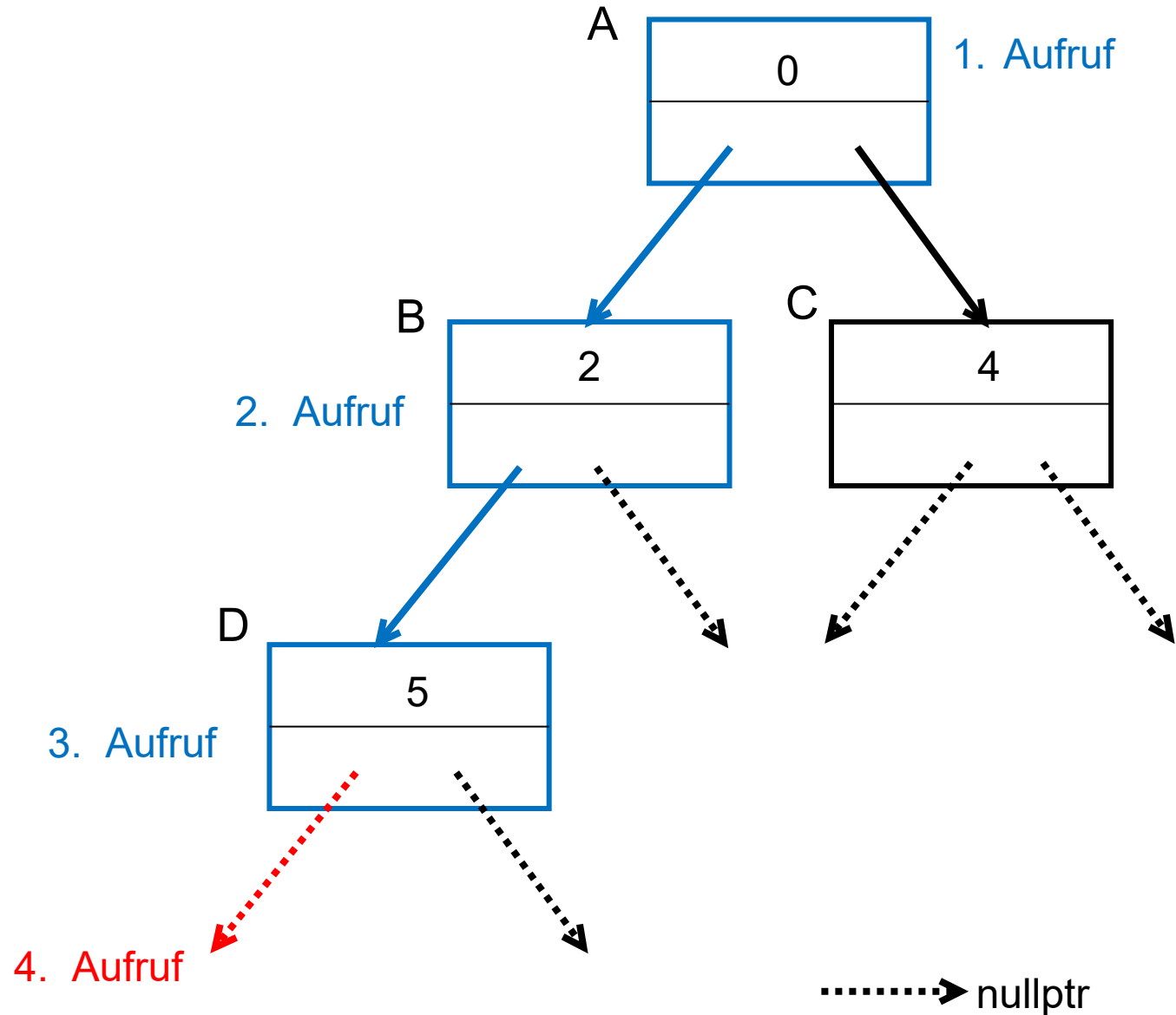
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; } ←
}
```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

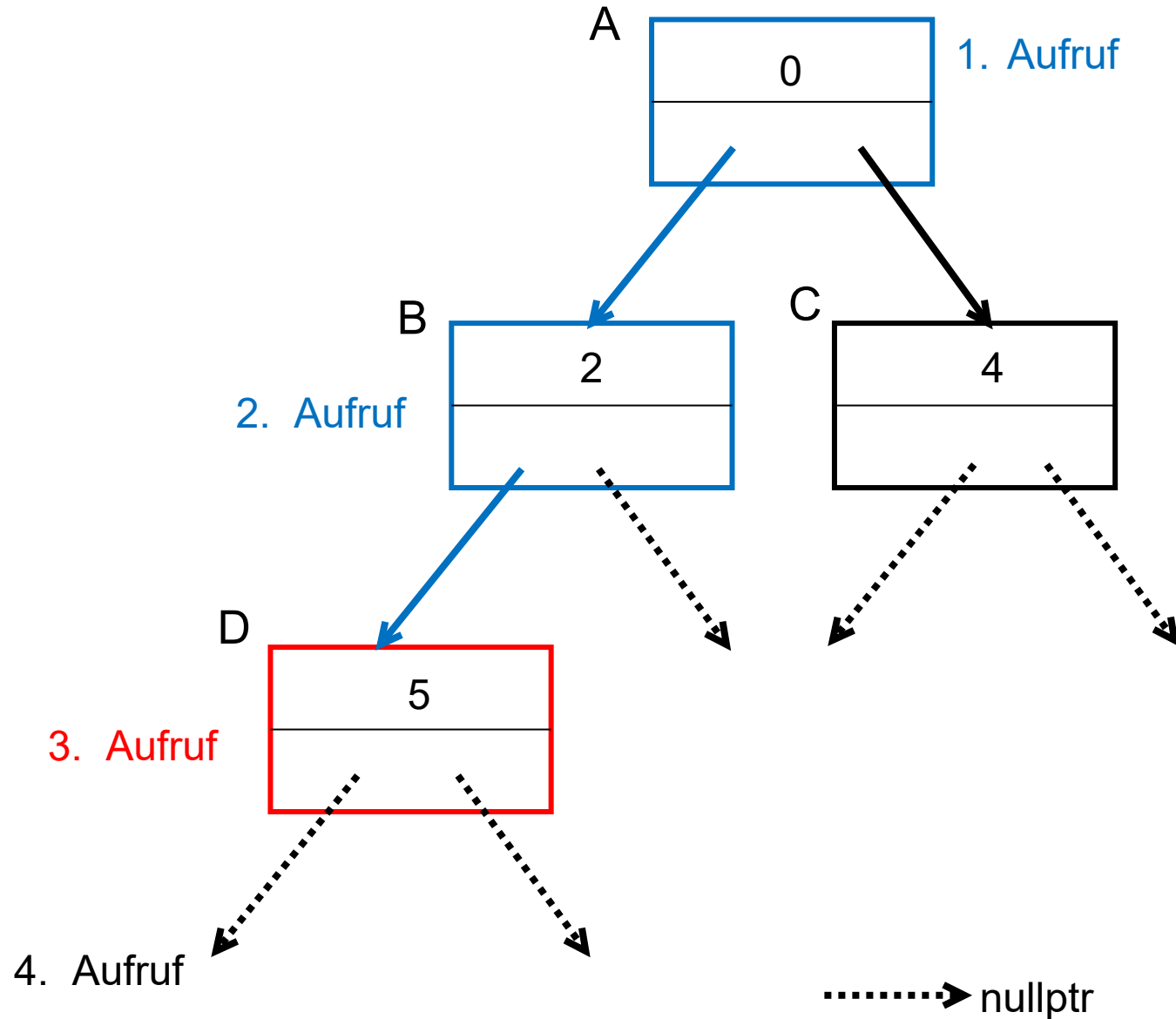
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

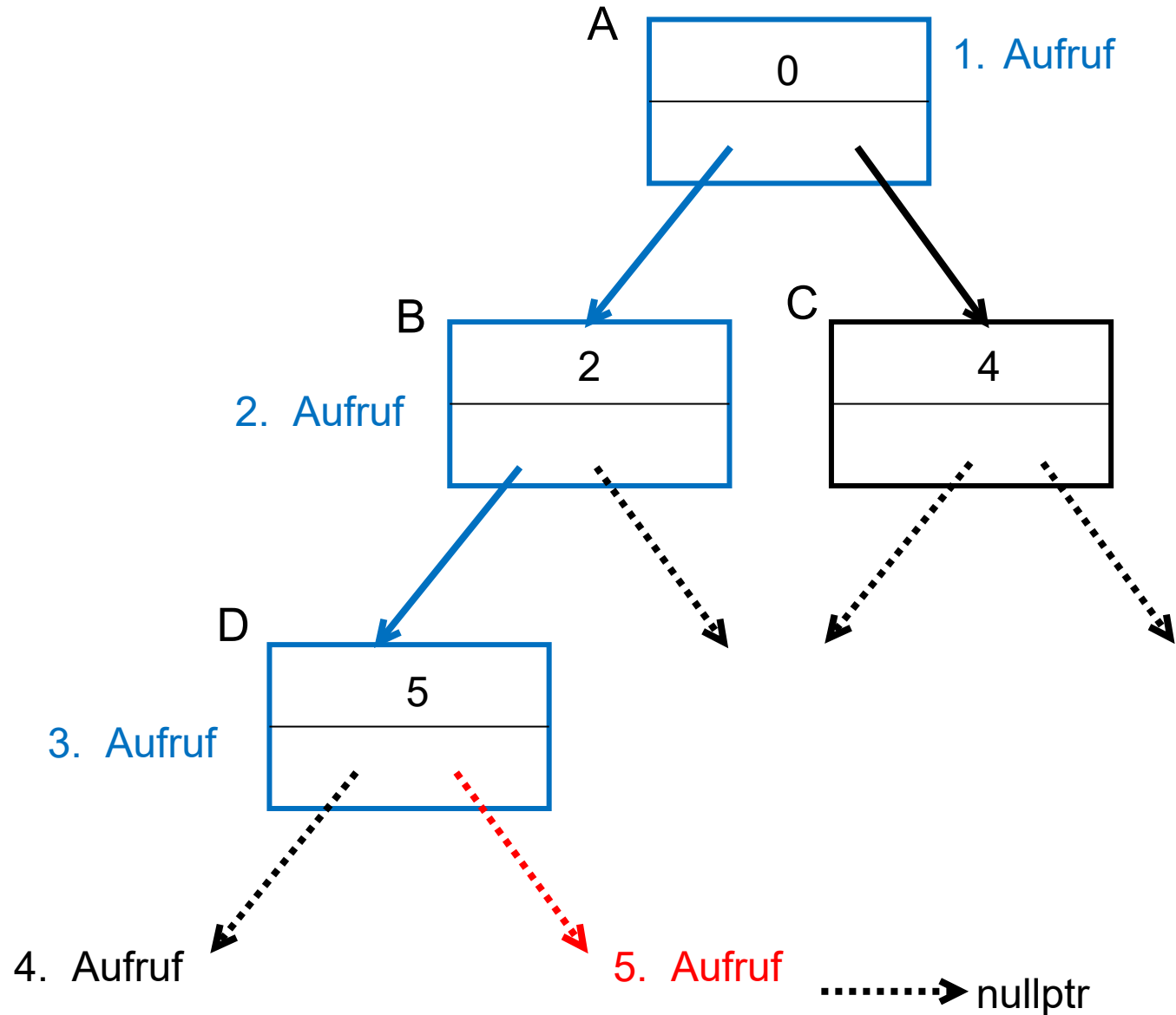
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);
    } else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

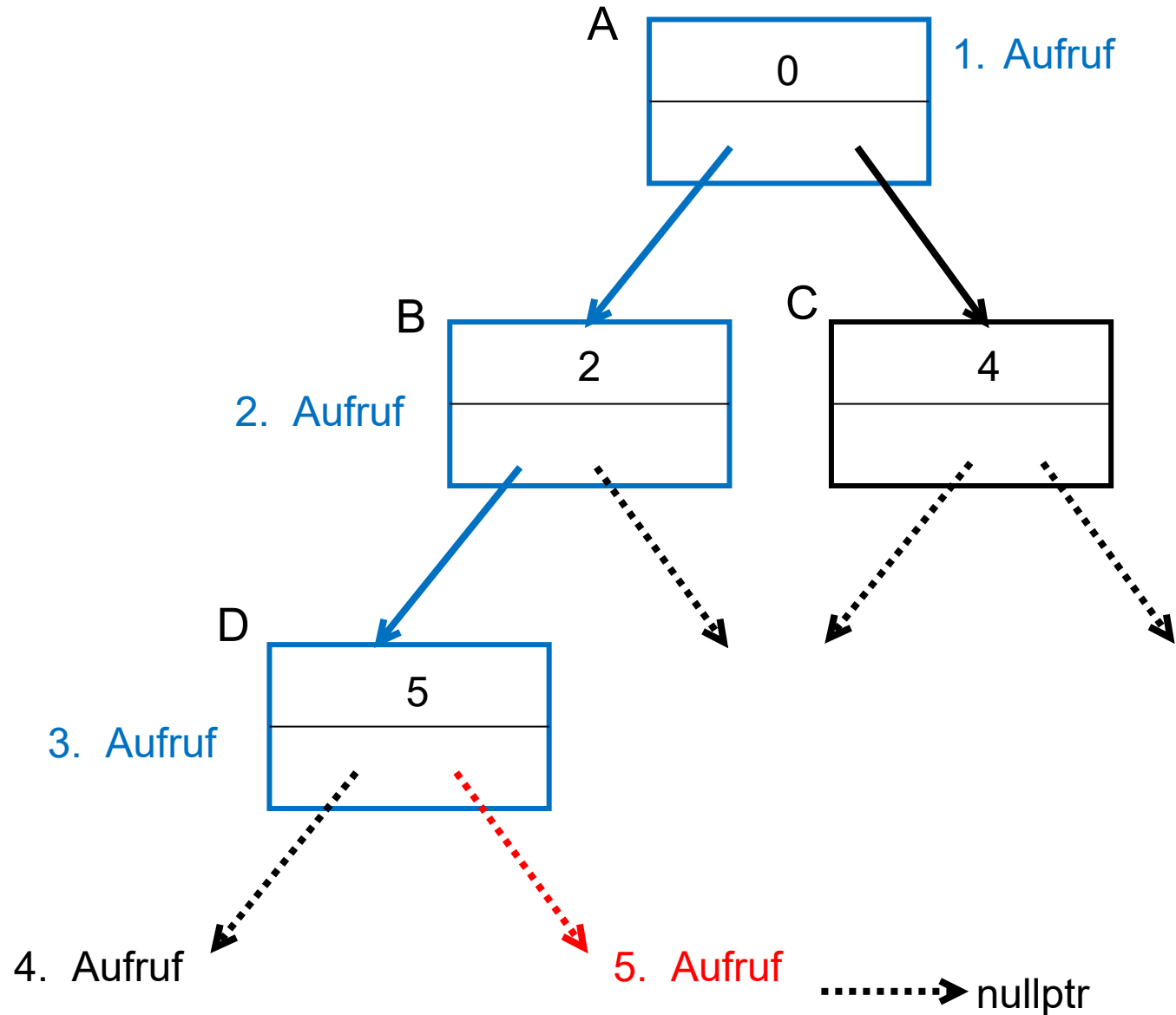
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

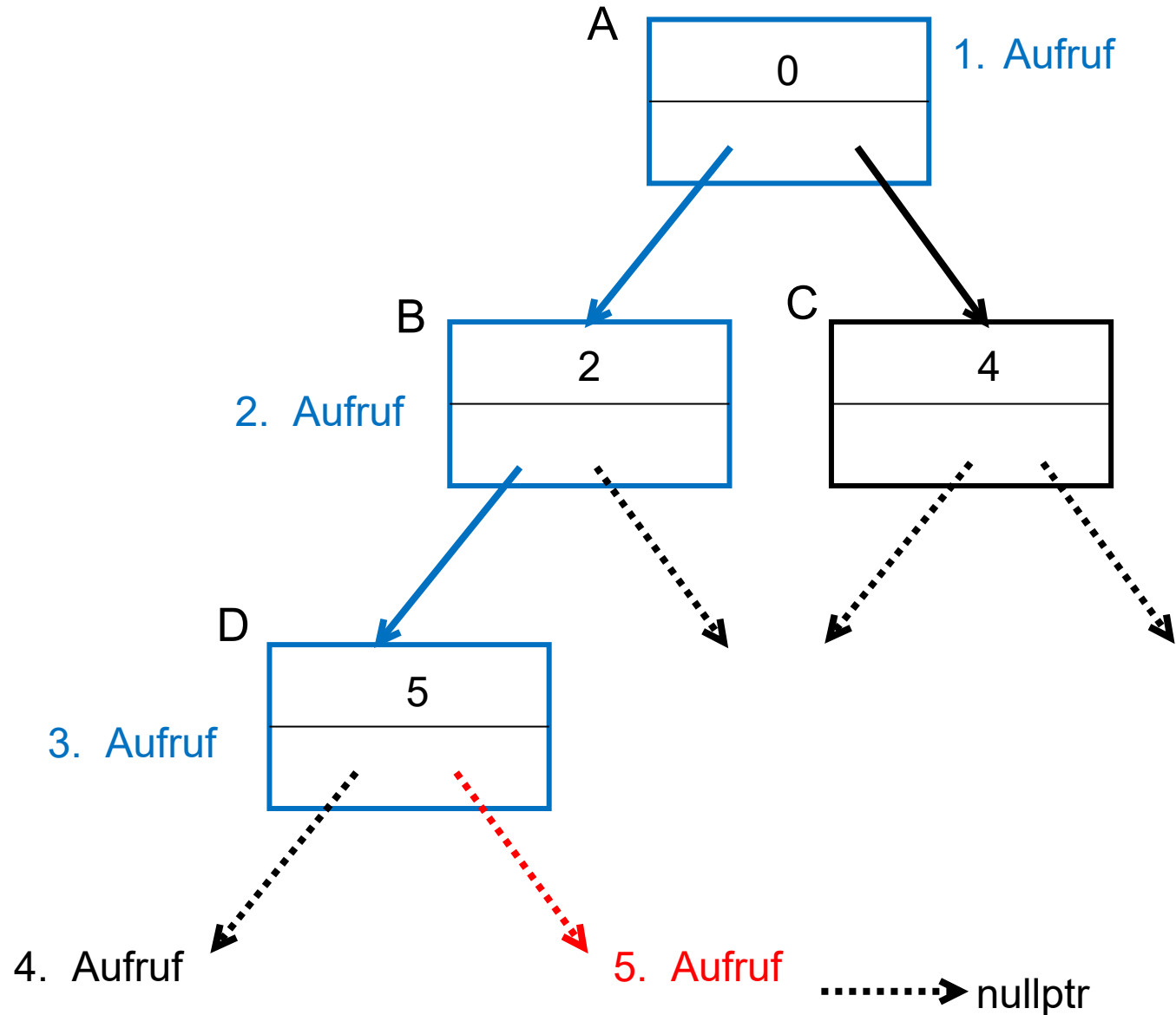
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; } ←
}
```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

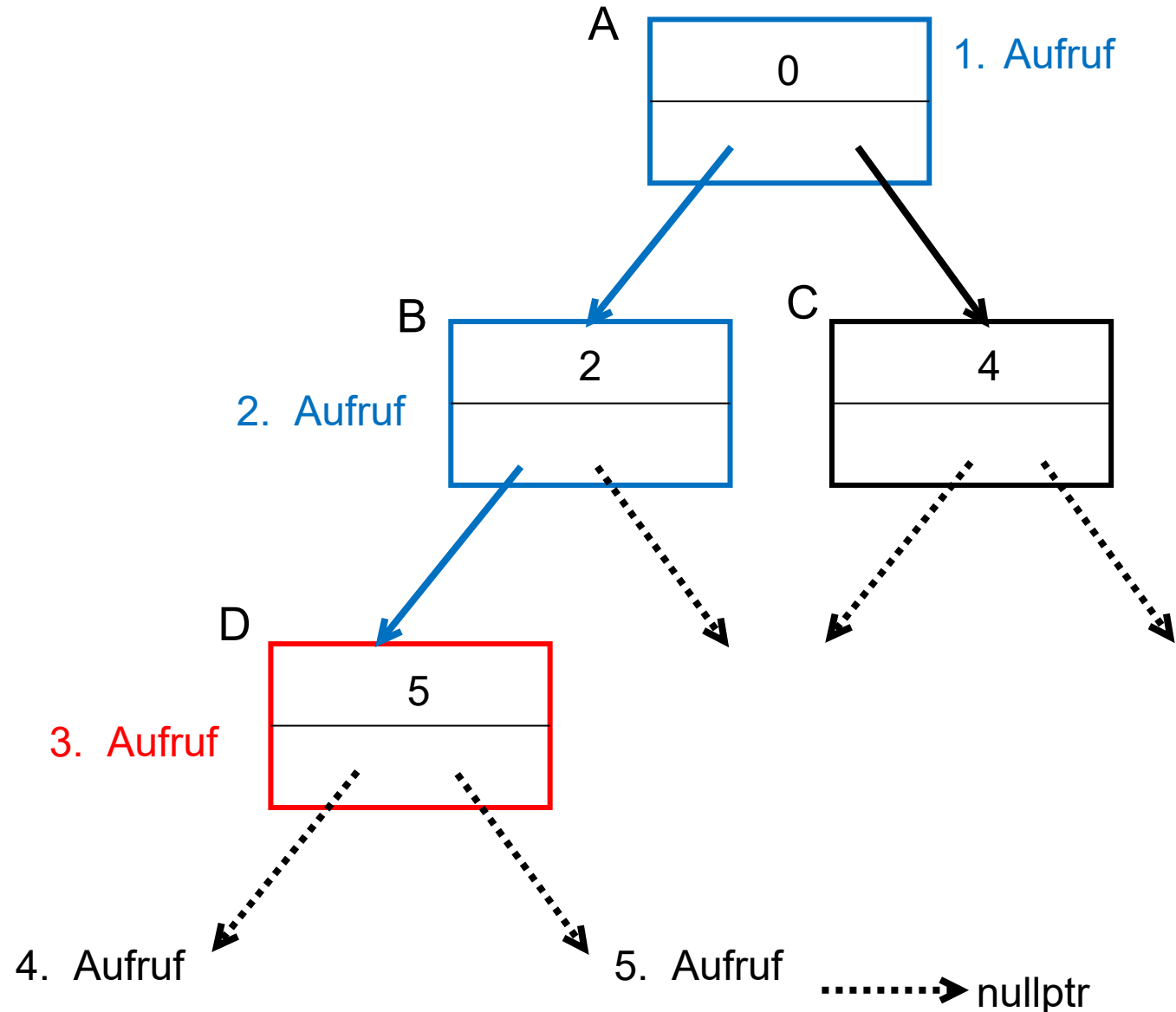
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe

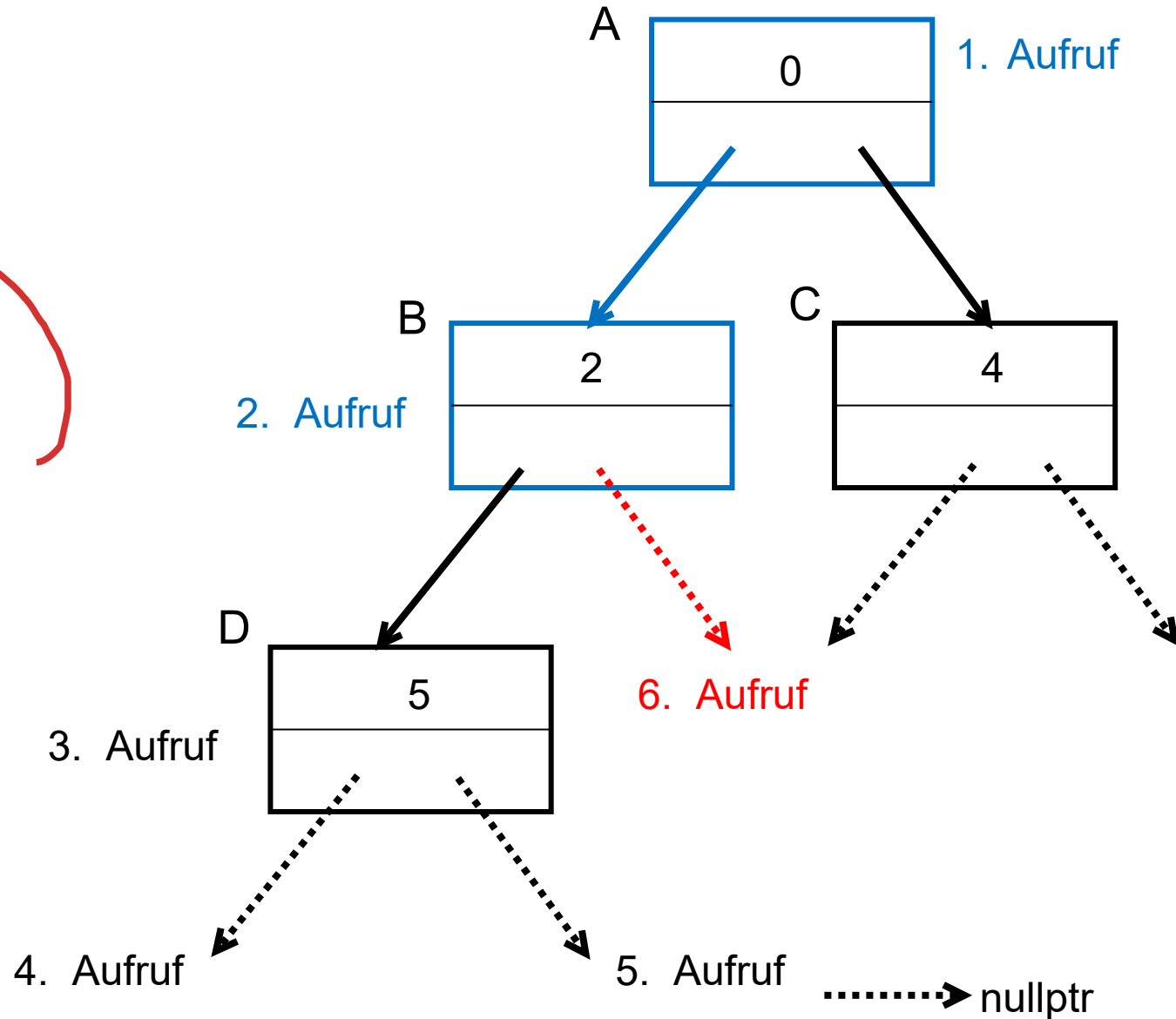


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

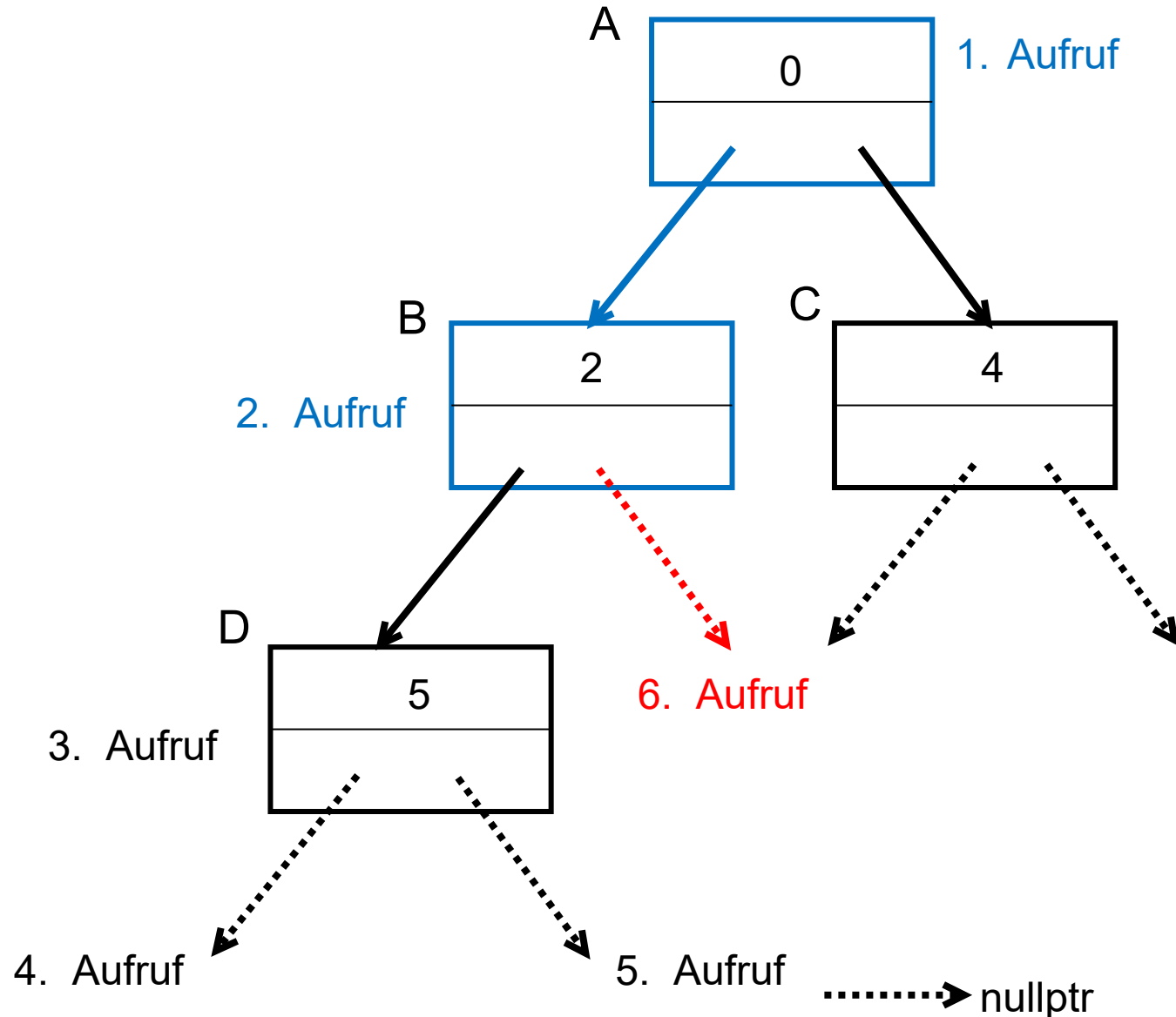
```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);
    }
    else { return; }
}

```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



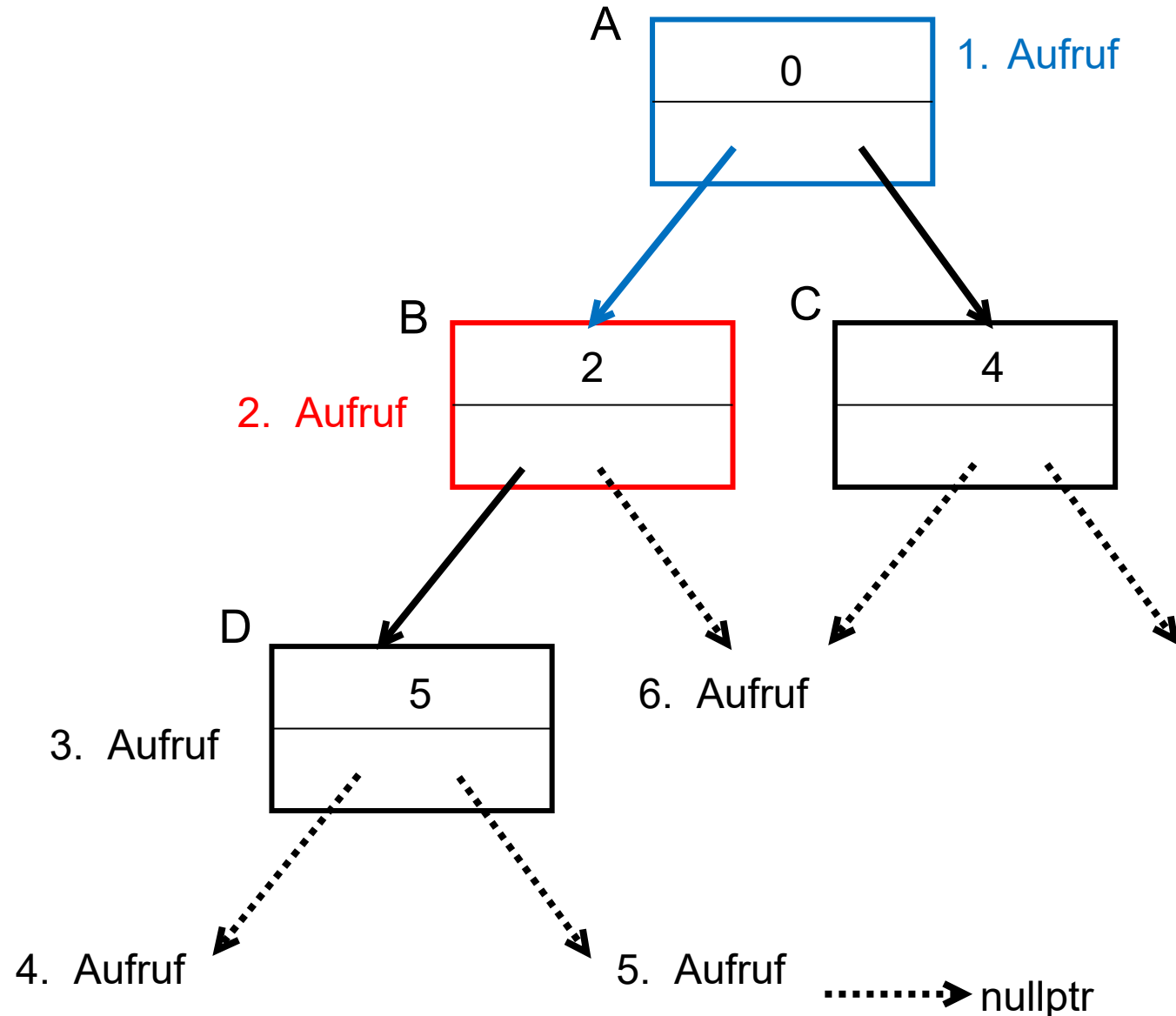
```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

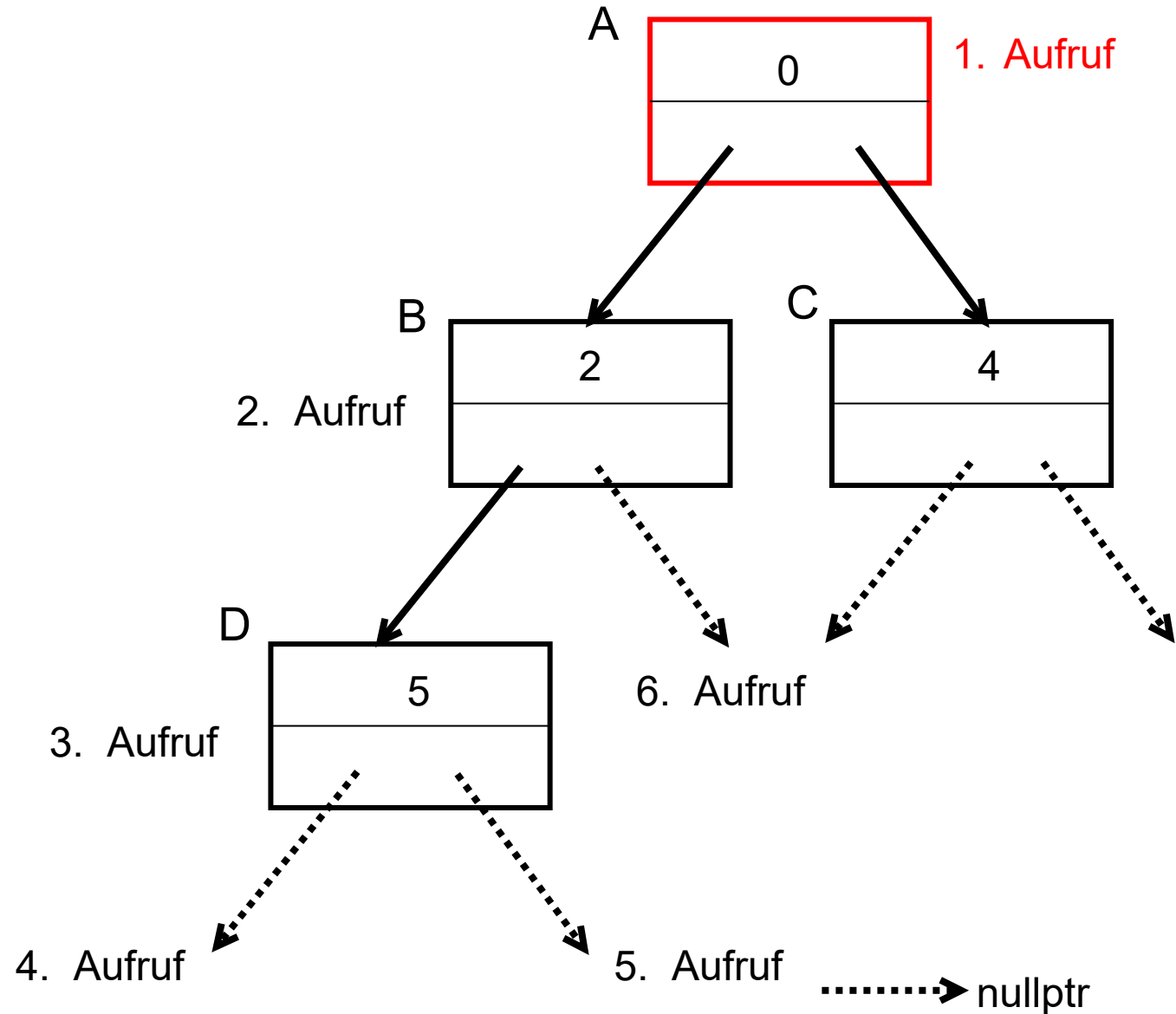
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);} ←
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe

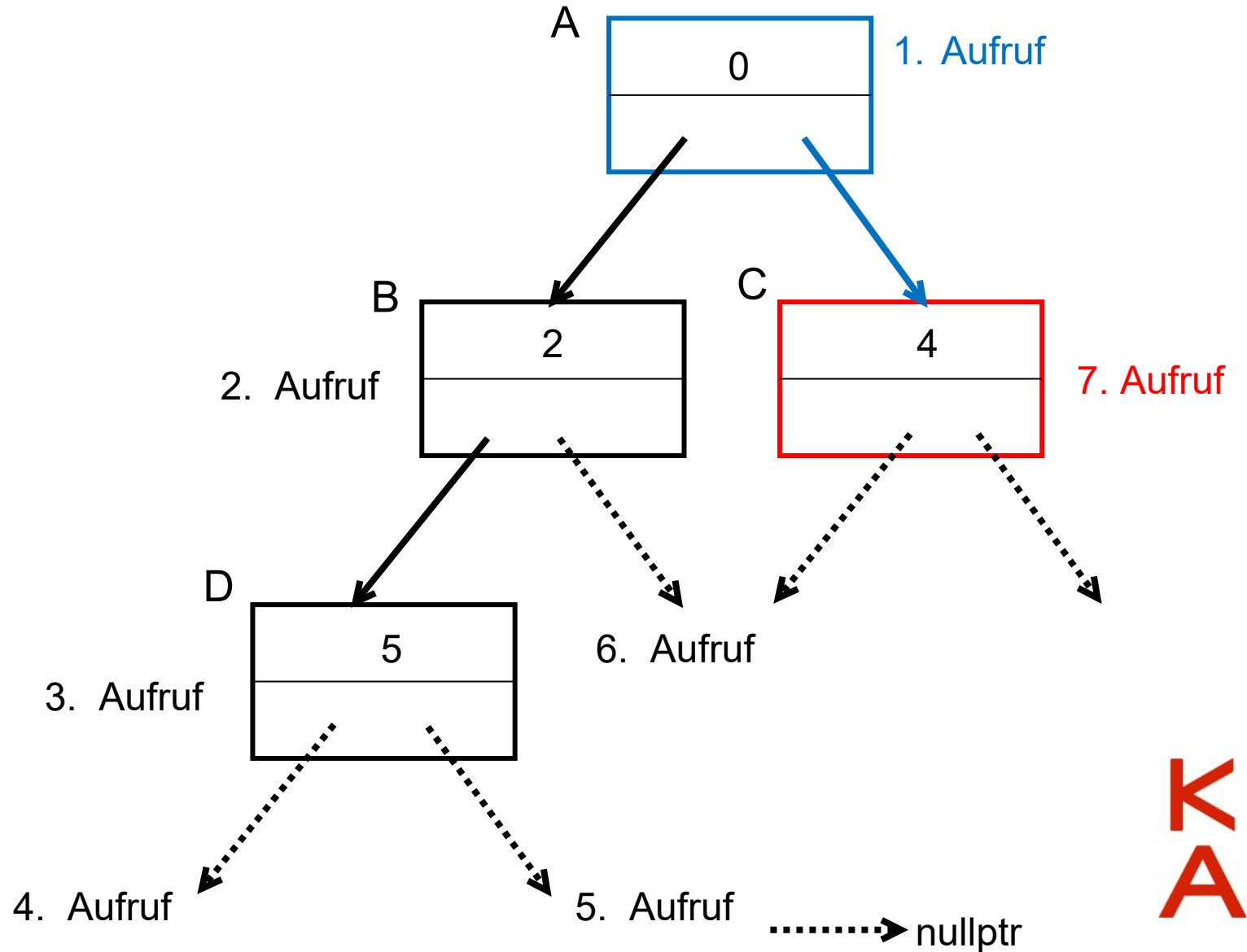


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

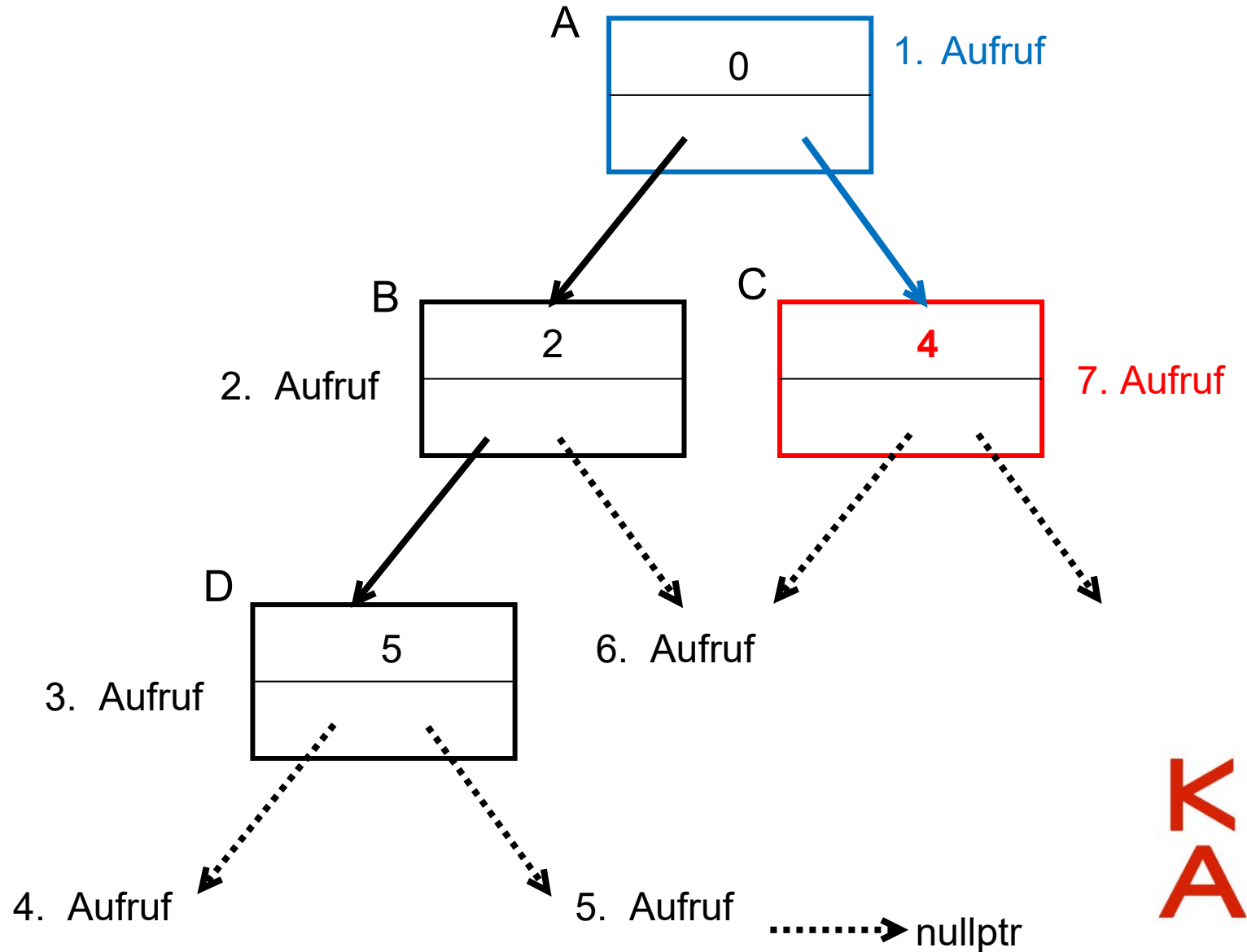
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);
    } else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe

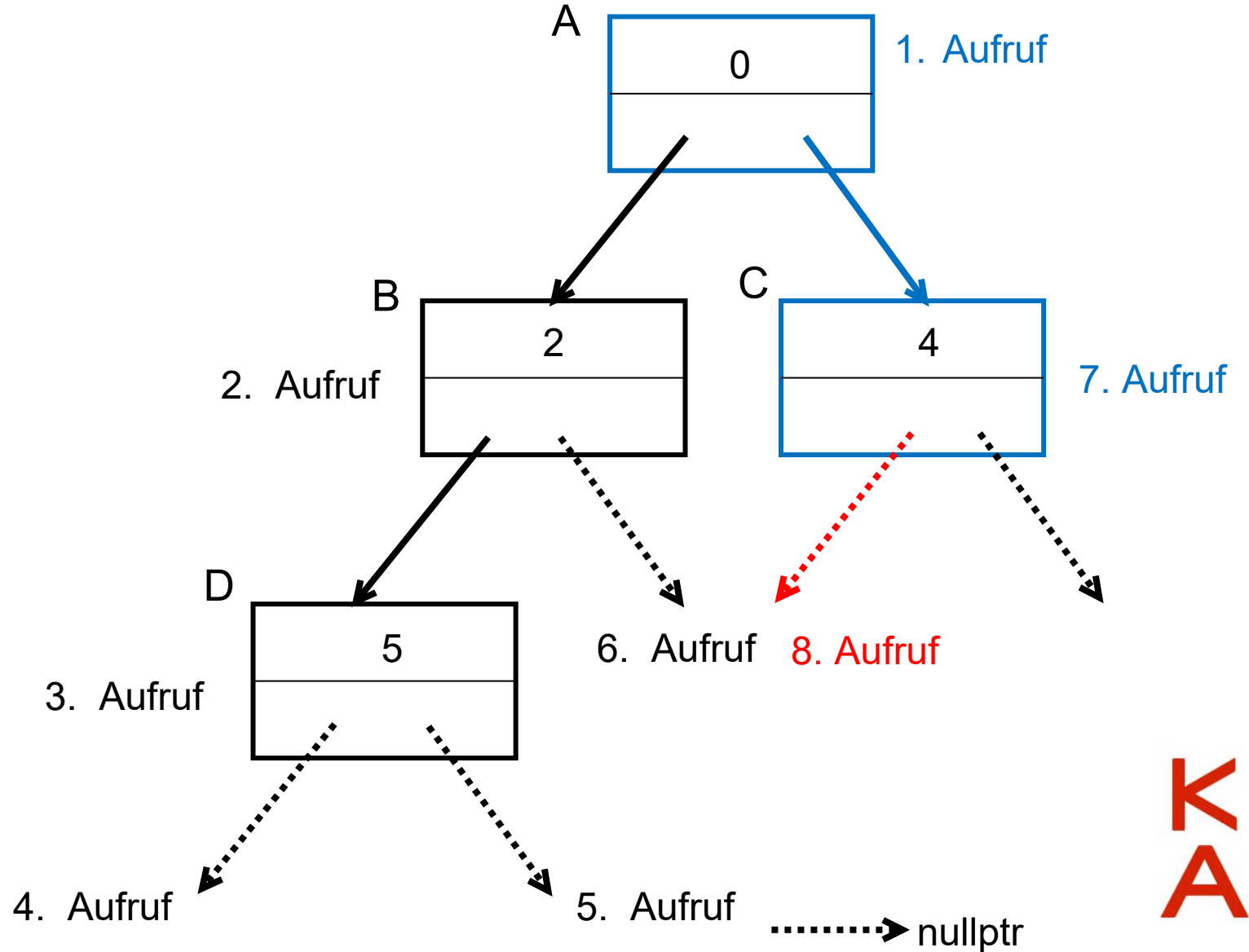


```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

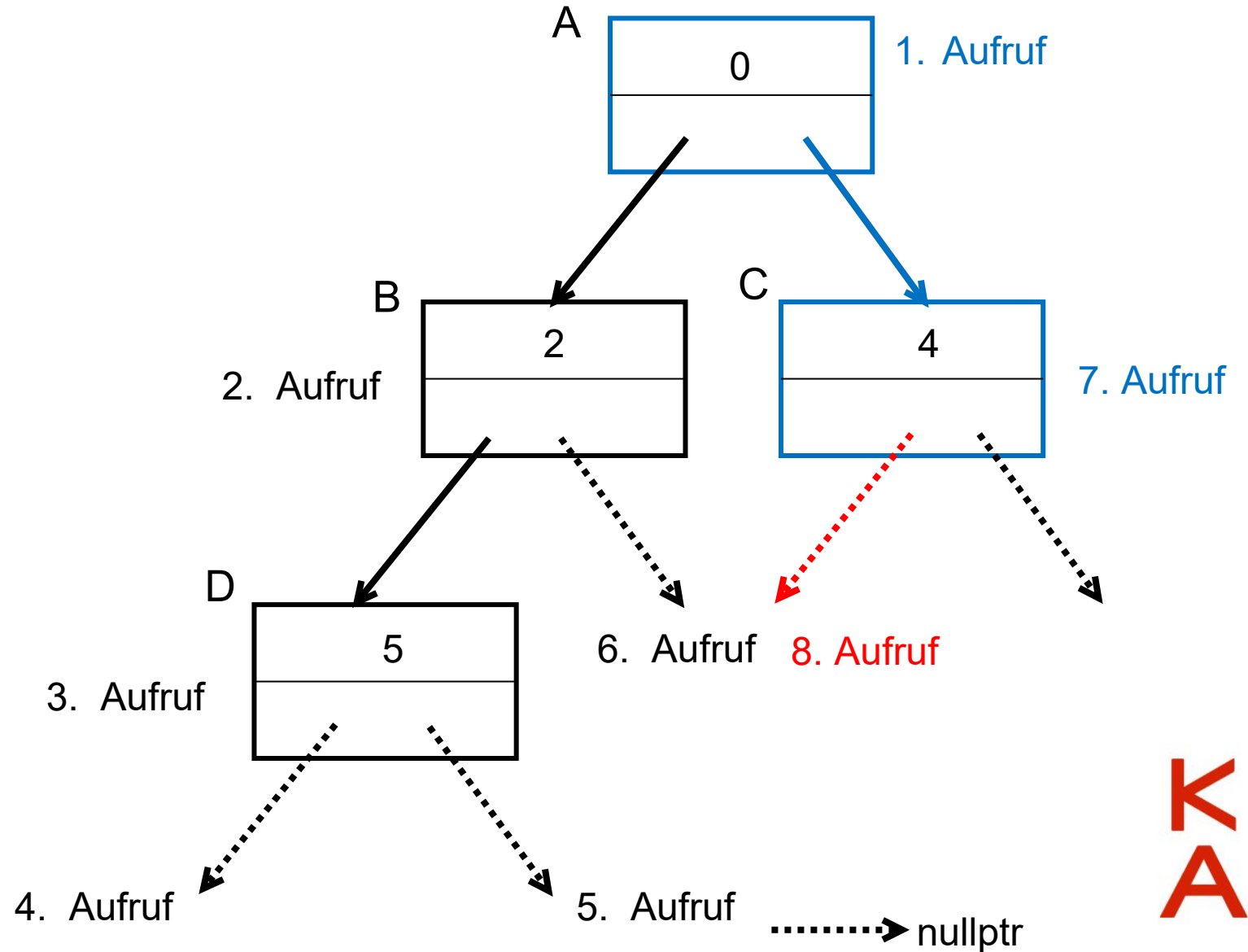
```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) { ←
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}

```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

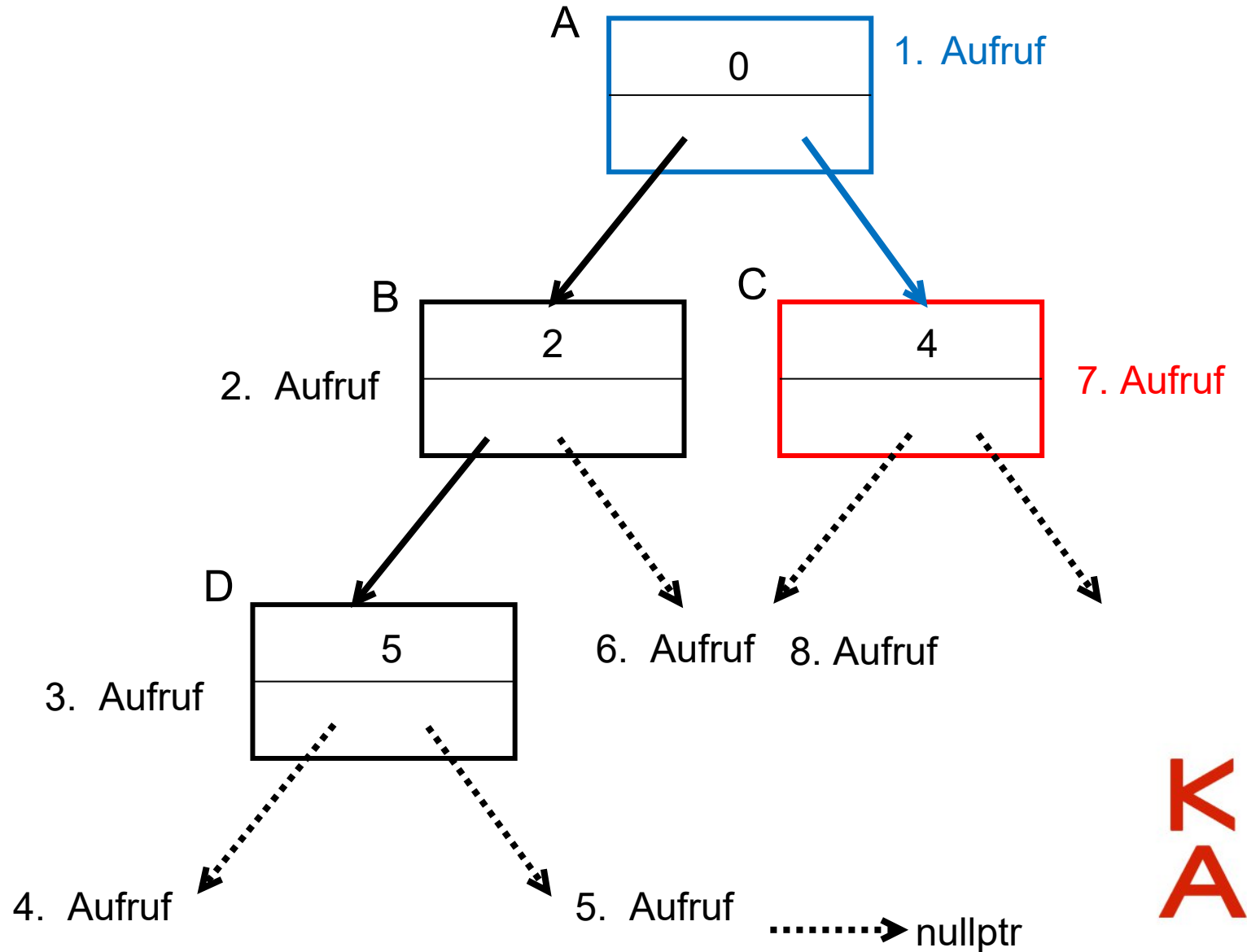
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



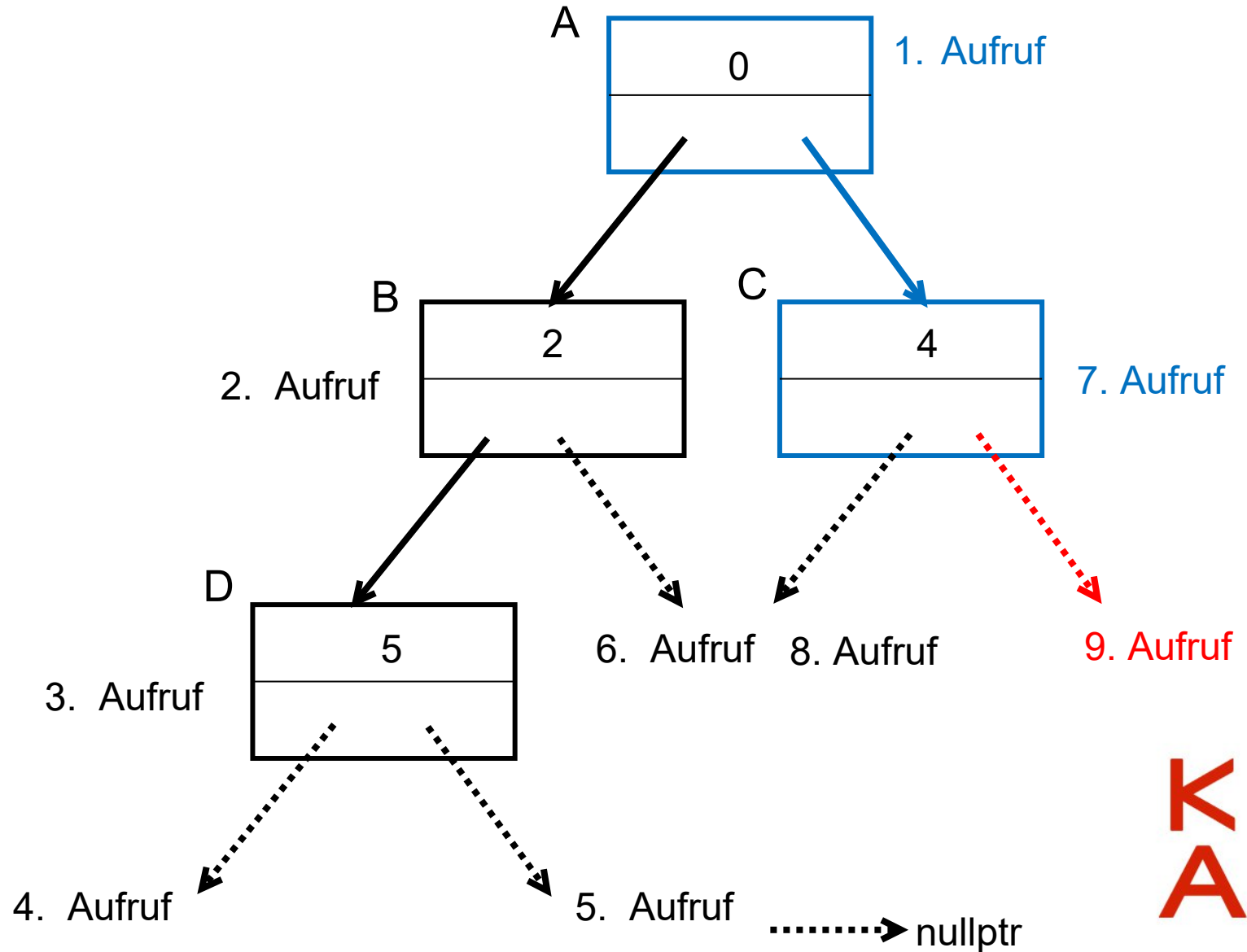
```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };
```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```




```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links   = &B;
    A.rechts  = &C;
    B.links   = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

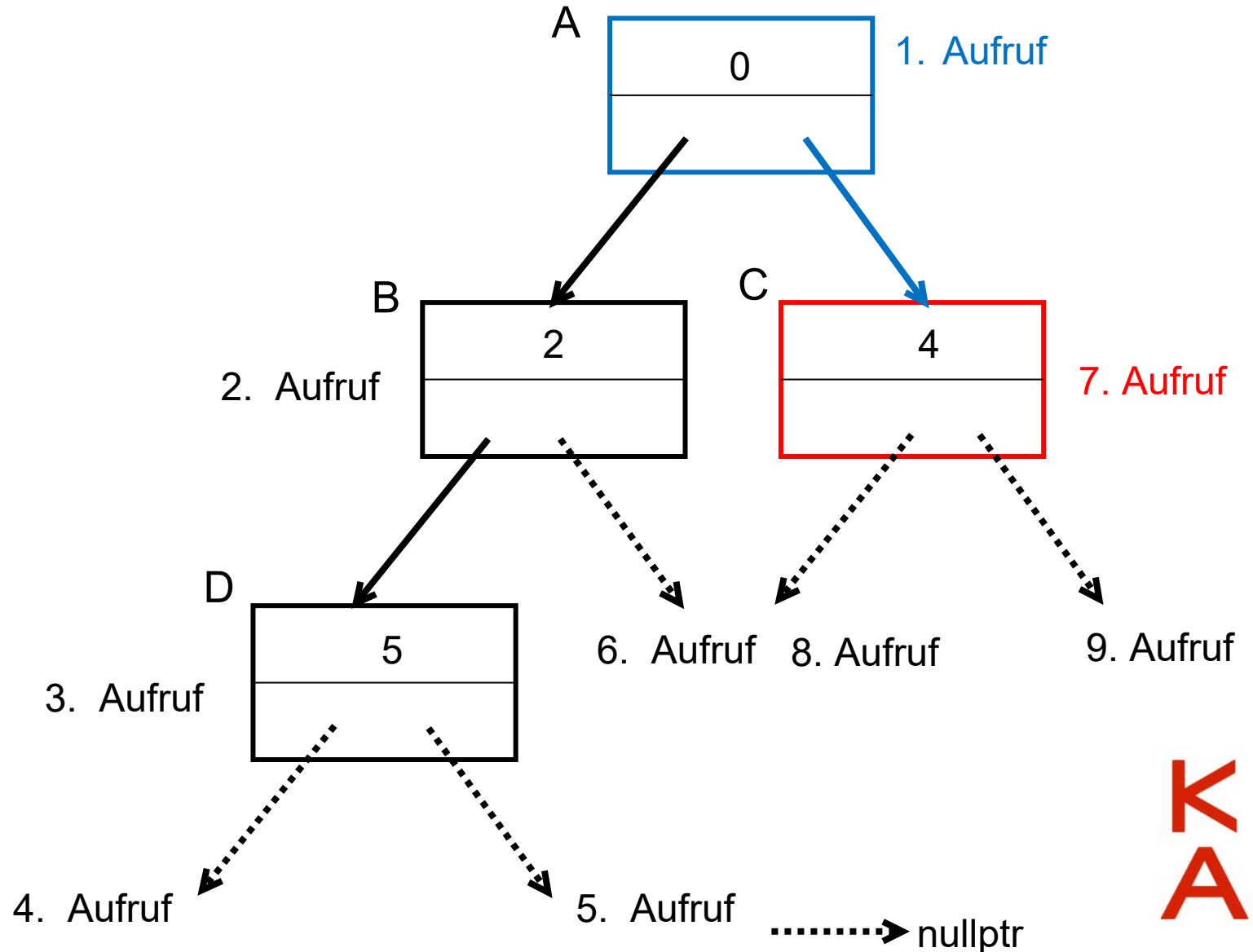
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);
    } else { return; }
}
```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

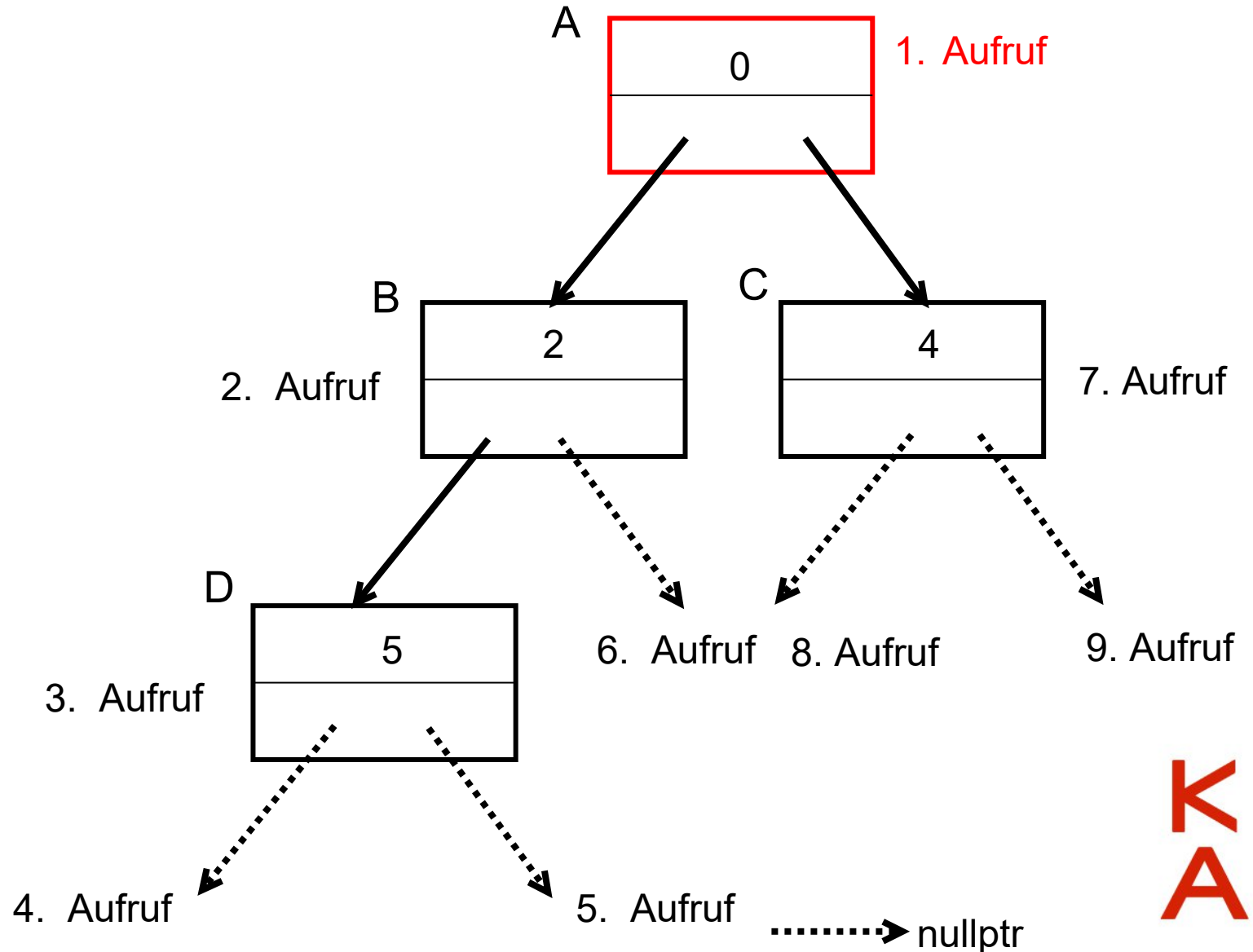
```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}
```

← Zurück zur aufrufenden Funktion!

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
}
```



Rekursive Baumsuche/-ausgabe



```
#include <iostream>
using namespace std;
```

```
struct Node {
    int daten = 0;
    Node* links = nullptr;
    Node* rechts = nullptr; };

```

```
void Baumausgabe(Node* wobinich) {
    if (wobinich != nullptr) {
        cout << wobinich->daten << endl;
        Baumausgabe(wobinich->links);
        Baumausgabe(wobinich->rechts);}
    else { return; }
}

```

```
int main() {
    Node A, B, C, D;
    A.daten = 0; B.daten = 2;
    C.daten = 4; D.daten = 5;
    A.links = &B;
    A.rechts = &C;
    B.links = &D;
    Baumausgabe(&A);
} ←
```

